



دانشکده مهندسی کامپیوتر

دانشگاه اصفهان

فاز اول پروژه‌های زبان‌های برنامه‌نویسی

استاد: دکتر آرش شفیعی

اعضای تیم:

زهره صفاری ۹۹۳۶۱۳۰۴۲

زهره امیری نژاد ۹۹۳۶۱۳۰۰۸

مهر والسادات نوحی ۹۹۳۶۱۳۰۶۱

پاییز ۱۴۰۲

فهرست

- ۱- مقدمه ۶
- ۱-۱ تاریخچه زبان برنامه نویسی چیست؟ ۶
- ۲-۱ این زبان برنامه نویسی در چه زمینه‌ها و حوزه‌هایی کاربرد دارد؟ ۲۰
- ۳-۱ این زبان برنامه نویسی در ابتدا به چه منظوری تهیه شده است؟ ۲۲
- ۴-۱ این زبان برنامه نویسی برای رفع چه مشکلاتی ابداع شده است؟ ۲۳
- ۵-۱ ابداع این زبان در جهت بهبود چه زبان‌هایی بوده است؟ ۲۳
- ۶-۱ این زبان در ابتدای ظهور خود چه مشکلاتی را رفع می‌کرده است؟ ۲۴
- ۷-۱ این زبان در مقایسه با زبان‌های دیگر و به خصوص زبان‌هایی که شبیه آن هستند چگونه ارزیابی می‌شود؟ ۲۵
- ۸-۱ این زبان چه ویژگی‌های خاصی دارد که آن را از زبان‌های مشابه آن متمایز می‌کند؟ ۲۷
- ۹-۱ این زبان را بر اساس خوانایی، قابلیت اطمینان، هزینه و سایر معیارها مانند قابلیت جابجایی چگونه می‌توان ارزیابی کرد؟ ۲۸
- ۱۰-۱ آیا برای پیاده سازی این زبان از کامپایلر استفاده شده است یا مفسر یا پیاده سازی ترکیبی؟ ۳۰
- ۱۰، ۱۱-۱ چه کامپایلرها یا مفسرهایی در حال حاضر برای این زبان وجود دارند و این کامپایلرها یا مفسرها توسط چه تیم‌هایی یا چه شرکت‌هایی تهیه شده‌اند؟ ۳۱
- ۲- نحو و معناشناسی ۳۲
- ۱-۲ فهرستی از کلمات کلیدی زبان برنامه نویسی انتخاب شده نوشته شود و شرحی در مورد کاربرد آن‌ها داده شود. ۳۲
- ۱-۱-۲ فهرستی از کلمات کلیدی زبان برنامه نویسی و شرحی در مورد کاربرد ۳۲

- ۲-۱-۲ حوه دقیق استفاده از بعضی کلمه‌های کلیدی پر کاربرد هم در ادامه آمده است(این بخش داخل
داک نبوده و صرفا به علت پرکاربرد بودن آورده شده است) ۳۷
- ۲-۲ گرامر برای زیرمجموعه‌ای از زبان برنامه‌نویسی انتخاب شده نوشته شود. ۴۱
- ۳-۲ یک برنامه در این زبان نوشته شود و درخت تجزیه برای آن رسم شود..... ۷۱
- ۴-۲ در مورد تقدم عملگرها و همچنین وابستگی عملگرها توضیحات لازم داده شود و در مورد نحوه توصیف
گرامر برای پیروی از تقدم‌های ذکر شده، توضیحاتی داده شود..... ۷۵
- ۱-۴-۲ تقدم عملگر ۷۵
- ۲-۴-۲ وابستگی عملگرها ۸۰
- ۳-۴-۲ توصیف گرامر برای پیروی از تقدم ۸۱
- ۵-۲ معناشناسی عملیاتی ۸۴
- ۱-۵-۲ ساختارهای تصمیم ۸۵
- ۲-۵-۲ عملگر شرطی ۹۰
- ۲-۵-۳ تکرار ۹۱
- ۲-۵-۴ تابع ۹۸
- ۳- متغیرها و نوع های داده‌ای ۱۰۱
- ۱-۳ در زبان جاوااسکریپت انقیاد نوع و مقدار چگونه و در چه زمانی انجام می‌شود؟ ۱۰۱
- ۱-۱-۳ در زبان جاوااسکریپت انقیاد نوع چگونه انجام می‌شود؟ ۱۰۱
- ۲-۱-۳ انقیاد نوع در چه زمانی انجام می‌شود؟ ۱۰۱
- ۳-۱-۳ انقیاد مقدار در جاوااسکریپت چگونه و چه زمانی انجام می‌شود؟ ۱۰۲

- ۲-۳ آیا تعاریف متغیرها ضمنی است یا صریح و یا هر دو نوع تعریف وجود دارد؟ با ذکر مثال توضیح داده شود. ۱۰۴
- ۳-۳ آیا در زبان انتخاب شده متغیرهای ایستا، پویا در پشته، پویا در هیپ به طور صریح، پویا در هیپ به طور ضمنی وجود دارند؟ با مثال توضیح داده شود. ۱۱۶
- ۳-۳-۱ آیا در جاوااسکریپت متغیرهای ایستا در پشته وجود دارد؟ توضیح دهید. ۱۱۶
- ۳-۳-۲ متغیرهای پویا در جاوااسکریپت چگونه پیاده سازی شده‌اند؟ ۱۱۷
- ۳-۳-۳ آیا در جاوااسکریپت متغیرهای پویا در پشته وجود دارد؟ توضیح دهید. ۱۲۰
- ۳-۳-۴ آیا در جاوااسکریپت متغیرهای پویا در هیپ به طور صریح وجود دارد؟ توضیح دهید. ۱۲۱
- ۳-۳-۵ آیا در جاوااسکریپت متغیرهای پویا در هیپ به طور ضمنی وجود دارد؟ توضیح دهید. ۱۲۳
- ۳-۶-۳ متغیرهای ایستا در جاوااسکریپت چگونه پیاده سازی شده‌اند؟ ۱۲۴
- ۳-۴ مقایسه سرعت تخصیص متغیرهای ایستا و پویا را در جاوااسکریپت ۱۲۷
- ۳-۵ آیا حوزه تعریف در زبان جاوااسکریپت ایستا است یا پویا؟ با ذکر مثال توضیح دهید. ۱۲۹
- ۳-۶ پیاده‌سازی حوزه تعریف پویا در جاوااسکریپت ۱۳۰
- ۳-۷ بلوک‌ها در جاوااسکریپت چگونه تعریف شده‌اند؟ ۱۳۲
- ۳-۸ آیا کلمات کلیدی ویژه‌ای برای اعمال تغییر در حوزه تعریف متغیرها در جاوااسکریپت وجود دارند؟ ۱۳۴
- ۳-۹ در مورد همه نوع‌های داده‌ای در زبان انتخاب شده توضیح داده شود. ۱۳۶
- ۳-۱۰ هر یک از این نوع‌های داده‌ای چگونه در حافظه تخصیص داده می‌شوند. هر یک از این نوع‌ها چگونه پیاده سازی شده‌اند؟ هر یک از نوع‌ها چه ویژگی‌هایی دارند و در چه مواردی استفاده می‌شوند؟ ۱۴۹
- ۳-۱۱ عبارات و عملگرها ۱۵۵

۱۲-۳ اگر لیست ها و رشته‌ها و آرایه‌های انجمنی در زبان تعریف شده اند، پیاده سازی آن‌ها چگونه است؟

۱۷۰

۱۷۰ ۱-۱۲-۳ کار با آرایه ها

۱۷۴ ۲-۱۲-۳ لیست ها

۱۷۵ ۳-۱۲-۳ رشته ها

۱۳-۳ اگر اشاره گرها و متغیرهای مرجع در زبان تعریف شده اند، پیاده سازی آن‌ها چگونه است؟ چه

۱۷۷ سازوکارهایی برای رفع مشکلات نشستی حافظه و اشاره گر معلق وجود دارد؟

۱۴-۳ آیا بازیافت کننده حافظه وجود دارد و در صورت وجود چگونه پیاده‌سازی شده است. در صورتی که

۱۷۸ ... بازیافت کننده حافظه وجود ندارد، زبان با یک زبان دیگر که بازیافت کننده حافظه دارد مقایسه شود.

۱۸۱ منابع:

۱- مقدمه

۱-۱ تاریخچه زبان برنامه نویسی چیست؟

- نگاه کلی به زبان:

زبان برنامه‌نویسی جاوااسکریپت (JavaScript) یک زبان برنامه‌نویسی رایج برای توسعه وب است. این زبان ابتدا توسط شرکت Netscape Communications Corporation در سال ۱۹۹۵ معرفی شد و در حال حاضر توسط سازمان استاندارد سازی ECMA International به عنوان استاندارد ECMA-262 تعریف شده است.

- روند پیدایش زبان:

در ابتدا، Brendan Eich که یک توسعه دهنده نرم‌افزار و مهندس کامپیوتر بود، به دنبال یک زبان برنامه‌نویسی سمت کاربر برای مرورگر Netscape Navigator می‌گشت. او با همکاری برنامه‌نویسانی از شرکت سان میکروسیستمز (Sun Microsystems) که آن زمان زبان جاوا را توسعه می‌دادند، به ساخت یک زبان جدید برای مرورگر پرداختند. این زبان ابتدا با نام "LiveScript" معرفی شد، اما به دلایل تجاری و با هدف بهره‌برداری از محبوبیت زبان جاوا، نام آن به "JavaScript" تغییر کرد.

- اولین نسخه رسمی از جاوااسکریپت:

اولین نسخه رسمی از جاوااسکریپت در سال ۱۹۹۷ منتشر شد. این زبان به طور ابتدایی برای کنترل رویدادها در صفحات وب استفاده می‌شد. با گذر زمان و توسعه وب، جاوااسکریپت نقش مهمی در ایجاد برنامه‌های پویا و تعاملی در وب ایفا کرد. استفاده از جاوااسکریپت باعث شد تا وب سایت‌ها قابلیت‌های پیچیده‌تری مانند اعتبارسنجی فرم‌ها، افزودن انیمیشن، بروزرسانی محتوا بدون بارگذاری مجدد صفحه و بسیاری از ویژگی‌های دیگر را بهبود بخشند.

- روند توسعه:

از آن به بعد، جاوااسکریپت به عنوان زبان اصلی برای برنامه‌نویسی مرورگرها و توسعه وب تبدیل شد. با پیشرفت تکنولوژی و استفاده از جاوااسکریپت در سمت سرویس‌دهنده با استفاده از (Node.js)، این زبان امکان توسعه برنامه‌های وب، برنامه‌های موبایل، اپلیکیشن‌های دسکتاپ و حتی برنامه‌های رباتیک را فراهم کرد.

- نسخه‌هایی از جاوااسکریپت تاکنون منتشر شده:

تاکنون، چندین نسخه از زبان برنامه‌نویسی جاوااسکریپت منتشر شده است. در زیر، تاریخ انتشار و نام نسخه‌های مهم جاوااسکریپت آورده شده است:

۱. نسخه اولیه (1996) JavaScript 1.0

- این نسخه اولیه برای مرورگر Netscape Navigator منتشر شد.

به طور کلی JavaScript 1.0 یکی از نسخه‌های اولیه زبان جاوااسکریپت است که در سال ۱۹۹۶ معرفی شد. این نسخه به عنوان نسخه اولیه از زبان جاوااسکریپت، قابلیت‌های پایه‌ای این زبان را شامل می‌شد. در آن زمان، JavaScript ابزاری برای ایجاد تعامل در صفحات وب بود.

بعضی از ویژگی‌های کلیدی JavaScript 1.0 عبارتند از:

- پشتیبانی از مفهوم متغیرها و عملگرها برای انجام محاسبات ساده.
- قابلیت کنترل جریان برنامه با استفاده از شرطی‌ها (if-else) و حلقه‌ها (for)، (while).
- امکان تعریف توابع و استفاده از آن‌ها برای تجزیه‌بندی وظایف و تکرار استفاده‌های بعدی.
- قابلیت کنترل رویدادها (event) و واکنش به رویدادهای کاربر.
- پشتیبانی محدود از اشیاء (Objects) برای ساختاردهی و ترتیب‌بندی کد.
- امکان کار با رشته‌ها (Strings) و آرایه‌ها (Arrays).
- قابلیت دسترسی به المان‌ها و ویژگی‌های صفحات HTML با استفاده از مدل شیء‌گرا DOM^۱

از آن زمان تا به حال، زبان جاوااسکریپت تحت عنوان ECMAScript (استانداردی که بر اساس آن زبان جاوااسکریپت توسعه می‌یابد)، تغییرات و بهبودهای چشمگیری را تجربه کرده است. امروزه، JavaScript

¹ Document Object Model

به یک زبان برنامه‌نویسی قدرتمند و گسترده تبدیل شده است که در طی سال‌ها به طور فزاینده بهبود یافته و به عنوان یکی از اصلی‌ترین زبان‌های برنامه‌نویسی وب به شمار می‌رود.

۲. JavaScript 1.1 (1996)

این نسخه شامل تغییرات و بهبودهایی در مقایسه با نسخه ۱.۰ بود.

ورژن ۱.۱ از JavaScript در سال ۱۹۹۶ منتشر شد و برخی از تغییرات و بهبودهای مهم را به زبان اضافه کرد. در زیر به برخی از این تغییرات اشاره می‌کنیم:

- مدیریت خطا: ورژن ۱.۱ از JavaScript، ساختار try-catch-finally را به زبان اضافه کرد. این ساختار به برنامه‌نویسان اجازه می‌دهد که کدهای خود را در یک بلوک try قرار داده و در صورت بروز خطا، آن را با استفاده از بلوک catch کنترل کنند. بلوک finally همچنین به برنامه‌نویسان اجازه می‌دهد که قطعه کدی را اجرا کنند که همیشه بعد از بلوک try-catch اجرا شود، بدون در نظر گرفتن اینکه آیا خطایی رخ می‌دهد یا نه.
- افزودن متغیرهای صفحه وب: JavaScript 1.1 امکان استفاده از متغیرهای صفحه وب را به زبان اضافه کرد. این متغیرها به برنامه‌نویسان اجازه می‌دهند تا به اطلاعات مربوط به صفحه وب فعلی، مانند آدرس URL و عنوان صفحه، دسترسی پیدا کنند و آن‌ها را در برنامه خود استفاده کنند.
- معرفی متغیرهای بازیابی پذیر: ورژن ۱.۱ از JavaScript امکان استفاده از متغیرهای بازیابی پذیر را فراهم کرد. این متغیرها با استفاده از کلیدواژه "var" تعریف می‌شوند و در محدوده‌های مختلف قابل دسترسی هستند. به عنوان مثال، تعریف یک متغیر بازیابی پذیر در یک تابع، متغیر را در دسترس تمامی بخش‌های تابع قرار می‌دهد.

این تغییرات فقط برخی از بهبودهای موجود در ورژن ۱.۱ از JavaScript هستند. البته، این نسخه نسبتاً قدیمی است و بسیاری از تغییرات و بهبودهای بعدی در نسخه‌های جدیدتر JavaScript اعمال شده‌اند.

۳. JavaScript 1.2 (1997)

این نسخه شامل قابلیت‌های جدیدی مانند قابلیت کنترل پنجره مرورگر، کنترل پلاگین‌ها و قابلیت‌های صوتی بود.

این نسخه، یک نسخه از زبان برنامه‌نویسی JavaScript بود که در سال ۱۹۹۷ منتشر شد. این نسخه یک سری از تغییرات و اضافات را به JavaScript ارائه کرد. در زیر به برخی از ویژگی‌های مهم JavaScript 1.2 اشاره می‌کنیم:

- شیء‌گرایی: JavaScript 1.2 شیء‌گرایی را به زبان اضافه کرد. این به برنامه‌نویسان اجازه می‌دهد که تعریف کلاس‌ها و اشیاء را با استفاده از کلیدواژه‌هایی مانند `class` و `new` انجام دهند. این قابلیت امکانات مهمی را مانند وراثت، انتزاع و پلی‌مورفیسم را در برنامه‌نویسی JavaScript فراهم می‌کند.
- حلقه‌ها: JavaScript 1.2 از حلقه‌های `for...in` پشتیبانی می‌کند که به برنامه‌نویسان اجازه می‌دهد بر روی ویژگی‌های یک شیء یا اعضای یک آرایه حلقه بزنند. این حلقه به صورت خودکار از تمام ویژگی‌های موجود در یک شیء یا اعضای یک آرایه عبور می‌کند.
- تعریف توابع: JavaScript 1.2 به برنامه‌نویسان اجازه می‌دهد تا توابع را به عنوان متغیرهای قابل انتقال تعریف کنند. می‌توان توابع را به عنوان آرگومان به توابع دیگر منتقل کرد یا توابع را در داخل توابع دیگر تعریف کرد.

- مدیریت رویداد: JavaScript 1.2 بهبودهایی در مدیریت رویداد ارائه کرد. این شامل اضافه کردن رویدادها به عناصر صفحه (مانند کلیک موس، نمایش، پنهان شدن و ...) و مدیریت آنها با استفاده از توابع رویداد است.

این تغییرات تنها بخشی از بهبودها و اضافات موجود در JavaScript 1.2 هستند. این نسخه هم نسبتاً قدیمی است و تغییرات بسیاری در نسخه‌های جدیدتر آن اعمال شده است.

۴. JavaScript 1.3 (1998)

در این نسخه، پشتیبانی از قابلیت‌های مثل drag-and-drop و تعریف کلاس در زبان جاوااسکریپت بهبود یافت.

JavaScript 1.3، که در سال ۱۹۹۸ منتشر شد، یک نسخه دیگر از زبان برنامه‌نویسی JavaScript است که تغییرات و بهبودهایی را به آن اضافه کرد. در زیر به برخی از ویژگی‌های مهم JavaScript 1.3 اشاره می‌کنم:

- قابلیت‌های فرم: JavaScript 1.3 از قابلیت‌های پیشرفته‌تری برای کنترل و اعتبارسنجی فرم‌ها در صفحات وب پشتیبانی می‌کند که شامل اعتبارسنجی ورودی‌ها، کنترل پارامترهای فرم و دسترسی به عناصر فرم مانند دکمه‌ها و جعبه‌های انتخاب است.

- توابع شبه-شیء‌گرا: JavaScript 1.3 این امکان را فراهم می‌کند که توابع را درون شیء‌ها تعریف کنیم. به این ترتیب، می‌توانیم توابع را به عنوان متدهای شیء در برنامه خود استفاده کنیم.

- پشتیبانی از منابع‌های خارجی: JavaScript 1.3 قادر است با منابع خارجی مانند تصاویر و فریم‌ها (IFrame) تعامل کند. این قابلیت به برنامه‌نویسان اجازه می‌دهد تا عناصر صفحه را به صورت دینامیک تغییر دهند و با منابع خارجی ارتباط برقرار کنند.

- مدیریت رویدادها: JavaScript 1.3 بهبودهایی در مدیریت رویدادها ارائه می‌دهد که شامل اضافه کردن و حذف رویدادها، تعیین اولویت رویدادها و کنترل بیشتر بر رویدادها است.

این تغییرات هم تنها بخشی از بهبودها و اضافات موجود در JavaScript 1.3 هستند این نسخه هم قدیمی است و تغییرات بسیاری در نسخه‌های جدیدتر JavaScript اعمال شده است.

۵. JavaScript 1.4 (1999)

این نسخه شامل بهبودهایی در زمینه عملگرها، کلاس‌ها و رویدادها داشت.

JavaScript 1.4 یک نسخه از زبان برنامه‌نویسی JavaScript است که در سال ۱۹۹۹ منتشر شد و تغییرات و بهبودهایی را به زبان اضافه کرد. در زیر به برخی از ویژگی‌های مهم JavaScript 1.4 اشاره می‌کنیم:

- پشتیبانی از XML: JavaScript 1.4 قابلیت‌های بهبود یافته‌ای برای پردازش و کار با XML را ارائه می‌دهد. با استفاده از توابع و متدهای جدید، برنامه‌نویسان می‌توانند اطلاعات XML را بخوانند، تغییر دهند و با آنها تعامل کنند.

- کنترل شیء‌پیمانی: JavaScript 1.4 امکانات پیشرفته‌تری را برای کنترل شیء‌پیمانی (DOM) ارائه می‌دهد. با استفاده از این قابلیت‌ها، برنامه‌نویسان می‌توانند به طور دقیق عملیات‌های مربوط به عناصر صفحه و مستند DOM را کنترل کنند.

- افزایش کارایی: JavaScript 1.4 بهبودهایی در کارایی اجرای برنامه‌های JavaScript ارائه می‌دهد که شامل بهبود عملکرد پارسر و اجراگر JavaScript و بهینه‌سازی‌های دیگر است که اجرای برنامه‌ها را سریع‌تر می‌کند.

- قابلیت‌های رسمی: JavaScript 1.4 از قابلیت‌های رسمی برای کنترل رسمی صفحه استفاده می‌کند. این شامل امکاناتی مانند رنگ‌های پیشرفته، تعیین موقعیت دقیق المان‌ها در صفحه و کنترل دقیق المان‌ها است.

این تغییرات هم تنها بخشی از بهبودها و اضافات موجود در JavaScript 1.4 هستند. توجه داشته باشید که این نسخه نیز قدیمی است و تغییرات بسیاری در نسخه‌های جدیدتر JavaScript اعمال شده است.

۶. JavaScript 1.5 (2000)

در این نسخه، ویژگی‌هایی مانند مدیریت خطاها با استفاده از `try-catch` و پشتیبانی از نمادهای عبارت‌های منظم به زبان اضافه شد.

JavaScript 1.5، که در سال ۲۰۰۰ منتشر شد، یک نسخه دیگر از زبان برنامه‌نویسی JavaScript است که تغییرات و بهبودهای مهمی را به آن اضافه کرد. در زیر به برخی از ویژگی‌های مهم JavaScript 1.5 اشاره می‌کنیم:

- مدیریت خطا: JavaScript 1.5 بهبودهایی در مدیریت خطا و اشکال‌زدایی ارائه می‌دهد. این شامل امکان استفاده از واحد `try-catch` برای کنترل و رفع خطاها است. با استفاده از این واحد، برنامه‌نویسان می‌توانند خطاها را شناسایی کرده و با آنها به صورت مناسب برخورد کنند.
- توابع جدید: JavaScript 1.5 تعدادی توابع جدید را به زبان اضافه کرد. این شامل توابعی مانند `forEach`، `map`، `filter` و `reduce` است که امکانات پیمایش و تغییر داده‌های آرایه را بهبود می‌بخشند.
- مدیریت حافظه بهبود یافته: JavaScript 1.5 بهبودهایی در مدیریت حافظه ارائه می‌دهد. این شامل بهینه‌سازی‌هایی در مصرف حافظه و مدیریت آن است که عملکرد برنامه‌ها را بهبود می‌بخشد.
- افزایش کارایی: JavaScript 1.5 بهبودهایی در کارایی اجرای برنامه‌ها ارائه می‌دهد. این شامل بهینه‌سازی‌های جدید در موتور JavaScript و بهبودهایی در عملکرد عمومی زبان است که اجرای برنامه‌ها را سریع‌تر می‌کند.

۷. ECMAScript 3 (1999)

این نسخه بر اساس جاوااسکریپت ۱.۵ بود و به عنوان استاندارد رسمی ECMAScript تایید شد. ECMAScript نام استاندارد است که بر روی جاوااسکریپت تعریف شده و نسخه‌های جاوااسکریپت بر اساس آن توسعه می‌یابند.

ECMAScript 3 (ES3)، یک استاندارد برای زبان برنامه‌نویسی JavaScript است که در سال ۱۹۹۹ توسط ECMA International معرفی شد. این استاندارد شامل تغییرات و بهبودهای مهمی است که به زبان JavaScript اضافه شده است. در زیر به برخی از ویژگی‌های مهم ECMAScript 3 اشاره می‌کنیم:

- تعریف مجدد ساختارهای زبانی: ECMAScript 3 بازنگری جدیدی در ساختارهای زبانی JavaScript ارائه می‌دهد. این شامل تعریف مجدد و بهبود گرامر، نحو و نحوه اجرای دستورات JavaScript است. این تغییرات موجب بهبود قابلیت خوانایی و پشتیبانی از کدهای قدیمی تر می‌شود.
- اضافه شدن متدهای جدید به شیء‌ها: ECMAScript 3 امکانات جدیدی را برای شیء‌ها و متدهای آنها ارائه می‌دهد. این شامل توابع مانند `Object.defineProperty`، `Object.defineProperties` و `Object.create` است که کنترل بیشتری روی خصوصیت‌ها و رفتار شیء‌ها می‌دهند.
- مدیریت خطا بهتر: ECMAScript 3 بهبودهایی در مدیریت خطا و اشکال‌زدایی ارائه می‌دهد. این شامل امکان استفاده از واحد `try-catch-finally` برای کنترل خطاها و اجرای بخشی از کد در صورت بروز خطا است.

- پشتیبانی از استثناءها: ECMAScript 3 استثناءها را به زبان JavaScript اضافه کرده است. استثناءها به برنامه‌نویسان اجازه می‌دهند با خطاها و شرایط غیرمنتظره مواجه شده و به صورت مناسب برخورد کنند.

ECMAScript 3 ، در واقع یک نسخه پایه و مهم در تاریخ JavaScript است و از آن به بعد، نسخه‌های جدیدتری از ECMAScript مانند ES5، ES6 و غیره معرفی شده‌اند که تغییرات و بهبودهای بیشتری را به زبان JavaScript اضافه کرده‌اند.

۸. ECMAScript 4 (لغو شد)

این نسخه به عنوان یک نسخه بهبود یافته برای ECMAScript توسعه داده شد، اما به دلایلی نسخه ۴ لغو شد و به جای آن توسعه بر روی نسخه بعدی شروع شد.

ECMAScript 4 (ES4) ، که قرار بود نسخه بعدی استاندارد ECMAScript بعد از ES3 باشد، متأسفانه قبل از انتشار رسمی لغو شد و توسعه آن متوقف شد. این تصمیم در نهایت به دلایل فنی و سیاسی گرفته شد و منجر به تغییر روند توسعه زبان JavaScript شد.

ES4 در نظر داشت تغییرات و گسترش‌های اساسی را در مقایسه با ES3 ارائه دهد که شامل موارد زیر بود:

- مفسر جدید: ES4 قرار بود یک مفسر جدید و بهبود یافته برای زبان JavaScript ارائه دهد. این مفسر مبتنی بر معماری مبتنی بر شیء بوده و عملکرد و کارایی را بهبود می‌بخشد.
- توابع بلوکی: ES4 قرار بود قابلیت تعریف توابع بلوکی را به زبان JavaScript اضافه کند. توابع بلوکی به برنامه‌نویسان امکان می‌دادند توابعی را به عنوان بلوک‌های قابل استفاده و قابل تکرار تعریف کنند.

- پیکربندی‌پذیری بیشتر: ES4 قرار بود قابلیت پیکربندی‌پذیری بیشتری را در زبان JavaScript فراهم کند. این شامل امکان تعیین ویژگی‌های مورد نیاز برای یک برنامه‌نویسی مشخص بود که می‌توانست بهینه‌سازی‌های خاصی را اعمال کند.

- الگوهای طراحی: ES4 قرار بود الگوهای طراحی پیشرفته‌تری را در زبان JavaScript پشتیبانی کند. این شامل الگوهای مانند کلاس‌ها، وراثت و گسترش‌های زبانی بود که برنامه‌نویسان را قادر می‌ساخت تا برنامه‌های پیچیده‌تری را ایجاد و مدیریت کنند.

با این حال، به دلایلی از جمله عدم توافق بین اعضای کمیته معیاری ECMA و تناقض‌های فنی، توسعه ES4 لغو شد. به جای آن، تصمیم گرفته شد تغییرات را به صورت تدریجی و در قالب نسخه‌های کوچک‌تری در ES5، ES6 و نسخه‌های بعدی معرفی کنند. این استانداردهای جدید برای توسعه و بهبود زبان JavaScript پیشرفت‌های قابل توجهی را به همراه داشته‌اند.

۹. ECMAScript 5 (2009)

ECMAScript 5، پنجمین نسخه از استاندارد ECMAScript است که مشخصات زبان برنامه‌نویسی جاوا اسکریپت را تعریف می‌کند. این نسخه در دسامبر ۲۰۰۹ منتشر شد و ویژگی‌ها و بهبودهای جدیدی نسبت به نسخه قبلی به این زبان اضافه کرد.

در ES5، تغییرات و اصلاحات متعددی معرفی شده است که به توسعه‌دهندگان کمک می‌کند تا بهتر و به‌صورت پویا برنامه‌های جاوا اسکریپت را بنویسند. برخی از مهم‌ترین ویژگی‌های ES5 عبارتند از:

- پشتیبانی از "strict mode" که یک حالت اجباری برای نوشتن کدهاست و کمک می‌کند بهبودهای امنیتی و عملکردی را در برنامه‌ها اعمال کنید.
- قابلیت تعریف ویژگی‌های مشخص (property descriptors) برای کنترل ویژگی‌های شیء‌ها مانند قابل نوشتن یا قابل تغییر بودن.
- متدهای آرایه (array methods) مانند map، forEach، filter و reduce که به توسعه‌دهندگان امکان می‌دهد با آرایه‌ها بهبودهای عملکردی و قدرتمندتری اعمال کنند.
- پشتیبانی از JSON با استفاده از توابع JSON.parse و JSON.stringify که امکان تبدیل داده‌ها به فرمت JSON و برعکس را فراهم می‌کند.
- اضافه شدن متدهای مشابه Object و Array برای کار با متغیرها و مجموعه‌ها مانند Object.keys، Object.create و Array.isArray.

ES5 به صورت گسترده‌ای توسط مرورگرها و محیط‌های اجرای جاوا اسکریپت پشتیبانی می‌شود و تا به امروز همچنان استفاده می‌شود، اما معمولاً نسخه‌های جدیدتر از آن مانند ECMAScript6 (ES6) و ECMAScript 2015 (ES2015) نیز برای توسعه برنامه‌های جاوا اسکریپت استفاده می‌شوند.

این نسخه معروف به ES6 است و شامل بهبودهای قابل توجهی در زبان جاوااسکریپت بود. اضافه شدن کلمات کلیدی مانند `let` و `const` برای تعریف متغیرها، کلاس‌ها، ماژول‌ها و توابع جدیدی مانند `arrow functions` از جمله تغییراتی در این نسخه بود.

ECMAScript 6 (ES6)، که همچنین با عنوان ECMAScript 2015 نیز شناخته می‌شود، نسخه بعدی استاندارد ECMAScript بعد از ES5 است. در سال ۲۰۱۵ توسط ECMA International معرفی شد و بهبودها و تغییرات قابل توجهی را در زبان JavaScript ارائه می‌دهد. در زیر به برخی از ویژگی‌های مهم ES6 اشاره می‌کنم:

- متغیرهای قابل تعریف با استفاده از ``let`` و ``const`` : ES6، قابلیت تعریف متغیرهای بلوکی را با استفاده از کلیدواژه ``let`` و متغیرهای ثابت را با استفاده از ``const`` به زبان JavaScript اضافه کرد. این کلیدواژه‌ها به برنامه‌نویسان امکان می‌دهند متغیرهایی را تعریف کنند که محدوده‌ی عملکرد آنها تنها در بلوک محل تعریف آنها قرار دارد.

- تعریف توابع پارامتری ES6 : به برنامه‌نویسان اجازه می‌دهد توابعی را با پارامترهای پیش فرض تعریف کنند. این به معنای این است که می‌توانیم مقادیر پیش فرضی را برای پارامترهای تابع تعیین کنیم و در صورتی که مقداری برای آنها ارسال نشود، مقدار پیش فرض مورد استفاده قرار می‌گیرد.

- استفاده از Arrow Functions : ES6، امکان تعریف توابع فشرده شده و کوتاه‌تر را با استفاده از Arrow Functions فراهم می‌کند. این توابع از نحوه‌ی نوشتاری مختصرتری برخوردار هستند و معمولاً برای توابعی که به عنوان پارامتر یا بازگشت از توابع دیگر استفاده می‌شوند، مورد استفاده قرار می‌گیرند.

- کلاس‌ها : ES6، مکان تعریف کلاس‌ها به زبان JavaScript را فراهم می‌کند. این کلاس‌ها بر اساس الگوی کلاسیک شیء‌گرا مدل‌سازی شده‌اند و ساختاری منسجم برای تعریف و استفاده از توابع سازنده، وراثت، و متدهای عمومی و خصوصی را فراهم می‌کنند.

- ماژول‌ها : ES6، به زبان JavaScript ساختارهای ماژولاری را معرفی می‌کند که به برنامه‌نویسان اجازه می‌دهد کدها را به بخش‌های کوچک تر تقسیم کنند. این ماژول‌ها امکان وارد کردن کدهای خارجی را با استفاده از `import` و صادر کردن کدها به عنوان ماژول با استفاده از `export` فراهم می‌کنند.

- تعدادی از ویژگی‌های دیگر : ES6، شامل تعداد زیادی از تغییرات و بهبودهای دیگر است که عبارتند از: استفاده از Template Strings برای تعریف رشته‌های چندخطی و تعبیه عبارات متغیر درون آنها، استفاده از Spread و Rest Operators برای کپی آرایه‌ها و آرگومان‌ها، اضافه کردن Iterators و Generators برای حلقه‌بندی و تولید داده‌ها، و اضافه کردن Promises برای مدیریت بهتر عملیات‌های ناهمزمان.

از آن زمان به بعد، استاندارد ECMAScript با انتشار نسخه‌های بعدی مانند ES7، ES8 و غیره، بهبودها و ویژگی‌های جدیدی را به زبان JavaScript اضافه کرده است. این تغییرات به توسعه‌دهندگان اجازه می‌دهد کدهای بهتری را بنویسند و برنامه‌های پیچیده‌تری را با استفاده از زبان JavaScript ایجاد کنند.

۱۱. ECMAScript 2016 و نسخه‌های بعدی

از این پس، برای نسخه‌های بعد از ECMAScript 6، تصمیم گرفته شد که نسخه‌بندی زبان به صورت سالیانه انجام شود و هر نسخه با توجه به سال انتشار آن نامگذاری شود. به عنوان مثال، ECMAScript 2016 به عنوان نسخه بعدی از ES6 منتشر شد و سپس نسخه‌های ECMAScript 2017، ECMAScript 2018 و غیره به ترتیب معرفی شدند.

مهمترین نسخه‌های ECMAScript بعد از ES6 عبارتند از:

• ECMAScript 2017

• ECMAScript 2018

• ECMAScript 2019

• ECMAScript 2020

• ECMAScript 2021

هر نسخه از ECMAScript اغلب شامل تغییرات، بهبودها و اضافاتی است که زبان جاوااسکریپت را قدرتمندتر، قابلیت‌های آن را جدیدتر و سازگارتر می‌کند. توسعه‌دهندگان وب تلاش می‌کنند تا با استفاده از نسخه‌های جدید ECMAScript، از امکانات و ویژگی‌های جدید زبان بهره‌برداری کنند و تجربه کاربری بهتری را برای کاربران ایجاد کنند.

۱-۲ این زبان برنامه‌نویسی در چه زمینه‌ها و حوزه‌هایی کاربرد دارد؟

زبان برنامه‌نویسی جاوااسکریپت در چندین زمینه و حوزه کاربرد دارد. این زبان اصلی‌ترین زبان برنامه‌نویسی برای توسعه وب است.

• حوزه‌های اصلی کاربرد جاوااسکریپت:

۱- توسعه وب: جاوااسکریپت برای توسعه اپلیکیشن‌های وب استفاده می‌شود. با استفاده از جاوااسکریپت، می‌توان صفحات وب را پویا کرده، رفتارها و واکنش‌های کاربر را کنترل کرده، فرم‌ها را اعتبارسنجی کرده و ارتباط با سرور را برقرار کرد.

۲. برنامه‌نویسی سمت کاربر (Front-end): جاوااسکریپت به عنوان زبان اصلی برای توسعه سمت کاربر (front-end) وب استفاده می‌شود. با استفاده از کتابخانه‌ها و فریمورک‌های مختلفی مانند React، Angular و Vue.js، می‌توان به راحتی رابط کاربری پویا و تعاملی را برای وبسایت‌ها و برنامه‌های وب ایجاد کرد.

۳. توسعه برنامه‌های موبایل: با استفاده از فریمورک‌های مانند React Native و Ionic، می‌توان با استفاده از جاوااسکریپت برنامه‌های موبایل چندسکویی (cross-platform) را توسعه داد. این فریمورک‌ها امکان تبدیل کد جاوااسکریپت به کد نیتیو برای سیستم‌عامل‌های iOS و Android را فراهم می‌کنند.

۴. توسعه بازی‌ها: با استفاده از کتابخانه‌ها مانند Phaser و Three.js ، می‌توان با استفاده از جاوااسکریپت بازی‌های وب زیبا و پویا را توسعه داد. این کتابخانه‌ها قابلیت‌های گرافیکی قدرتمندی را به توسعه‌دهندگان ارائه می‌دهند.

۵. برنامه‌نویسی سمت سرور (Back-end): با استفاده از محیط‌های اجرای جاوااسکریپت مانند Node.js ، می‌توان برنامه‌های سمت سرور را با جاوااسکریپت توسعه داد که این امکان را می‌دهد تا توسعه‌دهندگان از یک زبان برنامه‌نویسی مشابه در سمت سرور و سمت کاربر استفاده کنند و به اشتراک قطعات کد بین دو سمت دسترسی داشته باشند.

۶. توسعه برنامه‌های دسکتاپ: با استفاده از فریمورک‌هایی مانند Electron ، می‌توان با استفاده از جاوااسکریپت برنامه‌های دسکتاپ چندسکویی را توسعه داد که بر روی سیستم‌عامل‌های مختلف اجرا می‌شوند.

۷. اینترنت اشیا (IoT) : با استفاده از پلتفرم‌هایی مانند Johnny-Five و Cylon.js ، می‌توان از جاوااسکریپت برای برنامه‌نویسی دستگاه‌های اینترنت اشیا استفاده کرد و با آنها تعامل کرد.

۸. توسعه برنامه‌های هیبریدی: با استفاده از فریمورک‌هایی مانند Ionic و Cordova ، می‌توان با استفاده از جاوااسکریپت برنامه‌های هیبریدی را توسعه داد که همزمان بر روی وب و دستگاه‌های موبایل اجرا می‌شوند.

۹. توسعه برنامه‌های هوش مصنوعی و یادگیری ماشین: با استفاده از کتابخانه‌هایی مانند TensorFlow.js و Brain.js ، می‌توان از جاوااسکریپت برای توسعه برنامه‌های هوش مصنوعی، شبکه‌های عصبی و یادگیری ماشین استفاده کرد.

۱۰. توسعه اپلیکیشن‌های دیگر: جاوااسکریپت در بسیاری از حوزه‌های برنامه‌نویسی کاربرد دارد، از جمله توسعه اپلیکیشن‌های تلفن همراه، اپلیکیشن‌های ادغام سیستم، اپلیکیشن‌های مدیریت پایگاه داده، و بسیاری دیگر.

به طور کلی، جاوااسکریپت به عنوان یک زبان برنامه‌نویسی چندمنظوره شناخته می‌شود و در بسیاری از حوزه‌ها و زمینه‌های برنامه‌نویسی مورد استفاده قرار می‌گیرد.

۱-۳ این زبان برنامه نویسی در ابتدا به چه منظوری تهیه شده است؟

جاوااسکریپت در ابتدا برای استفاده در مرورگرها و تعامل با صفحات وب توسعه داده شد. هدف اصلی ایجاد این زبان برنامه‌نویسی، امکان ایجاد صفحات وب پویا و تعاملی بود. در سال‌های ابتدایی وب، صفحات وب اغلب به صورت استاتیک و بدون تعامل با کاربران بودند. با ایجاد جاوااسکریپت، امکاناتی برای تغییر پویا در محتوا و رفتار صفحات وب اضافه شد.

استفاده از جاوااسکریپت، به توسعه‌دهندگان اجازه می‌دهد تا رویدادها (مانند کلیک کردن بر روی یک المان، ورود به فرم و غیره) را مشخص کنند و به آنها واکنش دهند. مثلاً می‌توانند المان‌ها را پنهان یا نمایش دهند، محتوا را به روز کنند، اعتبارسنجی فرم‌ها را انجام دهند و اطلاعات را از سرور بخوانند و نمایش دهند.

به مرور زمان، جاوااسکریپت گسترش بیشتری یافت و به طور گسترده در دیگر زمینه‌ها مانند توسعه برنامه‌های سمت سرور (با استفاده از Node.js) و توسعه برنامه‌های موبایل (با استفاده از فریمورک‌های مانند React Native) نیز استفاده شد. اما اصلی‌ترین منظور ایجاد جاوااسکریپت همچنان استفاده در محیط مرورگرها بود.

۴-۱ این زبان برنامه نویسی برای رفع چه مشکلاتی ابداع شده است؟

زبان برنامه نویسی جاوااسکریپت ابتدا برای رفع مشکلات مربوط به توسعه وب ابداع شد. در زمان ایجاد آن، صفحات وب بیشتر به صورت استاتیک و بدون تعامل بودند. با استفاده از جاوااسکریپت، توسعه دهندگان قادر شدند رفتارها و واکنش های کاربر را در صفحات وب پیاده سازی کنند. بنابراین، جاوااسکریپت به عنوان زبان برنامه نویسی اصلی برای توسعه صفحات وب به صورت پویا و تعاملی به وجود آمد.

بعدها، با پیشرفت فناوری و نیازهای جدید، جاوااسکریپت در حوزه های دیگری نیز استفاده شد. به عنوان مثال، با ظهور موبایل و نیاز به توسعه برنامه های موبایل، فریمورک هایی مانند React Native و Ionic برای توسعه برنامه های موبایل با استفاده از جاوااسکریپت ایجاد شدند.

در طول زمان، جاوااسکریپت توانسته است با پشتوانه فریمورک ها و کتابخانه های متنوع، به یک زبان برنامه نویسی چندمنظوره تبدیل شود. از طریق جاوااسکریپت، می توان برنامه های وب، برنامه های سمت سرور، برنامه های دسکتاپ، برنامه های هیبریدی، برنامه های IoT، برنامه های هوش مصنوعی و بسیاری دیگر را توسعه داد.

بنابراین، جاوااسکریپت نه تنها مشکلات مربوط به توسعه وب را حل کرده است، بلکه به یک زبان برنامه نویسی کلیدی در حوزه های مختلف تبدیل شده است و به توسعه دهندگان امکان می دهد تا با استفاده از یک زبان مشابه، در چندین حوزه برنامه نویسی فعالیت کنند و کد قابل استفاده و قابل بازنگری داشته باشند.

۵-۱ ابداع این زبان در جهت بهبود چه زبان هایی بوده است؟

زبان برنامه نویسی جاوااسکریپت در زمان ابداع خود بهبودی بر زبان های موجود در آن زمان، به ویژه زبان های اسکریپت مانند LiveScript و JScript، هدف داشت. جاوااسکریپت ابتدا توسط برنامه نویسان برای ایجاد اسکریپت های کوچک و محدود در صفحات وب استفاده می شد.

اما در طول زمان، با بهبودهای و اضافه شدن ویژگی‌های جدید، جاوااسکریپت توانست به یک زبان برنامه‌نویسی کامل و قدرتمند تبدیل شود. با استفاده از تکنولوژی‌های جدید مانند AJAX (Asynchronous JavaScript and XML)، جاوااسکریپت قابلیت برقراری ارتباط غیرهمزمان با سرور را فراهم کرد و تجربه کاربری تعاملی و پویا را در صفحات وب ارائه داد.

به علاوه، با انتشار موتورهای جاوااسکریپت قدرتمند مانند V8 از طرف گوگل، عملکرد جاوااسکریپت بهبود یافت و سرعت اجرای برنامه‌های جاوااسکریپت به مراتب افزایش یافت.

بنابراین، ابداع جاوااسکریپت در جهت بهبود زبان‌های اسکریپت موجود در زمان خود بوده است و موجب شده است تا جاوااسکریپت به یکی از زبان‌های برنامه‌نویسی محبوب و استاندارد در حوزه توسعه وب تبدیل شود.

۱-۶ این زبان در ابتدای ظهور خود چه مشکلاتی را رفع می‌کرده است؟

زمانی که جاوااسکریپت در ابتدای ظهور خود بود، صفحات وب اغلب به صورت استاتیک بودند و امکانات تعاملی و پویا در آن‌ها محدود بود. با ظهور جاوااسکریپت، برخی از مشکلات زیر را با استفاده از قابلیت‌ها و ویژگی‌های این زبان رفع کرد.

۱. تعامل کاربر: جاوااسکریپت به توسعه‌دهندگان اجازه می‌دهد تا با استفاده از رویدادها و توابع مربوطه، با واکنش‌های کاربر در صفحات وب تعامل کنند. این قابلیت باعث شد تا کاربران بتوانند فرم‌ها را پر کنند، دکمه‌ها را کلیک کنند، درخواست‌ها را ارسال کنند و به طور کلی با صفحات وب تعامل داشته باشند.

۲. اعتبارسنجی فرم‌ها: با استفاده از جاوااسکریپت، توسعه‌دهندگان قادر شدند قوانین و محدودیت‌های مربوط به ورودی‌های فرم‌ها را بررسی کنند و اعتبارسنجی را انجام دهند. به طور مثال، می‌توانند بررسی کنند که آیا یک فیلد ایمیل معتبر است یا خیر و یا آیا یک فیلد شماره تلفن در قالب صحیح است یا خیر.

۳. تغییر دینامیک صفحه: با استفاده از جاوااسکریپت، می‌توان صفحات وب را به صورت دینامیک تغییر داد. به عنوان مثال، می‌توان بر اساس ورودی کاربر، بخش‌های مختلف صفحه را نمایش یا مخفی کرد، مقادیر را به روز کرد و محتوا را پویا کرد.

۴. ارتباط با سرور: جاوااسکریپت اجازه برقراری ارتباط غیرهمزمان با سرور را فراهم می‌کند. با استفاده از تکنیک‌هایی مانند AJAX، اطلاعات را به صورت پویا از سرور دریافت کرده و به صفحه اضافه کرد و یا اطلاعات را به سرور ارسال کرد.

این مشکلات و قابلیت‌های جاوااسکریپت در دهه ۱۹۹۰ و اوایل دهه ۲۰۰۰ که وب در مراحل اولیه خود بود، بسیار مهم بودند و باعث شدند تا جاوااسکریپت به یکی از اصلی‌ترین زبان‌های برنامه‌نویسی وب تبدیل شود.

۷-۱ این زبان در مقایسه با زبان‌های دیگر و به خصوص زبان‌هایی که شبیه آن هستند چگونه ارزیابی می‌شود؟

جاوااسکریپت در مقایسه با زبان‌های دیگر و به خصوص زبان‌هایی که شبیه آن هستند، ارزیابی متعددی دارد. در زیر به برخی از جوانب ارزیابی می‌پردازیم:

۱. قابلیت اجرا در مرورگرها: یکی از مزیت‌های اصلی جاوااسکریپت این است که تقریباً در همه مرورگرها قابل اجرا است. برنامه‌های نوشته شده با جاوااسکریپت بدون نیاز به نصب اضافه‌ای در مرورگر می‌توانند اجرا شوند. این امر باعث شده است تا جاوااسکریپت به زبان استاندارد و پایه‌ای برای توسعه وب تبدیل شود.

۲. برنامه‌نویسی مبتنی بر رویداد: جاوااسکریپت یک زبان برنامه‌نویسی مبتنی بر رویداد است که با استفاده از تابع‌ها و رویدادها، تعامل کاربر را مدیریت می‌کند. این رویکرد پویا و تعاملی باعث می‌شود تا برنامه‌های جاوااسکریپت قابلیت‌های تعاملی و پویایی بیشتری داشته باشند.

۳. پشتیبانی از برنامه‌نویسی شیء‌گرا: جاوااسکریپت از برنامه‌نویسی شیء‌گرا پشتیبانی می‌کند، که این امکان را می‌دهد تا برنامه‌های پیچیده‌تر و قابل توسعه‌تری را بنویسیم. این قابلیت شامل تعریف کلاس‌ها، ارث‌بری، انتزاع و پلی‌مورفیسم است.

۴. اکوسیستم و کتابخانه‌های پیشرفته: جاوااسکریپت دارای یک اکوسیستم بسیار پرنرنگ است که شامل مجموعه‌ای از کتابخانه‌ها و فریمورک‌های پیشرفته است. این کتابخانه‌ها و فریمورک‌ها به توسعه‌دهندگان امکانات و قابلیت‌های بیشتری را در اختیار می‌گذارند و فرآیند توسعه برنامه‌ها را سریع‌تر و آسان‌تر می‌کنند.

۵. کارایی و سرعت اجرا: در گذشته، جاوااسکریپت به دلیل عملکرد ضعیف و سرعت پایین در مقایسه با زبان‌های دیگر، مورد انتقاد بود. اما با بهبود موتورهای جاوااسکریپت مانند V8 از طرف گوگل و بهینه‌شدن زبان، کارایی و سرعت اجرای جاوااسکریپت بهبود یافته است و اکنون برنامه‌های جاوااسکریپت قادر به اجرای سریع‌تر و بهتر هستند.

به طور کلی، جاوااسکریپت به عنوان یک زبان برنامه‌نویسی توسعه وب و تعاملی محبوب است و به دلیل وجود یک اکوسیستم پویا و وسیع، قابلیت‌های تعاملی بالا، پشتیبانی از برنامه‌نویسی شیء‌گرا و اجرا در مرورگرهای مختلف، به خوبی ارزیابی می‌شود. همچنین، وجود فریمورک‌ها و کتابخانه‌های پیشرفته جاوااسکریپت، توسعه‌دهندگان را در توسعه برنامه‌های پیچیده و قابل توسعه کمک می‌کند.

۸-۱ این زبان چه ویژگی‌های خاصی دارد که آن را از زبان‌های مشابه آن متمایز می‌کند؟

جاوااسکریپت یک زبان برنامه‌نویسی منحصر به فرد است و نمی‌توان آن را با زبان‌های مشابه کاملاً مقایسه کرد. با این حال، تعدادی از ویژگی‌ها که جاوااسکریپت را از برخی از زبان‌های مشابه متمایز می‌کنند عبارتند از:

۱. اجرا در محیط مرورگر: جاوااسکریپت اصلی‌ترین زبان برنامه‌نویسی است که در محیط مرورگرها اجرا می‌شود. این به توسعه‌دهندگان امکان می‌دهد تا اسکریپت‌های جاوااسکریپت را برای تعامل با صفحات وب و ایجاد برنامه‌های سمت کاربر (Front-end) استفاده کنند.

۲. سینتکس ساده: جاوااسکریپت دارای سینتکسی ساده است که به توسعه‌دهندگان اجازه می‌دهد به راحتی کدهای خود را بنویسند و فهم بالایی داشته باشند. این زبان شباهت‌هایی به زبان‌هایی مانند C و Java دارد، اما سینتکس آن ساده‌تر از آنهاست.

۳. پویایی نوع: یکی از ویژگی‌های مهم جاوااسکریپت، پویایی نوع در آن است. این بدان معناست که نیازی به تعریف دقیق نوع متغیرها در زمان کامپایل ندارید و می‌توانید متغیرها را با انواع مختلف داده‌ها مقداردهی کنید. این ویژگی انعطاف‌پذیری بیشتری به توسعه‌دهندگان می‌دهد.

۴. پشتیبانی از برنامه‌نویسی کلاینت-سرور: جاوااسکریپت به عنوان زبان برنامه‌نویسی کلاینت-سرور در محیط وب استفاده می‌شود. با استفاده از آن، می‌توانید به سرور درخواست ارسال کنید، داده‌ها را دریافت کنید و صفحات وب را به صورت پویا به روز رسانی کنید.

۵. پشتیبانی از برنامه‌نویسی رویدادمحور: جاوااسکریپت به رویدادها (مانند کلیک کردن بر روی المان، ورود به فرم و غیره) واکنش نشان می‌دهد و به آنها پاسخ می‌دهد. این ویژگی به توسعه‌دهندگان اجازه می‌دهد تا به رویدادهای کاربران و محیط برنامه واکنش نشان دهند و عملکرد صفحه را تغییر دهند.

به طور کلی، جاوااسکریپت با ترکیب این ویژگی‌ها به توسعه‌دهندگان امکان می‌دهد برنامه‌های پویا و تعاملی را در محیط وب ایجاد کنند.

۱-۹ این زبان را بر اساس خوانایی، قابلیت اطمینان، هزینه و سایر معیارها مانند قابلیت جابجایی چگونه می‌توان ارزیابی کرد؟

ارزیابی زبان جاوا اسکریپت (JavaScript) بر اساس معیارهای مختلف می‌تواند به شکل زیر انجام شود:

۱. خوانایی:

- زبان جاوا اسکریپت به طور کلی یک زبان با خوانایی بالا است. نحو و سینتکس آن شبیه به زبان‌های C و C++ است و برخلاف برخی زبان‌های دیگر، نوشتن کد در زبان جاوا اسکریپت نسبتاً ساده است.
- با وجود این، به دلیل پشتیبانی از ویژگی‌های پیچیده‌تر مانند تابع‌ها و اشیاء، کد نوشته شده با جاوا اسکریپت ممکن است پیچیدگی بیشتری داشته باشد. بنابراین، استفاده از روش‌های مناسب برای سازماندهی کد و استفاده از الگوها و فرمت‌های استاندارد می‌تواند خوانایی کد را بهبود بخشد.

۲. قابلیت اطمینان:

- زبان جاوا اسکریپت یک زبان نوع پویا (Dynamic Typing) است که به برنامه‌نویسان امکان می‌دهد به راحتی از متغیرها استفاده کنند بدون اینکه نوع دقیق آنها را مشخص کنند. این مزیت می‌تواند در تسریع فرآیند توسعه کمک کند، اما در عین حال، با خطرات احتمالی خطاهای نوع مرتبط است.
- با استفاده از تست و ابزارهای کیفیت کد مناسب، می‌توان بهبود قابلیت اطمینان برنامه‌های جاوا اسکریپت را افزایش داد.

۳. هزینه (کارایی و بهره وری):

- زبان جاوا اسکریپت معمولاً در محیط مرورگرها برای اجرای کدها استفاده می‌شود و به طور کلی سرعت اجرای آن نسبتاً سریع است. با این حال، عملکرد برنامه‌های جاوا اسکریپت به عواملی مانند حجم کد، پیچیدگی الگوریتم‌ها و بهینه‌سازی‌های مورد استفاده بستگی دارد.

- برای بهبود کارایی و بهره وری، می‌توان از تکنیک‌هایی مانند کش‌سازی (caching)، بهینه‌سازی کد (code optimization) و کتابخانه‌ها و فریم‌ورک‌های موجود استفاده کرد.

۴. هزینه مورد نیاز برای یادگیری و برنامه‌نویسی:

- زبان جاوا اسکریپت یک زبان پرکاربری است که بسیار محبوبیت بالایی در بین برنامه‌نویسان دارد. به دلیل پختگی و بلوغ زبان، منابع آموزشی و مستندات زیادی در این زمینه وجود دارد.

- برای شروع یادگیری جاوا اسکریپت، می‌توانید از منابع آموزشی آنلاین رایگان، کتاب‌ها، دوره‌های آموزشی و جامعه‌های برنامه‌نویسی استفاده کنید. همچنین، وجود ابزارهای توسعه متن‌باز (مانند محیط‌های توسعه یکپارچه مانند Visual Studio Code) نیز هزینه یادگیری و برنامه‌نویسی را کاهش می‌دهد.

۵. قابلیت جابجایی:

- زبان جاوا اسکریپت به عنوان یک زبان مفسر (interpreted) در مرورگرها و همچنین در محیط‌های سمت سرور (مانند Node.js) قابل استفاده است. این ویژگی باعث می‌شود کدهای جاوا اسکریپت بتوانند در محیط‌های مختلف اجرا شوند و قابلیت جابجایی را داشته باشند.

- با این حال، باید توجه داشت که وجود API‌ها و ویژگی‌های خاص مربوط به هر محیط (مانند DOM در مرورگرها) ممکن است باعث محدودیت قابلیت جابجایی کد شود. بنابراین، برای ارزیابی کامل قابلیت جابجایی، باید نیازهای خاص پروژه خود را در نظر بگیرید.

در نهایت، ارزیابی زبان جاوا اسکریپت بستگی به نیازها و شرایط خاص پروژه ما دارد. این زبان با توجه به مزایا و معایب خود، در بسیاری از موارد می‌تواند یک انتخاب مناسب برای توسعه وب و برنامه‌نویسی سمت کاربر باشد.

۱-۱۰ آیا برای پیاده سازی این زبان از کامپایلر استفاده شده است یا مفسر یا پیاده سازی ترکیبی؟

جاوا اسکریپت از ابتدا به عنوان یک زبان تفسیری (interpreted language) طراحی شده است که بدان معنی است که کدهای جاوا اسکریپت به صورت مستقیم توسط مفسر (interpreter) خوانده و تفسیر می‌شوند. این بدین معنی است که کد جاوا اسکریپت خط به خط خوانده شده و هر خط به ترتیب تفسیر و اجرا می‌شود.

زمانی که مرورگر وب صفحه‌ای که شامل کد جاوا اسکریپت است را بارگیری می‌کند، مرورگر کد جاوا اسکریپت را به صورت مستقیم توسط مفسر خود (مانند موتور V8 در مرورگر Google Chrome) تفسیر و اجرا می‌کند. این فرآیند به صورت زنده و در حین اجرای کد انجام می‌شود.

اما در سال‌های اخیر، با پیشرفت تکنولوژی‌ها و محیط‌های اجرای جاوا اسکریپت، رویکردهای ترکیبی نیز مورد استفاده قرار گرفته‌اند. به عنوان مثال، در برخی محیط‌های اجرای جاوا اسکریپت مانند Node.js، کد جاوا اسکریپت ابتدا به کد بایت (bytecode) ترجمه (کامپایل) می‌شود و سپس توسط مفسر اجرا می‌شود. این روش، بهینه‌سازی و افزایش سرعت اجرای کد را ممکن می‌سازد.

بنابراین، در حالت کلی، جاوا اسکریپت به عنوان یک زبان تفسیری شناخته می‌شود، اما با پیشرفت تکنولوژی و استفاده از محیط‌های اجرای پیشرفته، رویکردهای ترکیبی نیز ممکن است استفاده شوند.

۱-۱۰، ۱۱ چه کامپایلرها یا مفسرهایی در حال حاضر برای این زبان وجود دارند و این کامپایلرها یا مفسرها توسط چه تیم هایی یا چه شرکت هایی تهیه شده اند؟

برای زبان جاوا اسکریپت، مجموعه‌ای از کامپایلرها و مفسرها وجود دارد که از آنها برای اجرای کد استفاده می‌شود. در ادامه، برخی از مهم‌ترین کامپایلرها و مفسرهای جاوا اسکریپت را ذکر خواهیم کرد:

برخی از مهم‌ترین کامپایلرها و مفسرها برای زبان جاوا اسکریپت عبارتند از:

V8: V8، که توسط گوگل توسعه داده شده است، یکی از قدرتمندترین موتورهای جاوا اسکریپت است. این موتور اصلی مرورگر Chrome است و به طور گسترده در مرورگرها، سرورها و برنامه‌های دیگر استفاده می‌شود. **V8** از ترکیبی از تفسیر و کامپایل (Just-in-Time) JIT استفاده می‌کند.

SpiderMonkey: SpiderMonkey، موتور جاوا اسکریپت مرورگر Firefox است. این موتور نیز از ترکیبی از تفسیر و کامپایل JIT برای اجرای کدهای جاوا اسکریپت استفاده می‌کند.

JavaScriptCore: JavaScriptCore یا JSC، موتور جاوا اسکریپت مرورگر Safari است. این موتور توسط Apple توسعه داده شده است و از JIT compilation برای بهبود عملکرد استفاده می‌کند.

Chakra: Chakra، یک موتور جاوا اسکریپت است که توسط مایکروسافت توسعه داده شده است. در حال حاضر، Chakra به عنوان موتور جاوا اسکریپت در مرورگر Microsoft Edge استفاده می‌شود.

Nashorn: Nashorn، یک موتور جاوا اسکریپت بر پایه Java Virtual Machine (JVM) است. این موتور در نسخه‌های قدیمی‌تر مرورگرهای Java کاربرد داشته است و در جاوا ۱۱ و بعد از آن از جاوا اسکریپت خارج شده است.

علاوه بر این‌ها، وجود مفسرهای دیگری نیز برای اجرای کدهای جاوا اسکریپت در محیط‌های خاص و سیستم‌های سروری وجود دارد. همچنین، ابزارهای مختلفی مانند Node.js نیز از مفسرها و موتورهای جاوا اسکریپت مختلف استفاده می‌کنند تا بتوانند کد جاوا اسکریپت را اجرا کنند.

۲- نحو و معناسازی

۱-۲ فهرستی از کلمات کلیدی زبان برنامه نویسی انتخاب شده نوشته شود و شرحی در مورد کاربرد آن‌ها داده شود.

۱-۱-۲ فهرستی از کلمات کلیدی زبان برنامه نویسی و شرحی در مورد کاربرد

در این بخش لیستی از کلمات کلیدی در زبان برنامه نویسی جاوااسکریپت (JavaScript) فراهم کرده ایم و توضیحی کوتاه درباره کاربرد آنها ارائه داده ایم:

- **Var**: این کلیدواژه برای تعریف متغیرها در جاوااسکریپت استفاده می شود. متغیرهایی که با ``var`` تعریف می شوند، بصورت محدود به بلوک موجود هستند و می توانند مقادیر مختلفی در طول برنامه دریافت کنند.

- **Let** و **const**: این دو کلیدواژه نیز برای تعریف متغیرها استفاده می شوند ``let``. یک متغیر قابل تغییر را تعریف می کند، به این معنی که می توانید مقدار آن را در طول برنامه تغییر دهید. از طرف دیگر، ``const`` یک متغیر ثابت را تعریف می کند و مقدار آن پس از تعریف قابل تغییر نیست.

- **Function**: این کلیدواژه برای تعریف توابع در جاوااسکریپت استفاده می شود. توابع می توانند برای انجام عملیات های مختلف و بازگشت نتایج استفاده شوند. می توانید پارامترها را به توابع ارسال کنید و مقادیر را از طریق دستور ``return`` بازگردانید.

- **if, else if, else**: این سه کلیدواژه برای ساختارهای شرطی در جاوااسکریپت استفاده می شوند. با استفاده از آنها می توانید برنامه را به صورت شرطی کنترل کنید و بر اساس شرایط مختلف عملیات های مختلفی انجام دهید.

- **for, while, do while**: این کلیدواژه‌ها برای ساختارهای حلقه در جاوااسکریپت استفاده می‌شوند. با استفاده از آنها می‌توانید بخش‌هایی از کد را تکرار کنید تا شرایط مشخصی برآورده شوند.
- **switch**: این کلیدواژه برای ساختار کنترل شرطی چندگانه استفاده می‌شود. به جای استفاده از چند شرط `if-else` متوالی، می‌توانید از `switch` استفاده کنید تا بر اساس مقادیر مختلف یک عملیات مشخص را انجام دهید.
- **return**: این کلیدواژه در توابع استفاده می‌شود تا مقداری را به عنوان خروجی از تابع بازگرداند. هنگامی که دستور `return` اجرا می‌شود، اجرای تابع متوقف می‌شود و مقدار مشخص شده به محیط فراخواننده تابع بازگردانده می‌شود.
- **Class**: این کلیدواژه برای تعریف کلاس‌ها در جاوااسکریپت استفاده می‌شود. کلاس‌ها مجموعه‌ای از ویژگی‌ها (متغیرها) و روش‌ها (توابع) هستند که می‌توانند برای ساختاردهی به برنامه و ایجاد شیء‌های قابل استفاده استفاده شوند.
- **this**: این کلیدواژه درون توابعی که در یک شیء تعریف می‌شوند، اشاره به شیء فعلی (مثلاً خود تابع) دارد. با استفاده از `this` می‌توانید به ویژگی‌ها و روش‌های شیء فعلی دسترسی پیدا کنید.
- **export و import**: این دو کلیدواژه در جاوااسکریپت مربوط به ماژول‌ها و وارد کردن/صادر کردن عملکرد است. با استفاده از `import` می‌توانید توابع و متغیرهایی را از یک ماژول به ماژول دیگر

وارد کنید، و با استفاده از `export` می‌توانید عملکردها و متغیرهایی را در یک ماژول عمومی کنید تا بتوانند از سایر ماژول‌ها استفاده شوند.

- `try, catch, finally`: این سه کلیدواژه برای مدیریت خطاها (exception) در جاوااسکریپت استفاده می‌شوند. با استفاده از بلوک `try` می‌توانید کدی را که احتمال بروز خطا دارد اجرا کنید، و در صورت بروز خطا، با استفاده از `catch` می‌توانید خطا را مدیریت کرده و عملکرد مناسب را انجام دهید. بلوک `finally` اختیاری است و در هر صورت اجرا می‌شود.

- `new`: این کلیدواژه برای ایجاد نمونه‌های جدید از کلاس‌ها یا شیء‌های قابل استفاده می‌شود. به عنوان مثال، می‌توانید با استفاده از `new` یک شیء جدید از یک کلاس را ایجاد کنید.

- `delete`: این کلیدواژه برای حذف یک خاصیت یا یک شیء در جاوااسکریپت استفاده می‌شود. با استفاده از `delete` می‌توانید یک خاصیت از یک شیء را حذف کنید یا یک شیء را به طور کامل حذف کنید.

- `typeof`: این کلیدواژه برای بازگرداندن نوع یک عبارت در جاوااسکریپت استفاده می‌شود. با استفاده از `typeof` می‌توانید نوع یک متغیر را بررسی کنید و بر اساس آن عملیات‌های مختلفی را انجام دهید.

- `instance of`: این کلیدواژه برای بررسی اینکه آیا یک شیء یا نمونه از یک کلاس خاص استفاده می‌شود یا خیر، استفاده می‌شود. با استفاده از `instance of` می‌توانید بررسی کنید که آیا یک شیء متعلق به یک کلاس خاص است یا نه.

- **in:** این کلیدواژه برای بررسی وجود یک ویژگی در یک شیء استفاده می‌شود. با استفاده از ``in`` می‌توانید بررسی کنید که آیا یک ویژگی در یک شیء وجود دارد یا خیر.
- **null:** این کلیدواژه برای نشان دادن عدم وجود یک مقدار یا عدم تعیین مقدار یک متغیر استفاده می‌شود. مقدار ``null`` به معنای خالی بودن یک متغیر است.
- **undefined:** این کلیدواژه برای نشان دادن عدم تعریف یک متغیر یا عدم وجود یک مقدار استفاده می‌شود. وقتی یک متغیر تعریف شده، اما به آن مقداری اختصاص نیافته باشد، مقدار ``undefined`` را خواهد داشت.
- **Nan:** این کلیدواژه برای نمایش نتیجه‌ی غیرعدم‌تن قبلی از حدوداً ۲۵۰ کاراکتر بیشتر شد و بنابراین در ادامه به برخی از کلمات کلیدی دیگر در جاوااسکریپت اشاره خواهیم کرد:
- **async:** این کلمه کلیدی برای تعریف یک تابع ناهمگام (`asynchronous`) استفاده می‌شود. توابع `async` به صورت پیش فرض یک `Promise` برمی‌گردانند و از کلمه کلیدی ``await`` برای صبر بر اجرای یک عملیات ناهمگام استفاده می‌کنند.
- **await:** این کلمه کلیدی درون توابع `async` استفاده می‌شود و برای صبر بر اجرای یک عملیات ناهمگام استفاده می‌شود. با استفاده از ``await`` می‌توانید اجرای یک `Promise` را متوقف کنید تا زمانی که `Promise` حل شود و نتیجه آن دریافت شود.

- **yield**: این کلمه کلیدی برای تعریف یک تابع مولفه‌بندی‌کننده (generator function) استفاده می‌شود. تابع‌های مولفه‌بندی‌کننده مقادیر را به صورت ترتیبی تولید می‌کنند و با استفاده از کلمه کلیدی **'yield'** مقدار تولید شده را بازگردانده و اجرای تابع را متوقف می‌کنند.

- **super**: این کلمه کلیدی درون متدهای کلاس‌ها استفاده می‌شود و به ارجاع به متد ویژه (special method) پدر (superclass) اشاره می‌کند. با استفاده از **'super'** می‌توانید به متدهای پدر کلاس خود دسترسی پیدا کنید و از آن‌ها استفاده کنید.

این فقط یک فهرست مختصر از کلمات کلیدی در جاوااسکریپت است و زبان برنامه‌نویسی جاوااسکریپت توسعه پذیری بالایی دارد و کلمات کلیدی بیشتری نیز وجود دارد که در ساختارهای مختلف به کار می‌روند.

۲-۱-۲ حوه دقیق استفاده از بعضی کلمه‌های کلیدی پر کاربرد هم در ادامه آمده است(این بخش داخل داک نبوده و صرفا به علت پرکاربرد بودن آورده شده است)

1. var:

- کاربرد: تعریف متغیر.
- مثال:

javascript

 Copy code

```
var x = 5;
```

2. let:

- کاربرد: تعریف متغیر به صورت محلی در یک بلوک.
- مثال:

javascript

 Copy code

```
let y = 10;
```

3. const:

- کاربرد: تعریف یک ثابت که مقدار آن نمی‌تواند تغییر کند.
- مثال:

javascript

 Copy code

```
const PI = 3.14;
```

4. function:

- کاربرد: تعریف یک تابع.
- مثال:

javascript

 Copy code

```
function add(a, b) {  
    return a + b;  
}
```

5. if, else:

- کاربرد: اعلام شرط و اجرای بخشی از کد در صورت برقراری یا عدم برقراری شرط
- مثال:

javascript

 Copy code

```
if (x > 0) {  
  console.log('Positive');  
} else {  
  console.log('Non-positive');  
}
```

6. for:

- کاربرد: ایجاد یک حلقه (لوپ) برای تکرار اجرای یک بلوک کد
- مثال:

javascript

 Copy code

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

7. while:

- **کاربرد:** ایجاد یک حلقه تا زمانی که یک شرط خاص برقرار باشد.
- **مثال:**

```
javascript Copy code  
  
let j = 0;  
while (j < 3) {  
    console.log(j);  
    j++;  
}
```

8. Array:

- **کاربرد:** تعریف و مدیریت آرایه‌ها.
- **مثال:**

```
javascript Copy code  
  
let numbers = [1, 2, 3, 4, 5];
```

9. Object:

- با خصوصیات و مقادیر (Object) **کاربرد:** تعریف و مدیریت اشیاء.
- **مثال:**

```
javascript Copy code  
  
let person = { name: 'John', age: 30, job: 'developer' };
```

10. DOM (Document Object Model):

- مستند XML و HTML **کاربرد:** تعامل با ساختار.
- **مثال:**

```
javascript Copy code  
  
document.getElementById('myElement').innerHTML = 'Hello, World!';
```

11. AJAX (Asynchronous JavaScript and XML):

- **کاربرد:** ارسال درخواست‌های اطلاعات به سرور بدون نیاز به بارگذاری مجدد صفحه.
- **مثال:**

javascript

 Copy code

```
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        console.log(this.responseText);
    }
};
xhttp.open('GET', 'https://example.com/data', true);
xhttp.send();
```

12. Promise:

- **کاربرد:** مدیریت عملیات ناهمگام و اجازه به کد برای ادامه اجرا در حین انجام یک عملیات ناهمگام.
- **مثال:**

javascript

 Copy code

```
let myPromise = new Promise(function(resolve, reject) {
    // اجرای یک عملیات ناهمگام
    if (operationIsSuccessful) {
        resolve('Success');
    } else {
        reject('Error');
    }
});
```

۲-۲ گرامر برای زیرمجموعه‌ای از زبان برنامه‌نویسی انتخاب شده نوشته شود.

عملیات تعریف متغیر

تعریف متغیر به شکل `var` :

`<variable-definition> → var <identifier> = <expression>;`

تعریف متغیر به شکل `let` :

`<variable-definition> → let <identifier> = <expression>;`

تعریف متغیر به شکل `const` :

`<variable-definition> → const <identifier> = <expression>;`

گرامری که ارائه داده شد، یک قالب برای تعریف متغیر در زبان جاوااسکریپت است. برای توضیح بیشتر، اجزای این گرامر به شرح زیر هستند:

`<variable-definition>`: این نشان‌دهنده شروع تعریف متغیر است.

`var`: این کلیدواژه به معنای تعریف یک متغیر استفاده می‌شود. در جاوااسکریپت، می‌توان از `var`، `let` یا `const` برای تعریف متغیر استفاده کرد. این گرامر از `var` استفاده کرده است.

`<identifier>`: این بخش نمایانگر نام متغیر (شناسه) است. در جاوااسکریپت، نام متغیرها باید با یک حرف شروع شوند و می‌توانند شامل حروف، اعداد، یا خط تیره باشند.

`=`: علامت مساوی، که به متغیر مقدار اولیه اختصاص می‌دهد.

`<expression>`: عبارتی که مقدار اولیه متغیر را تعیین می‌کند. این می‌تواند هر نوع عبارت معتبر در جاوااسکریپت باشد.

در نهایت، نشان‌دهنده پایان دستور یا تعریف متغیر است.

مثال:

```
var x = 10;
```

این مثال یک متغیر به نام X تعریف کرده و مقدار اولیه آن را برابر با ۱۰ قرار داده است. این تعریف متغیر با گرامر ارائه شده سازگار است.

عملیات جمع

برای تعریف گرامر عملیات جمع در زبان جاوااسکریپت می‌توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات جمع ارائه شده است:

$\langle \text{addition-expression} \rangle \rightarrow \langle \text{operand} \rangle + \langle \text{operand} \rangle$

این گرامر یک عبارت جمع را توصیف می‌کند که شامل دو $\langle \text{operand} \rangle$ (اپرند) است که توسط + افزوده شده‌اند.

$\langle \text{addition-expression} \rangle$: این نشان‌دهنده شروع یک عبارت جمع است.

$\langle \text{operand} \rangle$: این بخش نمایانگر عملوندها یا اجزای عبارت جمع است.

+: این علامت جمع نشان‌دهنده عمل جمع است.

از این گرامر می‌توانید برای توصیف عبارت‌های جمعی مانند زیر استفاده کنید:

```
var result = 5 + 3;
```

در این مثال، $\langle \text{addition-expression} \rangle$ با $\langle \text{operand} \rangle$ های ۵ و ۳ توسط + جمع می‌شود و نتیجه در متغیر result ذخیره می‌شود.

عملیات تفریق

برای تعریف گرامر عملیات تفریق در زبان جاوااسکریپت می‌توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات تفریق ارائه شده است:

$\langle \text{subtraction-expression} \rangle \rightarrow \langle \text{operand} \rangle - \langle \text{operand} \rangle$

این گرامر یک عبارت تفریق را توصیف می‌کند که شامل دو $\langle \text{operand} \rangle$ (اپرند) است که توسط - تفریق شده‌اند.

$\langle \text{subtraction-expression} \rangle$: این نشان‌دهنده شروع یک عبارت تفریق است.

$\langle \text{operand} \rangle$: این بخش نمایانگر عملوندها یا اجزای عبارت تفریق است.

- : این علامت تفریق نشان‌دهنده عمل تفریق است.

از این گرامر می‌توانید برای توصیف عبارت‌های تفریق مانند زیر استفاده کنید:

```
var result = 8 - 3;
```

در این مثال، $\langle \text{subtraction-expression} \rangle$ با $\langle \text{operand} \rangle$ های ۸ و ۳ توسط - تفریق می‌شود و نتیجه در متغیر result ذخیره می‌شود.

عملیات ضرب

برای تعریف گرامر عملیات ضرب در زبان جاوااسکریپت می‌توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات ضرب ارائه شده است:

$\langle \text{multiplication-expression} \rangle \rightarrow \langle \text{operand} \rangle \times \langle \text{operand} \rangle$

این گرامر یک عبارت ضرب را توصیف می‌کند که شامل دو $\langle \text{operand} \rangle$ (اپرند) است که توسط $\times$ ضرب شده‌اند.

$\langle \text{multiplication-expression} \rangle$: این نشان‌دهنده شروع یک عبارت ضرب است.

$\langle \text{operand} \rangle$: این بخش نمایانگر عملوندها یا اجزای عبارت ضرب است.

$\times$: این علامت ضرب نشان‌دهنده عمل ضرب است.

از این گرامر می‌توانید برای توصیف عبارت‌های ضرب مانند زیر استفاده کنید:

```
var result = 4 * 6;
```

در این مثال، `< multiplication-expression >` با `< operand >` های ۴ و ۶ توسط `×` ضرب می شود و نتیجه در متغیر `result` ذخیره می شود.

عملیات توان

برای تعریف گرامر عملیات توان در زبان جاوااسکریپت می توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات توان ارائه شده است:

`< exponentiation-expression > → < operand > ** < operand >`

این گرامر یک عبارت توان را توصیف می کند که شامل دو `< operand >` (اپرند) است که توسط `**` به توان رسیده اند.

`< exponentiation-expression >` : این نشان دهنده شروع یک عبارت توان است.

`< operand >` : این بخش نمایانگر عملوندها یا اجزای عبارت توان است.

`**` : این علامت توان نشان دهنده عمل توان است.

از این گرامر می توانید برای توصیف عبارت های توان مانند زیر استفاده کنید:

```
var result = 2 ** 3;
```

در این مثال، `< exponentiation-expression >` با `< operand >` های ۲ و ۳ توسط `**` به توان می رسند و نتیجه در متغیر `result` ذخیره می شود.

عملیات تقسیم

برای تعریف گرامر عملیات تقسیم در زبان جاوااسکریپت می توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات تقسیم ارائه شده است:

`< division-expression > → < operand > / < operand >`

این گرامر یک عبارت تقسیم را توصیف می کند که شامل دو `< operand >` (اپرند) است که توسط `/` تقسیم شده اند.

< division-expression > : این نشان دهنده شروع یک عبارت تقسیم است.

< operand > : این بخش نمایانگر عملوندها یا اجزای عبارت تقسیم است.

/ : این علامت تقسیم نشان دهنده عمل تقسیم است.

از این گرامر می‌توانید برای توصیف عبارت‌های تقسیم مانند زیر استفاده کنید:

```
var result = 10 / 2;
```

در این مثال، < division-expression > با < operand > های ۲ و ۳ توسط / تقسیم شده‌اند و نتیجه در متغیر result ذخیره می‌شود.

عملیات باقی‌مانده تقسیم

برای تعریف گرامر عملیات باقی‌مانده تقسیم در زبان جاوااسکریپت می‌توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات باقی‌مانده تقسیم ارائه شده است:

< remainder-expression > → < operand > % < operand >

این گرامر یک عبارت باقی‌مانده تقسیم را توصیف می‌کند که شامل دو < operand > (اپرند) است که توسط % باقی‌مانده تقسیم یکی بر دیگری بدست می‌آید.

< remainder-expression > : این نشان دهنده شروع یک عبارت باقی‌مانده تقسیم است.

< operand > : این بخش نمایانگر عملوندها یا اجزای عبارت باقی‌مانده تقسیم است.

% : این علامت تقسیم نشان دهنده عمل باقی‌مانده تقسیم است.

از این گرامر می‌توانید برای توصیف عبارت‌های باقی‌مانده تقسیم مانند زیر استفاده کنید:

```
var result = 11 % 3;
```

در این مثال، `<remainder-expression>` با `<operand>` های 11 و ۳ توسط % باقی مانده تقسیمشان بدست می آید و نتیجه در متغیر `result` ذخیره می شود.

عملیات افزایش

برای تعریف گرامر عملیات افزایش در زبان جاوااسکریپت می توانید از گرامر زیر استفاده کنید. در اینجا یک گرامر ساده برای عملیات افزایش ارائه شده است:

`<increment-expression> → <operand> ++`

این گرامر می گوید که یک عبارت افزایش از یک `<operand>` تشکیل شده است که به آن یک واحد افزوده شده است.

`<increment-expression>` : این نشان دهنده شروع یک عبارت افزایش است.

`<operand>` : این بخش نمایانگر عملوندها یا اجزای عبارت افزایش است.

`++` : این علامت افزایش نشان دهنده عمل افزایش است.

از این گرامر می توانید برای توصیف عبارت های افزایش مانند زیر استفاده کنید:

```
var x = 5;  
var result = x++;
```

در این مثال مقدار `x` یک واحد افزایش یافته و در `result` ذخیره می شود و `x` مقدار قبلی خود را نگه می دارد.

عملیات کاهش

برای تعریف گرامر عملیات کاهش (`decrement`) در زبان جاوااسکریپت، می توانید از یک گرامر به صورت زیر استفاده کنید:

`<decrement-expression> → <operand> --`

در این گرامر:

`<decrement-expression>`: نشان دهنده شروع یک عبارت کاهش است.

<operand>: عملوندها یا اجزای عبارت کاهش.

--: عملگر کاهش.

این گرامر می گوید که یک عبارت کاهش از یک <operand> تشکیل شده است که یک واحد از آن کم می شود.

برای مثال:

```
var counter = 5;  
var result = counter--;
```

در اینجا:

<decrement-expression> توسط counter-- توصیف می شود.

<operand> یک بار تکرار شده و مقدار counter را نشان می دهد.

-- نشان دهنده عملگر کاهش است.

بعد از اجرای این کد، مقدار counter از ۵ به ۴ کاهش پیدا می کند، و مقدار result هم مقدار قبلی counter (یعنی ۵) را نگه می دارد. این عمل به معنای کاهش counter پس از استفاده از مقدار فعلی آن است.

عملیات مساوی

برای تعریف گرامر عملیات مساوی (equal to) در زبان جاوااسکریپت، می توانید از یک گرامر به صورت زیر استفاده کنید:

<equal-to-expression> → <operand> == <operand>

در این گرامر:

<equal-to-expression>: نشان دهنده شروع یک عبارت مساوی است.

<operand>: عملوندها یا اجزای عبارت مساوی.

==: علامت مساوی.

این گرامر می‌گوید که یک عبارت مساوی از دو `<operand>` که توسط `==` تساوی آن‌ها بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 3;  
var result = x == y;
```

در اینجا:

`<equal-to-expression>` توسط `x == y` توصیف می‌شود.

`<operand>` دو بار تکرار شده و به ترتیب مقادیر `x` و `y` را نشان می‌دهد.

`==` نشان‌دهنده علامت مساوی است.

در این مثال تساوی دو `operand` بررسی شده و نتیجه (`true, false`) در `result` ذخیره می‌شود.

عملیات مقدار مساوی و نوع مساوی

برای تعریف گرامر عملیات مقدار مساوی و نوع مساوی (`equal value and equal type`) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

`<equal-value-and-type-expression> → <operand> === <operand>`

در این گرامر:

`<equal-value-and-type-expression>`: نشان‌دهنده شروع یک عبارت مقدار مساوی و نوع مساوی است.

`<operand>`: عملوندها یا اجزای عبارت مقدار مساوی و نوع مساوی.

`===`: علامت مساوی و نوع مساوی.

این گرامر می‌گوید که یک عبارت مقدار مساوی و نوع مساوی از دو `<operand>` تشکیل شده که توسط `===` مساوی بودن مقدار و نوع آن‌ها بررسی می‌شود.

برای مثال:

```
var x = 5;  
var y = "5";  
var result = x === y;
```

در اینجا:

<equal-value-and-type-expression> توسط $x === y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

$===$ نشان‌دهنده علامت مساوی و نوع مساوی است.

در این مثال برابر بودن مقدار و نوع x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات نامساوی بودن مقدار و نوع

برای تعریف گرامر عملیات نامساوی مقدار و نامساوی نوع (`not equal value and not equal type`) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

$\langle \text{not-equal-value-and-type-expression} \rangle \rightarrow \langle \text{operand} \rangle !== \langle \text{operand} \rangle$

در این گرامر:

<not-equal-value-and-type-expression>: نشان‌دهنده شروع یک عبارت نامساوی مقدار و نامساوی نوع است.

<operand>: عملوندها یا اجزای عبارت نامساوی مقدار و نامساوی نوع.

$!==$: علامت نامساوی مقدار و نامساوی نوع.

این گرامر می‌گوید که یک عبارت نامساوی مقدار و نامساوی نوع از دو <operand> که توسط $!==$ نامساوی بودن operand ها بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = "5";  
var result = x !== y;
```

در اینجا:

<not-equal-value-and-type-expression> توسط $x !== y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

$==$ نشان‌دهنده علامت نامساوی مقدار و نامساوی نوع است.

در این مثال عدم برابر بودن مقدار و نوع x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات نامساوی مقدار یا نامساوی نوع

برای تعریف گرامر عملیات نامساوی مقدار یا نامساوی نوع (`not equal value or not equal type`) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

$\text{<not-equal-value-or-type-expression>} \rightarrow \text{<operand>} \text{!= <operand>}$

در این گرامر:

<not-equal-value-or-type-expression>: نشان‌دهنده شروع یک عبارت نامساوی مقدار یا نامساوی نوع است.

<operand>: عملوندها یا اجزای عبارت نامساوی مقدار یا نامساوی نوع.

!=: علامت نامساوی مقدار یا نامساوی نوع.

این گرامر می‌گوید که یک عبارت نامساوی مقدار یا نامساوی نوع از دو **<operand>** که توسط **!=** نامساوی بودن مقدار یا نامساوی نوع **operand** ها بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = "5";  
var result = x != y;
```

در اینجا:

<not-equal-value-or-type-expression> توسط $x \neq y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

$\neq$ نشان‌دهنده علامت نامساوی مقدار یا نامساوی نوع است.

در این مثال عدم برابر بودن مقدار یا نوع x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات بزرگ‌تر

برای تعریف گرامر عملیات بزرگ‌تر از (`greater than`) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

`<greater-than-expression> → <operand> > <operand>`

در این گرامر:

`<greater-than-expression>`: نشان‌دهنده شروع یک عبارت بزرگ‌تر از است.

`<operand>`: عملوندها یا اجزای عبارت بزرگ‌تر از.

`<`: علامت بزرگ‌تر از.

این گرامر می‌گوید که یک عبارت بزرگ‌تر از از دو `<operand>` که توسط `<` بزرگ‌تر بودن `operand` ها بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 10;  
var y = 5;  
var result = x > y;
```

در اینجا:

<greater-than-expression> توسط $x > y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

< نشان‌دهنده علامت بزرگ‌تر از است.

در این مثال بزرگ‌تر بودن x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات کوچک‌تر

برای تعریف گرامر عملیات کمتر از (`less than`) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

`<less-than-expression> → <operand> < <operand>`

در این گرامر:

<less-than-expression>: نشان‌دهنده شروع یک عبارت کمتر از است.

<operand>: عملوندها یا اجزای عبارت کمتر از.

>: علامت کمتر از.

این گرامر می‌گوید که یک عبارت کمتر از از دو <operand> که توسط > کوچک‌تر بودن یکی از دیگری بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = x < y;
```

در اینجا:

<less-than-expression> توسط $x < y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

> نشان‌دهنده علامت کم‌تر از است.

در این مثال کوچک‌تر بودن x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات بزرگ‌تر مساوی

برای تعریف گرامر عملیات بزرگ‌تر مساوی (`greater than or equal to`) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

`<greater-than-or-equal-to-expression> → <operand> >= <operand>`

در این گرامر:

<greater-than-or-equal-to-expression>: نشان‌دهنده شروع یک عبارت بزرگ‌تر مساوی است.

<operand>: عملوندها یا اجزای عبارت بزرگ‌تر مساوی.

<=: علامت بزرگ‌تر مساوی.

این گرامر می‌گوید که یک عبارت بزرگ‌تر مساوی از دو <operand> که توسط `<=` بزرگ‌تر یکی از دیگری بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 10;  
var y = 5;  
var result = x >= y;
```

در اینجا:

<greater-than-or-equal-to-expression> توسط $x \geq y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

<= نشان‌دهنده علامت بزرگ‌تر مساوی است.

در این مثال بزرگ‌تر مساوی بودن x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات کم‌تر مساوی

برای تعریف گرامر عملیات کم‌تر مساوی (`less than or equal to`) در زبان جاوااسکریپت، می‌توانید از یک گرامر مشابه به صورت زیر استفاده کنید:

$\langle \text{less-than-or-equal-to-expression} \rangle \rightarrow \langle \text{operand} \rangle \leq \langle \text{operand} \rangle$

در این گرامر:

<less-than-or-equal-to-expression>: نشان‌دهنده شروع یک عبارت کم‌تر مساوی است.

<operand>: عملوندها یا اجزای عبارت کم‌تر مساوی.

>=: علامت کم‌تر مساوی.

این گرامر می‌گوید که یک عبارت کم‌تر مساوی از دو <operand> که توسط $=>$ کم‌تر مساوی بودن یکی از دیگری بررسی می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = x <= y;
```

در اینجا:

<less-than-or-equal-to-expression> توسط $x \leq y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

=> نشان‌دهنده علامت کمتر مساوی است.

در این مثال کمتر مساوی بودن x و y بررسی می‌شود و نتیجه در `result` ذخیره می‌شود.

عملیات عامل سه‌گانه

برای تعریف گرامر عملیات عامل سه‌گانه (ternary operator) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

<ternary-operator> → if (<logic-expr>) <stmt> else <stmt>

در این گرامر:

<ternary-operator>: نشان‌دهنده شروع یک عملگر سه‌گانه است.

<logic-expr>: عبارت منطقی مورد بررسی.

<stmt>: دستورات (بلوک کد) که در صورت برقرار بودن یا نبودن عبارت منطقی اجرا می‌شوند.

if: کلمه کلیدی if.

else: کلمه کلیدی else.

این گرامر می‌گوید که یک عملگر سه‌گانه از یک شرط (<logic-expr>) و دو بلوک کد (<stmt>) که با if و else مشخص شده‌اند، تشکیل شده است.

برای مثال:

```
var x = 5;  
var result = (x > 0) ? "Positive" : "Non-positive";
```

در اینجا:

<ternary-operator> توسط $(x > 0) ? \text{"Positive"} : \text{"Non-positive"}$ توصیف می‌شود.

<logic-expr> بخش $(x > 0)$ را نشان می‌دهد.

<stmt> دو بار تکرار شده و به ترتیب بلوک‌های کد "Positive" و "Non-positive" را نشان می‌دهد.

? نشان‌دهنده عملگر سه‌گانه است که شرط را بررسی می‌کند.

: نشان‌دهنده جداکننده بین دو بلوک کد است.

در این مثال مثبت بودن x بررسی می‌شود و مقدار "Positive" یا "Non-positive" در result ذخیره می‌شود.

عملیات و منطقی

برای تعریف گرامر عملیات و منطقی (logical AND) در زبان جاوااسکریپت، می‌توانید از یک گرامر به صورت زیر استفاده کنید:

$\langle \text{logical-and-expression} \rangle \rightarrow \langle \text{logic-expr} \rangle \&\& \langle \text{logic-expr} \rangle$

در این گرامر:

<logical-and-expression>: نشان‌دهنده شروع یک عبارت و منطقی است.

<logic-expr>: عبارت‌های منطقی که با یکدیگر به صورت و منطقی مرتبط هستند.

&&: علامت و منطقی.

این گرامر می‌گوید که یک عبارت و منطقی از دو `<logic-expr>` که توسط `&&`، و منطقی می‌شوند، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = (x > 0) && (y > 0);
```

در اینجا:

`<logical-and-expression>` توسط `(x > 0) && (y > 0)` توصیف می‌شود.

`<logic-expr>` دو بار تکرار شده و به ترتیب عبارت‌های `(x > 0)` و `(y > 0)` را نشان می‌دهد.

`&&` نشان‌دهنده علامت و منطقی است.

در این مثال `x` و `y`، and منطقی شده و حاصل در `result` ذخیره می‌شود.

عملیات یا منطقی

برای تعریف گرامر عملیات یا منطقی (`logical OR`) در زبان جاوااسکریپت، می‌توانید از یک گرامر مشابه به صورت زیر استفاده کنید:

`<logical-or-expression> → <logic-expr> || <logic-expr>`

در این گرامر:

`<logical-or-expression>`: نشان‌دهنده شروع یک عبارت یا منطقی است.

`<logic-expr>`: عبارت‌های منطقی که با یکدیگر به صورت یا منطقی مرتبط هستند.

`||`: علامت یا منطقی.

این گرامر می‌گوید که یک عبارت یا منطقی از دو `<logic-expr>` که توسط `||`، یا منطقی می‌شوند، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = (x > 0) || (y > 0);
```

در اینجا:

<logical-or-expression> توسط $(x > 0) || (y > 0)$ توصیف می‌شود.

<logic-expr> دو بار تکرار شده و به ترتیب عبارت‌های $(x > 0)$ و $(y > 0)$ را نشان می‌دهد.

$||$ نشان‌دهنده علامت یا منطقی است.

در این مثال x و y ، or منطقی شده و حاصل در `result` ذخیره می‌شود.

عملیات OR بیتی

در زبان جاوا اسکریپت، عملیات OR بیتی با استفاده از عملگر `|` انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب به صورت زیر استفاده کنید:

$\langle \text{bitwise-or-expression} \rangle \rightarrow \langle \text{operand} \rangle | \langle \text{operand} \rangle$

در این گرامر:

<bitwise-or-expression>: نشان‌دهنده شروع یک عبارت OR بیتی است.

<operand>: عملوندها یا اجزای عبارت OR بیتی.

|: علامت OR بیتی.

این گرامر می‌گوید که یک عبارت OR بیتی از دو <operand> که توسط `|`، OR می‌شوند، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = x | y;
```

در اینجا:

<bitwise-or-expression> توسط $x \mid y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

| نشان‌دهنده علامت OR بیتی است.

در این مثال x و y ، or بیتی شده و حاصل در `result` ذخیره می‌شود.

عملیات AND بیتی

در زبان جاوااسکریپت، عملیات AND بیتی با استفاده از عملگر `&` انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب مشابه به صورت زیر استفاده کنید:

<bitwise-and-expression> → <operand> & <operand>

در این گرامر:

<bitwise-and-expression>: نشان‌دهنده شروع یک عبارت AND بیتی است.

<operand>: عملوندها یا اجزای عبارت AND بیتی.

&: علامت AND بیتی.

این گرامر می‌گوید که یک عبارت AND بیتی از دو <operand> که توسط `&`، AND می‌شوند، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = x & y;
```

در اینجا:

<bitwise-and-expression> توسط $x \& y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

$\&$ نشان‌دهنده علامت AND بیتی است.

در این مثال x و y ، and بیتی شده و حاصل در `result` ذخیره می‌شود.

عملیات NOT بیتی

در زبان جاوااسکریپت، عملیات NOT بیتی با استفاده از عملگر $\sim$ انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب مشابه به صورت زیر استفاده کنید:

$\<\text{bitwise-not-expression}\> \rightarrow \sim \<\text{operand}\>$

در این گرامر:

<bitwise-not-expression>: نشان‌دهنده شروع یک عبارت NOT بیتی است.

<operand>: عملوند یا اجزای عبارت NOT بیتی.

$\sim$: علامت NOT بیتی.

این گرامر می‌گوید که یک عبارت NOT بیتی از یک <operand> که توسط $\sim$ ، NOT می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var result = ~x;
```

در اینجا:

<bitwise-not-expression> توسط $\sim x$ توصیف می‌شود.

<operand> یک بار تکرار شده و مقدار x را نشان می‌دهد.

$\sim$ نشان‌دهنده علامت NOT بیتی است.

در این مثال x ، not بیتی شده و حاصل در `result` ذخیره می‌شود.

عملیات XOR بیتی

در زبان جاوا اسکریپت، عملیات XOR بیتی با استفاده از عملگر $\wedge$ انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب به صورت زیر استفاده کنید:

$\langle \text{bitwise-xor-expression} \rangle \rightarrow \langle \text{operand} \rangle \wedge \langle \text{operand} \rangle$

در این گرامر:

<bitwise-xor-expression>: نشان‌دهنده شروع یک عبارت XOR بیتی است.

<operand>: عملوندها یا اجزای عبارت XOR بیتی.

$\wedge$: علامت XOR بیتی.

این گرامر می‌گوید که یک عبارت XOR بیتی از دو <operand> که توسط $\wedge$ ، XOR بیتی می‌شوند، تشکیل شده است.

برای مثال:

```
var x = 5;  
var y = 10;  
var result = x ^ y;
```

در اینجا:

<bitwise-xor-expression> توسط $x \wedge y$ توصیف می‌شود.

<operand> دو بار تکرار شده و به ترتیب مقادیر x و y را نشان می‌دهد.

$\wedge$ نشان‌دهنده علامت XOR بیتی است.

در این مثال x و y ، xor بیتی شده و حاصل در `result` ذخیره می‌شود.

عملیات Left Shift بیتی

در زبان جاوااسکریپت، عملیات Left Shift بیتی با استفاده از عملگر `>>` انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب به صورت زیر استفاده کنید:

<bitwise-left-shift-expression> $\rightarrow$ <operand> << <operand>

در این گرامر:

<bitwise-left-shift-expression>: نشان‌دهنده شروع یک عبارت Left Shift بیتی است.

<operand>: عملوندها یا اجزای عبارت Left Shift بیتی.

<>: علامت Left Shift بیتی.

این گرامر می‌گوید که یک عبارت Left Shift بیتی از یک <operand> به تعداد بیت‌ها که توسط `>>` به چپ شیفت داده می‌شود، تشکیل شده است.

برای مثال:

```
var x = 5;  
var result = x << 2;
```

در اینجا:

<bitwise-left-shift-expression> توسط $x \ll 2$ توصیف می‌شود.

<operand> یک بار تکرار شده و مقدار x را نشان می‌دهد.

>> نشان‌دهنده علامت Left Shift بیتی است.

در این مثال، x به چپ شیفت داده شده و نتیجه در `result` ذخیره می‌شود.

عملیات Right Shift بیتی

در زبان جاوااسکریپت، عملیات Right Shift بیتی با استفاده از عملگر $\gg$ انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب به صورت زیر استفاده کنید:

<bitwise-right-shift-expression> $\rightarrow$ <operand> >> <operand>

در این گرامر:

<bitwise-right-shift-expression>: نشان‌دهنده شروع یک عبارت Right Shift بیتی است.

<operand>: عملوندها یا اجزای عبارت Right Shift بیتی.

<<: علامت Right Shift بیتی.

این گرامر می‌گوید که یک عبارت Right Shift بیتی از یک <operand> به تعداد بیت‌ها که توسط $\gg$ به راست شیفت داده می‌شود، تشکیل شده است.

برای مثال:

```
var x = 10;  
var result = x >> 2;
```

در اینجا:

<bitwise-right-shift-expression> توسط 2 >> x توصیف می‌شود.

<operand> یک بار تکرار شده و مقدار x را نشان می‌دهد.

<< نشان‌دهنده علامت Right Shift بیتی است.

در این مثال، x به راست شیفت داده شده و نتیجه در result ذخیره می‌شود.

عملیات Unsigned Right Shift بیتی

در زبان جاوااسکریپت، عملیات Unsigned Right Shift بیتی با استفاده از عملگر <<< انجام می‌شود. برای تعریف گرامر این عملیات، می‌توانید از یک قالب به صورت زیر استفاده کنید:

<bitwise-unsigned-right-shift-expression> → <operand> >>> <operand>

در این گرامر:

<bitwise-unsigned-right-shift-expression>: نشان‌دهنده شروع یک عبارت Unsigned Right Shift بیتی است.

<operand>: عملوندها یا اجزای عبارت Unsigned Right Shift بیتی.

<<<: علامت Unsigned Right Shift بیتی.

این گرامر می‌گوید که یک عبارت Unsigned Right Shift بیتی از یک <operand> به تعداد بیت‌ها که توسط <<< به شکل بی علامت به راست شیفت داده می‌شود، تشکیل شده است.

برای مثال:

```
var x = -10;  
var result = x >>> 2;
```

در اینجا:

<bitwise-unsigned-right-shift-expression> توسط 2 >>> x توصیف می‌شود.

<operand> یک بار تکرار شده و مقدار x را نشان می‌دهد.

<<< نشان‌دهنده علامت Unsigned Right Shift بیتی است.

در این مثال، x به راست به شکل بی‌علامت شیفت داده شده و نتیجه در result ذخیره می‌شود.

حلقه for

برای تعریف گرامر عبارت for در زبان جاوااسکریپت، می‌توانیم از یک ساختار مشابه به صورت زیر استفاده کنیم:

<for-statement> → for (<initialization>; <condition>; <update>) <statement>

<for-statement> → for (<initialization>; <condition>; <update>) <statement> else
<statement>

در این گرامر:

<for-statement>: نشان‌دهنده شروع یک عبارت for است.

<initialization>: مرحله شروعی for که معمولاً تعیین متغیرها یا مقداردهی اولیه انجام می‌دهد.

<condition>: شرط تا زمان ادامه حلقه.

<update>: عملیات به‌روزرسانی که در هر ایتريشن اجرا می‌شود.

<statement>: بلوک کد یا عبارتی که در هر ایتريشن اجرا می‌شود.

else: اختیاری و نشان‌دهنده بلوک کدی که اگر شرط for اجرا نشود، اجرا می‌شود.

برای مثال:

```
for (var i = 0; i < 5; i++) {  
    console.log(i);  
}
```

در اینجا:

- <for-statement> توسط `for (var i = 0; i < 5; i++) { console.log(i); }` توصیف می‌شود.

- <initialization> نشان‌دهنده `var i = 0` است.

- <condition> نشان‌دهنده `i < 5` است.

- <update> نشان‌دهنده `i++` است.

- <statement> نشان‌دهنده `console.log(i);` است.

اگر بخواهیم این گرامر را با بلوک `else` همراه کنیم:

<for-statement> → `for (<initialization>; <condition>; <update>) <statement> else <statement>`

مثال:

```
for (var i = 0; i < 5; i++) {  
    console.log(i);  
} else {  
    console.log("Loop finished");  
}
```

در اینجا، `else` نشان‌دهنده بلوک کدی است که اگر شرط `for` اجرا نشود (مثلاً اگر `i < 5` از ابتدا صحیح نباشد)، آن بلوک اجرا می‌شود. در مثال فوق، این بلوک `else` به ازای شرط نقض اجرا می‌شود.

حلقه `while`

برای تعریف گرامر عبارت `while loop` در زبان جاوااسکریپت، می‌توانیم از یک ساختار به صورت زیر استفاده کنیم:

<while-loop> → `while (<condition>) <statement>`

<while-loop> → `while (<condition>) <statement> else <statement>`

در این گرامر:

<while-loop>: نشان‌دهنده شروع یک حلقه while است.

<condition>: شرط تا زمانی که حلقه ادامه دارد.

<statement>: بلوک کد یا عبارتی که در هر ایتريشن اجرا می‌شود.

else: اختیاری و نشان‌دهنده بلوک کدی که اجرا می‌شود اگر شرط while اجرا نشود.

برای مثال:

```
var i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

در اینجا:

<while-loop> توسط while (i < 5) { console.log(i); i++; } توصیف می‌شود.

<condition> نشان‌دهنده $i < 5$ است.

<statement> نشان‌دهنده console.log(i); i++; است.

اگر بخواهیم این گرامر را با بلوک else همراه کنیم:

<while-loop> → while (<condition>) <statement> else <statement>

مثال:

```
var i = 0;
while (i < 5) {
    console.log(i);
    i++;
} else {
    console.log("Loop finished");
}
```

در اینجا، `else` نشان‌دهنده بلوک کدی است که اجرا می‌شود اگر شرط `while` اجرا نشود (مثلاً اگر $i < 5$ از ابتدا صحیح نباشد). در مثال فوق، این بلوک `else` به ازای شرط نقض اجرا می‌شود.

عبارات شرطی `if`

گرامر زیر یک نمونه از گرامر مرتبط با عبارات شرطی `if` در زبان جاوااسکریپت است:

`<if-stmt>` → `if (<logical-expr>) <stmt>`

`<if-stmt>` → `if (<logical-expr>) <stmt> else <stmt>`

در این گرامر:

`<if-stmt>`: نشان‌دهنده شروع یک عبارت شرطی `if` است.

`<logical-expr>`: عبارت منطقی که بررسی می‌شود.

`<stmt>`: بلوک کد یا عبارتی که در صورت برقرار بودن شرط اجرا می‌شود.

`else`: اختیاری و نشان‌دهنده بلوک کدی که اجرا می‌شود اگر شرط `if` برقرار نباشد.

برای مثال:

```
if (x > 0) {
    console.log("x is positive");
} else {
    console.log("x is non-positive");
}
```

در این مثال:

`if (x > 0) { console.log("x is positive"); } else { console.log("x is non-positive"); }` توسط `<if-stmt>` توصیف می‌شود.

`<logical-expr>` نشان‌دهنده `x > 0` است.

`<stmt>` نشان‌دهنده `console.log("x is positive");` و `console.log("x is non-positive");` است.

حال برای جمع‌بندی، یک گرامر برای کد زیر می‌نویسیم:

```
// Initialize a variable
let total = 0;

// Use a for loop to sum numbers from 1 to 5
for (let i = 1; i <= 5; i++) {
    total += i;
}

// Check if the total is even or odd using an if clause
if (total % 2 === 0) {
    console.log(`The total ${total} is even.`);
} else {
    console.log(`The total ${total} is odd.`);
}

// Use a while loop to countdown from 5 to 1
let countdown = 5;
while (countdown > 0) {
    console.log(`Countdown: ${countdown}`);
    countdown--;
}
```

برای تبدیل کد جاوااسکریپت بالا به گرامر با توجه به قالب ارائه شده، می‌توانیم از گرامر زیر استفاده کنیم:

```
<if-stmt> → if (<logical-expr>) <stmt>  
<if-stmt> → if (<logical-expr>) <stmt> else <stmt>  
<for-loop> → for (<initialization>; <condition>; <update>) <stmt>  
<while-loop> → while (<condition>) <stmt>  
<assignment> → let <variable> = <expression>;
```

حالا کد جاوااسکریپت بالا را به گرامر تبدیل می کنیم:

```
<if-stmt> → if (<logical-expr>) <stmt>  
<for-loop> → for (<initialization>; <condition>; <update>) <stmt>  
<if-stmt> → if (<logical-expr>) <stmt> else <stmt>  
<assignment> → let total = 0;  
<for-loop> → for (let i = 1; i <= 5; i++) <stmt>  
<stmt> → total += i;  
<if-stmt> → if (total % 2 === 0) <stmt> else <stmt>  
<stmt> → console.log(`The total ${total} is even.`);  
<stmt> → console.log(`The total ${total} is odd.`);  
<assignment> → let countdown = 5;  
<while-loop> → while (countdown > 0) <stmt>  
<stmt> → console.log(`Countdown: ${countdown}`);  
<stmt> → countdown--;
```

۳-۲ یک برنامه در این زبان نوشته شود و درخت تجزیه برای آن رسم شود.

برنامه جاوا اسکریپت برای جمع دو عدد در زبان جاوااسکریپت آن به صورت زیر است:

```
javascript Copy code

// تابع برای جمع دو عدد
function addNumbers(num1, num2) {
    return num1 + num2;
}

// ورودی‌ها
const number1 = parseFloat(prompt("عدد اول را وارد کنید"));
const number2 = parseFloat(prompt("عدد دوم را وارد کنید"));

// جمع دو عدد و نمایش نتیجه
const result = addNumbers(number1, number2);
console.log(`برابر است با ${number2} و ${number1} جمع: ${result}`);
```

گرامر مستقل از متن (BNF) مربوط به آن به صورت زیر است:

```
<addition expression> ::= <number> "+" <number>

<number> ::= <digit> | <digit> <number>
<digit> ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"
```

برای این گرامر، می‌توانیم یک درخت تجزیه (Parse Tree) بسازیم. درخت تجزیه نمایش دهنده ترکیب و ساختار یک عبارت در زبان گرامر مشخص است. برای عبارت " $3 + 2$ " می‌توانیم درخت تجزیه زیر را بسازیم:

```

    <addition expression>
      /      \
    <number>  <number>
      / \    / \
    <digit> "+" <digit>
      / \    / \
      2   3

```

این درخت تجزیه نشان می‌دهد که "۳ + ۲" از دو عدد تشکیل شده است که هر کدام یک رقم دارند. در اینجا، `<addition expression>` به `<number>` ها متصل است و هر `<number>` از یک `<digit>` یا چند `<digit>` تشکیل شده است.

برنامه جاوا اسکریپت برای فاکتوریل عدد در زبان جاوا اسکریپت به صورت زیر است:

javascript

 Copy code

```
// تعریف تابع برای محاسبه فاکتوریل
function calculateFactorial(number) {
  if (number === 0 || number === 1) {
    return 1;
  } else {
    return number * calculateFactorial(number - 1);
  }
}

// تابع برای محاسبه مجموع اعداد اول تا n
function calculateSumOfPrimes(n) {
  let sum = 0;
  for (let i = 2; i <= n; i++) {
    if (isPrime(i)) {
      sum += i;
    }
  }
  return sum;
}

// تابع برای بررسی اعداد اول
function isPrime(num) {
```

```

function isPrime(num) {
  for (let i = 2; i < num; i++)
    if (num % i === 0) return false;
  return num !== 1;
}

// ورودی ها
const userInput = parseInt(prompt("Enter a number for factorial and sum of primes"));

// محاسبات
const factorialResult = calculateFactorial(userInput);
const sumOfPrimesResult = calculateSumOfPrimes(userInput);

// نمایش نتایج
console.log(`Factorial of ${userInput} is: ${factorialResult}`);
console.log(`Sum of prime numbers up to ${userInput} is: ${sumOfPrimesResult}`);

```

گرامر مستقل از متن (BNF) محاسبه فاکتوریل به صورت زیر است:

```

<factorial-program> ::= <function-definition> <user-input> <factorial-calculation>

<function-definition> ::= "function calculateFactorial(number)" "{" <factorial-base-case> "}"

<factorial-base-case> ::= "if (number <= 1) return 1;"

<user-input> ::= "const userInput = parseInt(prompt('Enter a number for factorial calculation:'));"

<factorial-calculation> ::= "const factorialResult = calculateFactorial(userInput);"
                           | "console.log('Factorial of ${userInput} is: ${factorialResult}');"

```

برای این گرامر، می‌توانیم یک درخت تجزیه (Parse Tree) بسازیم. درخت تجزیه نمایش دهنده ترکیب و ساختار یک عبارت در زبان گرامر مشخص است. برای عبارت $2 + 3$ می‌توانیم درخت تجزیه زیر را بسازیم:

```

<factorial-program>
|
<function-definition>
|           |
"function calculateFactorial(number)" "("
|           |
<factorial-base-case>      <user-input>
|                           |
"if (number <= 1) return 1;"  "const userInput = parseInt(prompt('Enter a number for factorial calculation:'));"
|                           |
<factorial-calculation>      <factorial-calculation>
|                           |
"const factorialResult = calculateFactorial(userInput);"  "console.log('Factorial of ${userInput} is: ${factorialResult}');"

```

در این درخت تجزیه، هر گره از درخت نمایانگر یک قسمت از گرامر است و از چپ به راست ساخته شده است. مثل درخت تجزیه قبلی برای جمع دو عدد، این درخت نمایانگر ساختار و ترکیب عبارت محاسبه فاکتوریل است.

۲-۴ در مورد تقدم عملگرها و همچنين وابستگي عملگرها توضيحات لازم داده شود و در مورد نحوه توصيف گرامر براي پيروي از تقدم‌هاي ذکر شده، توضيحاتي داده شود.

۲-۴-۱ تقدم عملگرا

تقدم عملگرها مشخص میکند که در محاسباتی که بیش از دو عملوند دارند ابتدا کدام عملگر اثرش را اعمال کند. عملگرها در جاوا اسکریپت در محاسبات دارای حق تقدم هستند. به عنوان مثال :

```
var number = 1 + 2 * 3 / 1;
```

اگر حق تقدم عملگرها را رعایت نکنیم و عبارت بالا را از سمت چپ به راست انجام دهیم نتیجه ۹ خواهد شد $3 = 2 + 1$ سپس $9 = 3 \times 3$ و در آخر $9 = 1/9$ اما کامپایلر با توجه به تقدم عملگرها محاسبات را انجام میدهد. برای مثال عمل ضرب و تقسیم نسبت به جمع و تفریق تقدم دارند. بنابراین در مثال فوق ابتدا عدد ۲ ضربدر ۳ و سپس نتیجه آنها تقسیم بر ۱ میشود که نتیجه ۶ به دست میآید. در آخر عدد ۶ با ۱ جمع میشود و عدد ۷ حاصل میشود. در جدول زیر تقدم برخی از عملگرهای جاوااسکریپت آمده است :

اولویت	شرکت پذیری	عملگر
بیشترین اولویت	از چپ به راست	()
1	از چپ به راست	[]
1	از چپ به راست	.
2	از راست به چپ	++
2	از راست به چپ	-
2	از راست به چپ	-
2	از راست به چپ	!
2	از راست به چپ	~
2	از راست به چپ	delete
2	از راست به چپ	new
2	از راست به چپ	typeof
2	از راست به چپ	void
3	از چپ به راست	/

3	از چپ به راست	*
3	از چپ به راست	%
4	از چپ به راست	+
4	از چپ به راست	+
4	از چپ به راست	-
5	از چپ به راست	>>
5	از چپ به راست	<<
6	از چپ به راست	>, >=
6	از چپ به راست	<, <=
7	از چپ به راست	==
7	از چپ به راست	!=
7	از چپ به راست	===
7	از چپ به راست	!==
8	از چپ به راست	&
9	از چپ به راست	^
10	از چپ به راست	
11	از چپ به راست	&&
12	از چپ به راست	
13	از چپ به راست	?:
14	از راست به چپ	=
14	از راست به چپ	*, /=, %=, +=, -, <<=, >>=, >>>=, &=, ^=, =
کمترین اولویت	از چپ به راست	,

ابتدا عملگرهای با بالاترین و سپس عملگرهای با پایینترین حق تقدم در محاسبات تأثیر میگذارند. به این نکته توجه کنید که تقدم عملگرها ++ و - به مکان قرارگیری آنها بستگی دارد (در سمت چپ یا راست عملوند باشند). به عنوان مثال :

```
var number = 3;

var number1 = 3 + ++number; //results to 7
var number2 = 3 + number++; //results to 7
```

در هر دو عبارت بالا به دلیل تقدم بالتر عملگر ++ نسبت به عملگر +، ابتدا به مقدار number یک واحد اضافه شده و ۴ میشود و سپس مقدار جدید با عدد ۳ جمع میشود و در نهایت عدد ۷ به دست میآید. برای ایجاد خوانایی در تقدم عملگرها و انجام محاسباتی که در آنها از عملگرهای زیادی استفاده میشود از پرانتز استفاده میکنیم.

```
var number = ( 1 + 2 ) * ( 3 / 4 ) % ( 5 - ( 6 * 7 ) );
```

در مثال بالا ابتدا هر کدام از عباراتی که داخل پرانتز هستند مورد محاسبه قرار میگیرند. به نکتهای در مورد عبارتی که در داخل پرانتز سوم قرار دارد توجه کنید. در این عبارت ابتدا مقدار داخلترین پرانتز مورد محاسبه قرار میگیرد یعنی مقدار ۶ ضربدر ۷ شده و سپس از ۵ کم میشود. اگر دو یا چند عملگر با حق تقدم یکسان موجود باشد ابتدا باید هر کدام از عملگرها را که در ابتدای عبارت میآیند مورد ارزیابی قرار دهید. به عنوان مثال:

```
var number = 3 * 2 + 8 / 4;
```

هر دو عملگر * و / دارای حق تقدم یکسانی هستند. بنابر این شما باید از چپ به راست آنها را در محاسبات تأثیر دهید. یعنی ابتدا ۳ را ضربدر ۲ میکنید و سپس عدد ۸ را بر ۴ تقسیم میکنید. در نهایت نتیجه دو عبارت را جمع کرده و در متغیر number قرار میدهید.

وقتی در یک عبارت محاسباتی، چند عملگر وجود داشته باشد، یک مشکل معنایی که به وجود می آید، ترتیب ارزیابی عملگرهاست. برای مثال، در عبارت $z * y + x$ آیا باید ابتدا عملگر جمع ارزیابی شود و یا عملگر ضرب؟

- بدین منظور، تقدم عملگرها تعریف می شوند. برای مثال، اگر تقدم ضرب بیشتر از جمع تعریف شود، آنگاه عملگر ضرب باید قبل از جمع ارزیابی شود.

- برای گرامر مبهم قبلی می توانیم یک گرامر غیر مبهم بنویسیم به طوری که درخت تجزیه ابتدا عملگر جمع و سپس عملگر ضرب را تجزیه کند. بدین ترتیب وقتی از برگ های درخت تجزیه برای ارزیابی یک عبارت آغاز می کنیم ابتدا عملگر ضرب را اعمال می کنیم.
- گرامر غیر مبهم زیر معادل گرامر مبهم قبلی است.

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

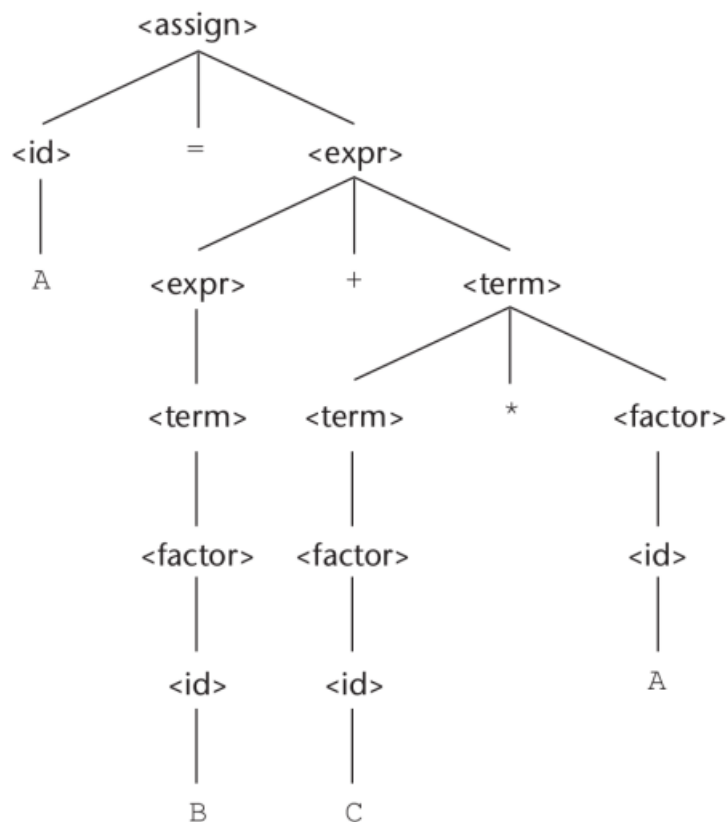
$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

$\langle \text{factor} \rangle \rightarrow (\langle \text{term} \rangle) \mid \langle \text{id} \rangle$

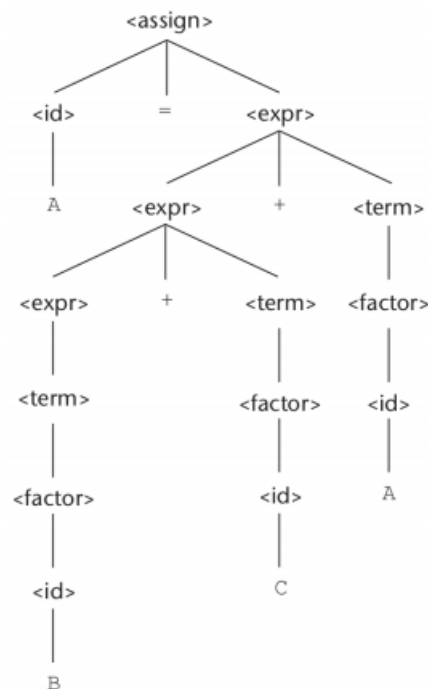
بدین ترتیب برای عبارت $A * C + B = A$ تنها یک درخت تجزیه به صورت زیر وجود خواهد داشت.



۲-۴-۲ وابستگی عملگرها

وقتی در یک عبارت دو عملگر وجود داشته باشد که تقدم برابر داشته باشند، به قوانین معنایی نیاز داریم تا مشخص می کند که در شرایطی که تقدم یکسان بدانیم کدام عملگر باید زودتر اجرا شود. وابستگی عملگرها است به کدام عملگر اولویت بالاتری داده می شود.

برای مثال عبارت $A + C + B = A$ را در نظر بگیرید. درخت تجزیه برای این عبارت طبق گرامر غیر مبهمی که قبلا ارائه شده به صورت زیر است.



در این درخت تجزیه عملگر جمع اول زودتر محاسبه می شود. این عملیات صحیح است اگر وابستگی عملگر جمع از سمت چپ باشد که معمولا هم همینطور است.

در ریاضیات می گوییم عملگر جمع خاصیت شرکت پذیری دارد، بدین معنی که وابستگی از چپ با وابستگی از راست معادل است یعنی: $(A+B) + C = A + (B+C)$

- در طراحی گرامر یک زبان برنامه نویسی وابستگی عملگرها باید در نظر گرفته شود.

- برای مثال در زبان سی تقدم عملگر * و ++ یکسان است، اما وابستگی آنها از راست به چپ است، بدین معنا که اگر این دو عملگر در کنار یکدیگر قرار بگیرند، کامپایلر ابتدا عملگر سمت راست را محاسبه می کند.
- عبارت ++p ابتدا مقدار اشاره گر را افزایش می دهد و سپس مقدار آن را ارزیابی می کند، زیرا تقدم این دو عملگر یکسان و وابستگی آنها از راست به چپ است.

۲-۴-۳ توصیف گرامر برای پیروی از تقدم

برای نوشتن گرامری که از تقدم عملگرها پیروی کند، می توانید از یک گرامر بازگشتی پویا استفاده کنید. در اینجا یک گرامر ساده برای تقدم عملگرها با استفاده از اعداد صحیح و عملگرهای جمع و ضرب به شکل BNF آورده شده است:

```
<expr> → <expr> + <term> | <term>
<term> → <term> * <factor> | <factor>
<factor> → <integer> | (<expr>)
<integer> → 0 | 1 | 2 | ... | 9
```

این گرامر به ما این امکان را می دهد که عباراتی با اعداد صحیح، جمع، و ضرب بسازیم. عملگرهای جمع و ضرب با توجه به این گرامر اولویت دارند، به این معنی که ضرب از جمع اولویت بیشتری دارد.

```
2 + 3 * (4 + 1)
```

می تواند با استفاده از این گرامر نمایش داده شود. این نوع گرامرها معمولاً در توصیف اصول محاسباتی و زبان های برنامه نویسی به کار می روند.

```
<expr> → <expr> + <term> | <expr> - <term> | <term>
<term> → <term> * <factor> | <term> / <factor> | <factor>
<factor> → <integer> | (<expr>)
<integer> → 0 | 1 | 2 | ... | 9
```

این گرامر به شما اجازه می دهد تا عباراتی با عملگرهای جمع، تفریق، ضرب، تقسیم و پرانتزها بسازید. عملگرهای با اولویت بالاتر بهترین اولویت را دارند.

$$2 * (3 + 4) - 5 / 2$$

می‌تواند با استفاده از این گرامر نمایش داده شود. این نوع گرامرها بسیار مفیدند در توصیف عبارات ریاضی و زبان‌های برنامه‌نویسی که عملیات محاسباتی مختلف را پشتیبانی می‌کنند.

```
<program>           → <statement_list>

<statement_list>     → <statement> | <statement_list> <statement>

<statement>          → <variable_declaration> | <expression_statement>

<variable_declaration> → var <identifier> = <assignment_expression>;

<expression_statement> → <expression>;

<if_statement>        → if ( <expression> ) <statement> [ else <statement> ]

<while_statement>     → while ( <expression> ) <statement>

<for_statement>       → for ( [ <expression_statement> ] ; [ <expression> ] ; <statement> )

<expression>          → <assignment_expression>

<assignment_expression> → <conditional_expression> | <left_hand_side> <assignment_operator> <expression>

<conditional_expression> → <logical_or_expression> | <logical_or_expression> <conditional_operator> <expression>

<logical_or_expression> → <logical_and_expression> | <logical_or_expression> <logical_or_operator> <expression>

<logical_and_expression> → <equality_expression> | <logical_and_expression> <logical_and_operator> <expression>

<equality_expression> → <relational_expression> | <equality_expression> <equality_operator> <expression>
```

```

<relational_expression> → <additive_expression> | <relational_expression>
<additive_expression> → <multiplicative_expression> | <additive_expression>
<multiplicative_expression> → <unary_expression> | <multiplicative_expression>
<unary_expression>      → <postfix_expression> | <unary_operator> <unary_expression>
<postfix_expression>    → <primary_expression> | <postfix_expression> [ <
<primary_expression>    → <literal> | <identifier> | ( <expression> )
<literal>               → <numeric_literal> | <string_literal> | true | false
<numeric_literal>       → <integer_literal> | <floating_point_literal>
<integer_literal>       → [0-9]+
<floating_point_literal> → [0-9]+.[0-9]+
<identifier>            → [a-zA-Z_][a-zA-Z0-9_]*
<unary_operator>        → delete | void | typeof | ++ | -- | + | - | ~ | !
<assignment_operator>   → = | *= | /= | %= | += | -= | <<= | >>= | >>>= |

```

این گرامر یک ساختار کلی از جاوا اسکریپت را نشان می‌دهد و شامل عبارات، تعریف متغیر، دستورات شرطی، حلقه‌ها، و عملگرهای مختلف است. توجه داشته باشید که این گرامر بسیار ساده‌تر از گرامر واقعی جاوا اسکریپت است.

۵-۲ معنانشناسی عملیاتی

در معنانشناسی عملیاتی معنای عبارات یک برنامه با استفاده از تأثیر اجرای آنها بر روی ماشین توصیف می‌شود.

تأثیر بر روی ماشین به معنی دنباله‌ای از تغییرات بر روی حالت ماشین است و حالت ماشین مجموعه‌ای از مقادیر بر روی حافظه آن است.

عملگر جمع (+) در جاوا اسکریپت:

زبان اسمبلی:

```
ADD destination, operand1, operand2
```

عملگر تفریق (-) در جاوا اسکریپت:

زبان اسمبلی:

```
SUB destination, operand1, operand2
```

عملگر ضرب (*) در جاوا اسکریپت:

زبان اسمبلی:

```
MUL destination, operand1, operand2
```

عملگر تقسیم (/) در جاوا اسکریپت:

زبان اسمبلی:

```
DIV destination, operand1, operand2
```

۲-۵-۱ ساختارهای تصمیم

تقریباً همه زبانهای برنامه نویسی به شما اجازه اجرای کد را در شرایط مطمئن می دهند. حال تصور کنید که یک برنامه دارای ساختار تصمیم گیری نباشد و همه کدها را اجرا کند. این حالت شاید فقط برای چاپ یک پیغام در صفحه مناسب باشد ولی فرض کنید که شما بخواهید اگر مقدار یک متغیر با یک عدد برابر باشد سپس یک پیغام چاپ شود آن وقت با مشکل مواجه خواهید شد. JavaScript راه های مختلفی برای رفع این نوع مشکلات ارائه می دهد. در این بخش با مطالب زیر آشنا خواهید شد :

• دستور if

میتوان با استفاده از دستور if و یک شرط خاص که باعث ایجاد یک کد میشود یک منطق به برنامه خود اضافه کنید. دستور if سادهترین دستور شرطی است که برنامه میگوید اگر شرطی برقرار است کد معینی را انجام بده. ساختار دستور if به صورت زیر است :

```
if (condition)
{
    code to execute;
}
```

قبل از اجرای دستور if ابتدا شرط بررسی میشود. اگر شرط برقرار باشد یعنی درست باشد سپس کد اجرا میشود. شرط یک عبارت مقایسه‌ای است. میتوان از عملگرهای مقایسه‌ای برای تست درست یا اشتباه بودن شرط استفاده کرد. اجازه بدهید که نگاهی به نحوه استفاده از دستور if در داخل برنامه بیندازیم. برنامه زیر پیغام World Hello را اگر مقدار number کمتر از ۱۰ و World Goodbye را اگر مقدار number از ۱۰ بزرگتر باشد در صفحه نمایش میدهد.

```

1 //Declare a variable and set it a value less than 10
2 var number = 5;
3
4 //If the value of number is less than 10
5 if (number < 10)
6     console.log("Hello World.");
7
8 //Change the value of a number to a value which
9 // is greater than 10
10 number = 15;
11
12 //If the value of number is greater than 10
13 if (number > 10)
14     console.log("Goodbye World.");

```

```

Hello World.
Goodbye World.

```

در خط ۲ یک متغیر با نام `number` تعریف و مقدار ۵ به آن اختصاص داده شده است. وقتی به اولین دستور `if` در خط ۵ میرسیم برنامه تشخیص میدهد که مقدار `number` از ۱۰ کمتر است یعنی ۵ کوچکتر از ۱۰ است.

منطقی است که نتیجه مقایسه درست میباشد بنابراین دستور `if` دستور را اجرا میکند (خط ۶) و پیام `World Hello` چاپ میشود. حال مقدار `number` را به ۱۵ تغییر میدهم (خط ۱۰). وقتی به دومین دستور `if` در خط ۱۳ میرسیم برنامه مقدار `number` را با ۱۰ مقایسه میکند و چون مقدار `number` یعنی ۱۵ از ۱۰ بزرگتر است برنامه پیام `World Goodbye` را چاپ میکند (خط ۱۴). به این نکته توجه کنید که دستور `if` را میتوان در یک خط نوشت :

```

if (number > 10) console.log("Goodbye World.");

```

شما میتوانید چندین دستور را در داخل دستور `if` بنویسید. کافیست که از یک آکوالد برای نشان دادن ابتدا و انتهای دستورات استفاده کنید. همه دستورات داخل بین آکوالد جز بدنه دستور `if` هستند. نحوه تعریف چند دستور در داخل بدنه `if` به صورت زیر است :

```
if (condition)
{
    statement1;
    statement2;
    .
    .
    .
    statementN;
}
```

این هم یک مثال ساده :

```
var x = 11;

if (x > 10)
{
    console.log("x is greater than 10.");
    console.log("This is still part of the if statement.");
}
```

در مثال بالا اگر مقدار x از ۱۰ بزرگتر باشد دو پیغام چاپ میشود. حال اگر به عنوان مثال آکوالد را حذف کنیم و مقدار x از ۱۰ بزرگتر نباشد مانند کد زیر :

```
var x = 5;

if (x > 10)
    console.log("x is greater than 10.");
    console.log("This is still part of the if statement.");
```

کد بالا در صورتی بهتر خوانده میشود که بین دستورات فاصله بگذاریم.

```
if (x > 10)
    console.log("x is greater than 10.");

    console.log("This is still part of the if statement.");
```

ممی بیند که دستور دوم یعنی خط آخر در مثال بالا، جز دستور if نیست. اینجاست که چون ما فرض را بر این گذاشته ایم که مقدار x از ۱۰ کوچکتر است پس خط (statement ?) Really.

if the of part still is This چاپ میشود. در نتیجه اهمیت وجود آکوالد مشخص میشود. به عنوان تمرین همیشه حتی اگر فقط یک دستور در بدنه if داشتید برای آن یک آکوالد بگذارید. فراموش نکنید که از قلم انداختن یک آکوالد باعث به وجود آمدن خطا شده و یافتن آن را سخت میکند. مثالی دیگر در مورد دستور if:

```
1 var firstNumber;
2 var secondNumb ;
```

```

4 firstNumber = parseInt(prompt("Enter a number: "));
5 secondNumber = parseInt(prompt("Enter another number: "));
6
7 if (firstNumber == secondNumber)
8 {
9     console.log(firstNumber + "==" + secondNumber);
10 }
11 if (firstNumber != secondNumber)
12 {
13     console.log(firstNumber + "!=" + secondNumber);
14 }
15 if (firstNumber < secondNumber)
16 {
17     console.log(firstNumber + "< " + secondNumber);
18 }
19 if (firstNumber > secondNumber)
20 {
21     console.log(firstNumber + "> " + secondNumber);
22 }
23 if (firstNumber <= secondNumber)
24 {
25     console.log(firstNumber + "<=" + secondNumber);
26 }
27 if (firstNumber >= secondNumber)
28 {
29     console.log(firstNumber + ">=" + secondNumber);
30 }

```

- if...else دستور

دستور if فقط برای اجرای یک حالت خاص به کار میرود یعنی اگر حالتی برقرار بود کار خاصی انجام شود. اما زمانی که شما بخواهید اگر شرط خاصی برقرار شد یک دستور و اگر برقرار نبود دستور دیگر اجرا شود باید از دستور else if استفاده کنید. ساختار دستور else if در زیر آمده است :

```
if (condition)
{
    code to execute if condition is true;
}
else
{
    code to execute if condition is false;
}
```

از کلمه کلیدی `else` نمیتوان به تنهایی استفاده کرد بلکه بدنه `else` دارید استفاده از آکوالد اختیاری است. کد داخل بلوک `else` فقط در صورتی اجرا میشود که شرط داخل دستور `if` نادرست باشد. در زیر نحوه استفاده از دستور `else...if` آمده است.

```
1  var number = 5;
2
3  //Test the condition
4  if (number < 10)
5  {
6      console.log("The number is less than 10.");
7  }
8  else
9  {
10     console.log("The number is either greater than or equal to 10.");
11 }
12
13 //Modify value of number
14 number = 15;
15
16 //Repeat the test to yield a different result
17 if (number < 10)
18 {
19     console.log("The number is less than 10.");
20 }
21 else
22 {
23     console.log("The number is either greater than or equal to 10.");
24 }
```

```
The number is less than 10.
The number is either greater than or equal to 10.
```

در خط ۱ یک متغیر به نام `number` تعریف کرده‌ایم و در خط ۴ تست میکنیم که آیا مقدار متغیر `number` از ۱۰ کمتر است یا نه و چون کمتر است در نتیجه کد داخل بلوک `if` اجرا میشود (خط ۶) و اگر مقدار `number` را تغییر دهیم و به مقداری بزرگتر از ۱۰ تغییر دهیم (خط ۱۴)، شرط نادرست میشود (خط ۱۷) و کد داخل بلوک `else` اجرا میشود (خط ۲۳). مانند بلوک `if` نباید به آخر کلمه کلیدی `else` سیمیکولن اضافه شود.

۲-۵-۲ عملگر شرطی

عملگر شرطی (?:) در جاوااسکریپت مانند دستور شرطی `else...if` عمل میکند. در زیر نحوه استفاده از این عملگر آمده است:

```
<condition> ? <result if true> : <result if false>
```

عملگر شرطی تنها عملگر سه تایی جاوااسکریپت است که نیاز به سه عملوند دارد، شرط، یک مقدار زمانی که شرط درست باشد و یک مقدار زمانی که شرط نادرست باشد. اجازه بدهید که نحوه استفاده این عملگر را در داخل برنامه مورد بررسی قرار دهیم.

```
1 var pet1 = "puppy";
2 var pet2 = "kitten";
3 var type1;
4 var type2;
5
6 type1 = (pet1 == "puppy" ) ? "dog" : "cat";
7 type2 = (pet2 == "kitten") ? "cat" : "dog";
8
9 console.log(type1);
10 console.log(type2);
```

```
dog
cat
```

برنامه بالا نحوه استفاده از این عملگر شرطی را نشان میدهد. خط ۶ به صورت زیر ترجمه میشود: اگر مقدار `pet1` برابر با `puppy` سپس مقدار `dog` را در `type1` قرار بده در غیر این صورت مقدار `cat` را `type1` قرار بده. خط ۷ به صورت زیر ترجمه میشود: اگر مقدار `pet2` برابر با `kitten` سپس مقدار `cat` را در `type2` قرار بده در غیر این صورت مقدار `dog` را با استفاده از دستور `else if` مینویسیم:

```
if (pet1 == "puppy")
    type1 = "dog";
else
    type1 = "cat";
```

زبان اسمبلی:

```
CMP operand1, operand2
JNE not_equal_label
; کد در صورت برقرار بودن شرط
JMP end_label
not_equal_label:
; کد در صورت عدم برقراری شرط
end_label:
```

۳-۵-۲ تکرار

ساختارهای تکرار به شما اجازه میدهند که یک یا چند دستور کد را تا زمانی که یک شرط برقرار است تکرار کنید. بدون ساختارهای تکرار شما مجبورید همان تعداد کدها را بنویسید که بسیار خسته کننده است. راه بهتر برای نوشتن کدهای بال استفاده از حلقه‌ها است. حلقه‌ها در جاوااسکریپت عبارتند از:

- While
- Do while
- For

• حلقه while

ابتدایی‌ترین ساختار تکرار در جاوااسکریپت حلقه While است. ابتدا یک شرط را مورد بررسی قرار میدهد و تا زمانی که شرط برقرار باشد کدهای درون بلوک اجرا میشوند. ساختار حلقه While به صورت زیر است :

```
while (condition)
{
    code to loop;
}
```

به یک متغیر شمارنده در داخل بدنه حلقه نیاز داریم. این شمارنده برای آزمایش شرط مورد استفاده قرار میگیرد و ادامه یا توقف حلقه به نوعی به آن وابسته است. این شمارنده را در داخل بدنه باید کاهش یا افزایش دهیم. در برنامه زیر نحوه استفاده از حلقه While آمده است :

```

1 var counter = 1;
2
3 while (counter <= 10)
4 {
5     console.log("Hello World!");
6     counter++;
7 }

```

اگر مقدار شمارنده یک بماند و آن را افزایش ندهیم و یا مقدار شرط هرگز false نشود یک حلقه بینهایت به وجود می‌آید. به این نکته توجه کنید که در شرط بال به جای عالمت < از <= استفاده شده است. اگر از عالمت < استفاده میکردیم کد ما ۹ بار تکرار میشد چون مقدار اولیه ۱ است و هنگامی که شرط به ۱۰ برسد false میشود چون ۱۰ > ۱۰ نیست. اگر میخواهید یک حلقه بی نهایت ایجاد کنید که هیچگاه متوقف نشود باید یک شرط ایجاد کنید که همواره درست (true) باشد.

```

while (true)
{
    //code to loop
}

```

این تکنیک در برخی موارد کارایی دارد و آن زمانی است که شما بخواهید با استفاده از دستورات break و return که در آینده توضیح خواهیم داد از حلقه خارج شوید.

زبان اسمبلی:

```

loop_start_label:
CMP counter, limit
JGE loop_end_label
; کد حلقه
INC counter
JMP loop_start_label
loop_end_label:

```

• حلقه do while

حلقه `while do` یکی دیگر از ساختارهای تکرار است. این حلقه بسیار شبیه حلقه `while` است با این تفاوت که در این حلقه ابتدا کد اجرا می شود و سپس شرط مورد بررسی قرار میگیرد. ساختار حلقه `while do` به صورت زیر است :

```
do
{
    code to repeat;
} while (condition);
```

زبان اسمبلی:

```
loop_start_label:
    ; کد حلقه

    ; اجرای شرط بررسی پایان حلقه
    ; while در حلقه JGE loop_end_label و CMP counter, limit مانند
    ; برمیگردد loop_start_label در اینجا بررسی انجام نمیشود و برنامه به صورت بی‌شرط به
    ; بازگشت به ابتدای حلقه JMP loop_start_label ;

    ; برچسب پایان حلقه
loop_end_label:
```

• حلقه `for`

یکی دیگر از ساختارهای تکرار حلقه `for` است. این حلقه عملی شبیه به حلقه `while` انجام میدهد و فقط دارای چند خصوصیت اضافی است. ساختار حلقه `for` به صورت زیر است :

```
for (initialization; condition; operation)
{
    code to repeat;
}
```

✓ مقدار دهی اولیه (`initialization`) (اولین مقداری است که به شمارنده حلقه می‌دهیم. شمارنده فقط در داخل حلقه `for` قابل دسترسی است).

- ✓ شرط (condition) در اینجا مقدار شمارنده را با یک مقدار دیگر مقایسه میکند و تعیین میکند که حلقه ادامه یابد یا نه.
- ✓ عملگر (operation) (که مقدار اولیه متغیر را کاهش یا افزایش میدهد.

زبان اسمبلی:

```
; for ابتدای حلقه
for_start_label:
    ; (initialization) عبارت ابتدایی
    ; مانند متغیرها و شروع شمارندها

    ; اجرای شرط پایان حلقه
    ; while در حلقه for_end_label و JGE counter, limit مانند
    ; می‌رویم for_end_label در اینجا بررسی انجام می‌شود و در صورت عدم برقراری شرط به

    CMP counter, limit
    JGE for_end_label

    ; کد حلقه

    ; (update) عبارت مرحله‌ای
    ; مانند افزایش یا کاهش شمارندها

    JMP for_start_label ; بازگشت به ابتدای حلقه

; for برجسب پایان حلقه
for_end_label:
```

- حلقه های تودرتو (Nested Loops)

جاوااسکریپت به شما اجازه میدهد که از حلقهها به صورت تو در تو استفاده کنید. اگر یک حلقه در داخل حلقه دیگر قرار بگیرد، به آن حلقه تو در تو گفته میشود. در این نوع حلقهها، به ازای اجرای یک بار حلقه بیرونی، حلقه داخلی به طور کامل اجرا میشود. در زیر نحوه ایجاد حلقه تو در تو آمده است :

```
for (init; condition; increment)
{
    for (init; condition; increment)
    {
        //statement(s);
    }
    //statement(s);
}

while (condition)
{
    while (condition)
```

```
do
{
    //statement(s);
    do
    {
        //statement(s);
    }
    while (condition);
}
while (condition);
```

نکته‌های که در مورد حلقه‌های تو در تو وجود دارد این است که، میتوان از یک نوع حلقه در داخل نوع دیگر استفاده کرد. مثال میتوان از حلقه for در داخل حلقه while استفاده نمود.

زبان اسمبلی:

```

; ابتدای حلقه بیرونی ;
outer_loop_start_label:
    ; عبارت ابتدایی حلقه بیرونی ;
    ; مانند متغیرها و شروع شمارندها ;

    ; اجرای شرط پایان حلقه بیرونی ;
    ; JGE outer_loop_end_label و CMP outer_counter, outer_limit مانند ;
    ; outer_loop_end_label در اینجا بررسی انجام می‌شود و در صورت عدم برقراری شرط به ;

    CMP outer_counter, outer_limit
    JGE outer_loop_end_label

; ابتدای حلقه درونی ;
inner_loop_start_label:
    ; عبارت ابتدایی حلقه درونی ;
    ; مانند متغیرها و شروع شمارندها ;

    ; اجرای شرط پایان حلقه درونی ;
    ; JGE inner_loop_end_label و CMP inner_counter, inner_limit مانند ;
    ; inner_loop_end_label در اینجا بررسی انجام می‌شود و در صورت عدم برقراری شرط به ;

    CMP inner_counter, inner_limit
    JGE inner_loop_end_label

    ; کد حلقه درونی ;

    ; عبارت مرحله‌ای حلقه درونی ;
    ; مانند افزایش یا کاهش شمارندها ;

    JMP inner_loop_start_label ; بازگشت به ابتدای حلقه درونی ;

; برچسب پایان حلقه درونی ;
inner_loop_end_label:

    ; عبارت مرحله‌ای حلقه بیرونی ;
    ; مانند افزایش یا کاهش شمارندها ;

    JMP outer_loop_start_label ; بازگشت به ابتدای حلقه بیرونی ;

; برچسب پایان حلقه بیرونی ;
outer_loop_end_label:

```

- خارج شدن از حلقه با استفاده از **break** و **continue**

گاهی اوقات با وجود درست بودن شرط می‌خواهیم حلقه متوقف شود. سؤال اینجاست که چطور این کار را انجام دهید؟ با استفاده از کلمه کلیدی **break** حلقه را متوقف کرده و با استفاده از کلمه کلیدی **continue** میتوان بخشی از حلقه را رد کرد و به مرحله بعد رفت. برنامه زیر نحوه استفاده از **continue** و **break** را نشان میدهد:

```
1 console.log("Demonstrating the use of break.\n");
2
3 for (var x = 1; x < 10; x++)
4 {
5     if (x == 5)
6         break;
7
8     console.log("Number " + x);
9 }
10
11 console.log("\nDemonstrating the use of continue.\n");
12
13 for (var x = 1; x < 10; x++)
14 {
15     if (x == 5)
16         continue;
17
18     console.log("Number " + x);
19 }
```

Demonstrating the use of break.

Number 1
Number 2
Number 3
Number 4

Demonstrating the use of continue.

Number 1
Number 2
Number 3
Number 4
Number 6
Number 7
Number 8
Number 9

در این برنامه از حلقه **for** برای نشان دادن کاربرد دو کلمه کلیدی فوق استفاده شده است اگر به جای **for** از حلقه‌های **while** و استفاده میشد نتیجه یکسانی به دست می‌آمد. همانطور که در شرط برنامه

خط ۵ آمده است، وقتی که مقدار X به عدد ۵ برسد، سپس دستور break اجرا (خط ۶) و حلقه بافاصله متوقف میشود، حتی اگر شرط $X > 10$ برقرار باشد. از طرف دیگر در خط ۱۵ حلقه for فقط برای یک تکرار خاص متوقف شده و سپس ادامه مییابد. (وقتی مقدار X برابر ۵ شود حلقه از ۵ رد شده و مقدار ۵ را چاپ نمیکند و بقیه مقادیر چاپ میشوند).

تعریف متغیر در جاوا اسکریپت: در بخش سوم به طور مفصل در مورد متغیر صحبت شده است.

زبان اسمبلی:

```
VAR variable_name
```

۴-۵-۲ تابع

تابع به شما اجازه می دهند که یک رفتار یا وظیفه را تعریف کنید و مجموعه ای از کدها است که در هر جای برنامه می توان از آنها استفاده کرد. توابع در جاوا اسکریپت و اکثر زبانهای برنامه نویسی بر دو نوعند:

۱. توابع از پیش تعریف شده

۲. توابعی که توسط کاربر تعریف می شوند.

```
1 function PrintMessage()  
2 {  
3     console.log("Hello World!");  
4 }  
5  
6 PrintMessage ();
```

```
Hello World!
```

• پارامترها و آرگومان ها

پارامترها، دادههای خامی هستند که تابع آنها را پردازش میکند و سپس اطلاعاتی را که به دنبال آن هستید در اختیار شما قرار میدهد. فرض کنید پارامترها مانند اطلاعاتی هستند که شما به یک کارمند میدهید که بر طبق آنها کارش را به پایان برساند. یک تابع میتواند هر تعداد پارامتر داشته باشد. هر پارامتر میتواند از انواع مختلف داده باشد. در زیر یک تابع با N پارامتر نشان داده شده است :

```
function FunctionName(param1, param2, ...paramN)
{
    code to execute;
}
```

زبان اسمبلی:

```
CALL function_name
```

```

.data
    # داده‌های متغیرها در اینجا قرار می‌گیرند

.text
    # کد اصلی شروع می‌شود
    .globl main

main:
    # ورودی‌ها را در رجیسترها قرار دهید
    li $a0, 5        # عدد اول
    li $a1, 7        # عدد دوم

    # تابع را فراخوانی کنید
    jal add_numbers

    # بررسی کنید $v0 نتیجه را در
    move $t0, $v0

    # انجام دیگر عملیات‌ها یا خروج
    # ...

add_numbers:
    # عملیات جمع
    add $v0, $a0, $a1

    # بازگشت از تابع
    jr $ra

```

۳- متغیرها و نوع های داده‌ای

۱-۳ در زبان جاوااسکریپت انقیاد نوع و مقدار چگونه و در چه زمانی انجام می‌شود؟

۱-۱-۳ در زبان جاوااسکریپت انقیاد نوع چگونه انجام می‌شود؟

در جاوااسکریپت، انقیاد نوع (type binding) یک مفهوم مستقیم مانند انقیاد داده نیست. با این حال، جاوااسکریپت یک زبان تایپ پویا است، به این معنی که متغیرها می‌توانند مقادیر هر نوع را بدون هیچ گونه اجرای نوع نگه دارند. این در مقایسه با زبان‌های استاتیک تایپ شده مانند جاوا یا C++، که در آن متغیرها با یک نوع خاص اعلام می‌شوند و فقط می‌توانند مقادیر آن نوع را نگه دارند کاملاً متفاوت است. هنگامی که شما یک مقدار را به یک متغیر در جاوااسکریپت اختصاص می‌دهید، نوع متغیر با نوع مقدار اختصاص داده شده تعیین می‌شود. در اینجا یک مثال ساده است:

```
1 let x = 10; // Here, the variable x is holding a number
2 x = "Hello, JavaScript!"; // Now, the same variable x is holding a string
3
```

در مثال بالا، متغیر x در ابتدا به یک عدد محدود شد، اما بعداً، بدون هیچ مشکلی به یک رشته بازگشت. این انعطاف پذیری یکی از ویژگی‌های تایپ پویا در جاوااسکریپت است.

اجبار نوع (Type coercion) نیز یک ویژگی رایج در جاوااسکریپت است، جایی که نوع یک مقدار به طور خودکار به نوع دیگری بر اساس عملیات انجام شده تبدیل می‌شود. به عنوان مثال:

```
1 let result = "3" + 4; // Here, "3" is coerced into a number and the result is "34"
2
```

در این مورد، رشته "۳" به طور خودکار به یک عدد تبدیل می‌شود زمانی که به عدد ۴ اضافه می‌شود.

۲-۱-۳ انقیاد نوع در چه زمانی انجام می‌شود؟

انقیاد نوع یا تعیین نوع به صورت پویا در زمان اجرا در جاوااسکریپت اتفاق می‌افتد. این به این دلیل است که جاوااسکریپت یک زبان تفسیری است و تعیین نوع با اجرای کد رخ می‌دهد.

نوع متغیر می‌تواند در طول اجرای برنامه تغییر کند، همان‌طور که در مثال قبلی نشان داده شده است. این ماهیت پویای انقیاد نوع برای انعطاف‌پذیری جاوااسکریپت اساسی است.

در حالی که تایپ پویا انعطاف‌پذیری را فراهم می‌کند و همچنین رخ دادن مسائل غیر منتظره مربوط به نوع را در زمان اجرا را به همراه دارد، به همین دلیل درک انقیاد نوع جاوااسکریپت و مدیریت کردن نوع‌ها به طور موثر برای نوشتن کد قوی و قابل پیش‌بینی بسیار مهم است.

بنابراین، به طور خلاصه، انقیاد نوع در جاوااسکریپت بیشتر در مورد ماهیت پویای سیستم تایپ زبان است و به صورت پویا در زمان اجرا رخ می‌دهد زیرا مقادیر به متغیرها اختصاص داده می‌شود و عملیات بر روی آن مقادیر انجام می‌شود.

۳-۱-۳ انقیاد مقدار در جاوااسکریپت چگونه و چه زمانی انجام می‌شود؟

در جاوااسکریپت، مفهوم انقیاد مقدار یک اصطلاح استاندارد در خود زبان نیست، اما می‌تواند مربوط به ایده "انقیاد یک مقدار به یک متغیر" یا "انقیاد یک مقدار به یک ویژگی یا عنصر" در زمینه چارچوب‌ها یا کتابخانه‌هایی باشد که انقیاد داده به UI را اداره می‌کنند، مانند توسعه وب.

در جاوااسکریپت، اتصال یک مقدار به یک متغیر از طریق تخصیص متغیر انجام می‌شود. هنگامی که شما یک مقدار را به یک متغیر اختصاص می‌دهید، اساساً آن مقدار را به نام متغیر متصل می‌کنید. به عنوان مثال:

```
1 let age = 25; // Here, the value 25 is bound to the variable 'age'
2 let message = "Hello, World!"; // Here, the string "Hello, World!" is bound to the variable 'message'
3
```

عمل اختصاص یک مقدار به یک متغیر، روش اساسی برای انقیاد مقادیر در جاوااسکریپت است.

- انقیاد مقادیر در زمینه UI

هنگامی که به توسعه وب و چارچوب‌هایی مانند React، Angular یا Vue.js اشاره می‌شود، مفهوم انقیاد مقدار اغلب به اتصال یک مقدار از مدل داده به یک عنصر UI مانند یک فیلد ورودی، یک منطقه متن یا هر جزء دیگر اشاره دارد.

به عنوان مثال، در React، شما می‌توانید یک مقدار را به یک عنصر ورودی با استفاده از ویژگی مقدار (value attribute) متصل کنید، مانند این:

```
1 class MyComponent extends React.Component {
2   constructor(props) {
3     super(props);
4     this.state = { inputValue: '' };
5   }
6
7   handleInputChange = (event) => {
8     this.setState({ inputValue: event.target.value });
9   }
10
11  render() {
12    return (
13      <input type="text" value={this.state.inputValue} onChange={this.handleInputChange} />
14    );
15  }
16 }
17
```

در این مثال، inputValue به مقدار عنصر ورودی محدود می‌شود، به این معنی که هنگامی که inputValue تغییر می‌کند، عنصر ورودی به روز می‌شود و هنگامی که کاربر در فیلد ورودی تایپ می‌کند، inputValue را به روز می‌کند.

- چه زمانی انقیاد داده در جاوااسکریپت انجام می‌شود؟

انقیاد داده در زمینه UI یا اتصال داده‌ها معمولاً زمانی انجام می‌شود که شما می‌خواهید تغییرات در مدل داده خود را به UI منعکس کنید و بالعکس. در چارچوب‌هایی مانند React، Angular یا Vue.js، این اغلب به عنوان بخشی از فرایند رندر اتفاق می‌افتد. هر زمان که مقدار محدود تغییر می‌کند، باعث به روز رسانی عنصر UI مرتبط می‌شود و هنگامی که کاربر با UI تعامل می‌کند، می‌تواند مقدار محدود را در مدل داده به روز کند.

زمان‌بندی انقیاد داده بستگی به چارچوب خاص و چرخه عمر آن دارد. به عنوان مثال، در React، به روز رسانی وضعیت یک جزء باعث رندر مجدد جزء و عناصر محدود آن می‌شود و UI را برای منعکس کردن حالت جدید به روز می‌کند. در Angular، تغییرات در مدل باعث ایجاد یک چرخه تشخیص تغییر می‌شود و دیدگاه را با مقادیر جدید به روز می‌کند.

به طور خلاصه، انقیاد داده در جاوااسکریپت می‌تواند به اتصال مقادیر به متغیرها یا در زمینه توسعه وب، مقادیر متصل شده به عناصر یا اجزای UI به منظور حفظ UI در همگام‌سازی با مدل داده‌های اساسی اشاره کند. این یک جنبه اصلی ساخت رابط‌های کاربری پویا و تعاملی است و نقش کلیدی در توسعه وب مدرن دارد.

۲-۳ آیا تعاریف متغیرها ضمنی است یا صریح و یا هر دو نوع تعریف وجود دارد؟ با ذکر مثال توضیح داده شود.

در جاوااسکریپت، تعاریف متغیر می‌تواند هم ضمنی و هم صریح باشد و تمایز بین این دو ریشه در نحوه تعریف و مقداردهی اولیه متغیرها است.

در جاوااسکریپت، انقیاد نوع (Type coercion) به تبدیل یک نوع داده به نوع دیگر اشاره دارد. این انقیاد ممکن است به صورت خودکار (Implicit) یا اختیاری (Explicit) انجام شود.

انقیاد نوع خودکار: (Implicit Type Coercion)

در عملیات ریاضی:

در جاوااسکریپت، اگر یک عملیات ریاضی بین دو نوع داده مختلف انجام شود، موتور جاوااسکریپت تلاش می‌کند تا به صورت خودکار نوع داده‌ها را تبدیل کند. برای مثال:

```
var x = 5;
var y = "10";
var result = x + y; // در اینجا x به رشته تبدیل می‌شود و جمع دو رشته انجام می‌شود
console.log(result); // "510"
```

در مقایسه:

در بعضی مواقع، مقایسه اشیاء با انواع مختلف باعث انجام انقیاد نوع خودکار می‌شود. برای مثال:

```
var a = 5;
var b = "5";
if (a == b) {
    // انجام می‌شود به دلیل انقیاد نوع خودکار
}
```

انقیاد نوع اختیاری: (Explicit Type Coercion)

برنامه‌نویس می‌تواند به صورت اختیاری نیز انقیاد نوع انجام دهد. این کار با استفاده از توابعی مانند `parseInt`، `parseFloat`، `String` و ... انجام می‌شود. برای مثال:

```
var x = "5";
var y = 10;
var result = parseInt(x) + y; // اینجا x به عدد تبدیل می‌شود و سپس جمع انجام می‌شود x اینجا
console.log(result); // 15
```

در کل، انقیاد نوع در جاوااسکریپت به صورت پویا و در زمان اجرا انجام می‌شود، و بستگی به عملیاتی که انجام می‌شود، ممکن است انقیاد نوع خودکار رخ دهد یا برنامه‌نویس مستقیماً انقیاد نوع انجام دهد.

انقیاد نوع در جاوااسکریپت به دلیل این که یک زبان تایپ دینامیک و از نوع خیلی ضعیف است، ممکن است در برخی مواقع به مشکلاتی منجر شود که برنامه‌نویسان باید مواظب آن باشند.

انقیاد نوع در عملیات ریاضی:

در عملیات ریاضی، انقیاد نوع خودکار می‌تواند به نتایج غیر منتظره منجر شود:

```
var x = 5;
var y = "10";
var result = x + y; // در اینجا x به رشته تبدیل می‌شود و جمع دو رشته انجام می‌شود x در اینجا
console.log(result); // "510"
```

انقیاد نوع در مقایسه:

مقایسه اعداد با رشته‌ها ممکن است منجر به نتایج غیر منتظره شود:

```
var a = 5;
var b = "5";
if (a == b) {
    // این شرط برقرار است به دلیل انقیاد نوع خودکار
}
```

انقیاد نوع اختیاری:

برنامه‌نویسان می‌توانند از توابعی مانند `parseInt` یا `parseFloat` برای انجام انقیاد نوع اختیاری استفاده کنند:

```
var x = "5";
var y = 10;
var result = parseInt(x) + y; // به عدد تبدیل شده و سپس جمع انجام می‌شود x
console.log(result); // 15
```

دوری کردن از انقیاد نوع ناخواسته:

برای جلوگیری از انقیاد نوع ناخواسته، برنامه‌نویسان معمولاً از اپراتور `===` (برابری به همه جهات و نوع داده) به جای `==` (برابری به همه جهات) در مقایسه‌ها استفاده می‌کنند. این کار باعث می‌شود که نوع داده‌ها هم بررسی شود و فقط در صورت برابری نوع‌ها، مقایسه به درستی انجام شود:

```
var a = 5;
var b = "5";
if (a === b) {
    // این شرط برقرار نمی‌شود
}
```

در کل، انقیاد نوع در جاوااسکریپت باید با دقت مدیریت شود تا اشکالات مربوط به نوع‌ها به وجود نیاید و کد به درستی اجرا شود.

۱. تعریف صریح متغیر

تعریف صریح متغیر زمانی اتفاق می‌افتد که یک توسعه‌دهنده، یک متغیر را با استفاده از کلمات کلیدی `let`، `const` یا `var` اعلام می‌کند و یک مقدار اولیه را در زمان اعلان اختصاص می‌دهد. به عنوان مثال:

```
1 let explicitVariable = 42; // This is an explicit variable definition using the 'let' keyword
2 const anotherExplicitVariable = "Hello, JavaScript!"; // This is an explicit variable definition using the 'const' keyword
3
```

در مثال بالا، هر دو متغیر به شکل صریح تعریف شده‌اند. انواع آن‌ها بر اساس نوع مقادیر اختصاص داده شده به آن‌ها (عدد و رشته و ...) تعیین می‌شوند.

۲. تعریف ضمنی متغیر

تعریف ضمنی متغیر زمانی اتفاق می‌افتد که یک مقدار به یک متغیر اختصاص داده می‌شود بدون اینکه به صراحت آن را با استفاده از `let`، `const` یا `var` اعلام کند. هنگامی که یک متغیر به طور ضمنی تعریف می‌شود، زمانی ایجاد می‌شود که شما یک مقدار را بدون اعلام صریح به آن اختصاص می‌دهید. به عنوان مثال:

```
1 implicitVariable = 10; // This is an implicit variable definition
2
```

در این مثال، `implicitVariable` به طور ضمنی تعریف می‌شود زمانی که مقدار ۱۰ به آن اختصاص داده می‌شود. جاوااسکریپت به طور خودکار نوع متغیر را بر اساس مقدار اختصاص داده شده تعیین می‌کند. این یک مثال از تایپ پویا است و قابلیت تعریف متغیر ضمنی جاوااسکریپت را نشان می‌دهد.

مهم است که توجه داشته باشید که تمام تعریف متغیرها در جاوااسکریپت به بالای تابع یا اسکریپت حاوی آن‌ها محدود میشوند. این بدان معنی است که تخصیص متغیرها در جایی که انتظار دارید اتفاق می‌افتد، تعریف واقعی به بالا محدود می‌رود. به عنوان مثال:

```
1 console.log(implicitVariable); // Outputs: undefined
2 var implicitVariable = 5;
3
```

در این مورد، متغیر `implicitVariable` به بالای دامنه خود محدود می‌شود، که منجر به کنسول می‌شود و دستور `console.log` خطا را پرتاب نمی‌کند، بلکه خروجی تعریف نشده است.

نقش `let`، `const` و `var`

- کلمه کلیدی `let` در ES6 معرفی شد (۲۰۱۵).
- متغیرهای تعریف شده با `let` نمی‌توانند دوباره تعریف شوند.

- متغیرهای تعریف شده با `let` باید قبل از استفاده تعریف شوند.

- متغیرهای تعریف شده با `let` دارای محدوده بلوک هستند.

متغیرهای تعریف شده با `let` نمی‌توانند دوباره تعریف شوند. شما نمی‌توانید به طور تصادفی یک متغیر تعریف شده با `let` را دوباره تعریف کنید. به طور مثال تعریف متغیر به شکل زیر اشتباه است :

```
1 let x = "John Doe";
2 let x = 0;
```

اما تعریف متغیر به شکل زیر صحیح است :

```
1 var x = "John Doe";
2 var x = 0;
```

- محدوده بلوک

قبل از ES6 (۲۰۱۵)، جاوااسکریپت دارای محدوده جهانی و محدوده تابع بود. در ES6 دو کلمه کلیدی مهم جدید جاوااسکریپت را معرفی شد: `let` و `const`. این دو کلمه کلیدی محدوده بلوک را در جاوااسکریپت معرفی می‌کنند. متغیرهای اعلام شده در داخل یک بلوک `{ }` از خارج از بلوک قابل دسترسی نیستند:

```
1 {
2   let x = 2;
3 }
4 // x can NOT be used here
```

متغیرهای اعلام شده با کلمه کلیدی `var` محدوده بلوک ندارند. متغیرهای اعلام شده در داخل یک بلوک `{ }` می‌توانند از خارج از بلوک قابل دسترسی باشند.

```
1 {
2   var x = 2;
3 }
4 // x CAN be used here
```

- بازتعریف متغیرها

بازتعریف یک متغیر با استفاده از کلمه کلیدی `var` می‌تواند مشکلاتی را تحمیل کند. بازتعریف یک متغیر در داخل یک بلوک همچنین متغیر خارج از بلوک را دوباره تعریف خواهد کرد:

```
1  var x = 10;
2  // Here x is 10
3
4  {
5  var x = 2;
6  // Here x is 2
7  }
8
9  // Here x is 2
```

بازتعریف یک متغیر با استفاده از کلمه کلیدی `let` می‌تواند این مشکل را حل کند. بازتعریف یک متغیر در داخل یک بلوک، متغیر خارج از بلوک را دوباره تعریف نخواهد کرد:

```
1  let x = 10;
2  // Here x is 10
3
4  {
5  let x = 2;
6  // Here x is 2
7  }
8
9  // Here x is 10
```

- تفاوت بین `var`، `let` و `const`

	Scope	Redeclare	Reassign	Hoisted	Binds this
var	No	Yes	Yes	Yes	Yes
let	Yes	No	Yes	No	No
const	Yes	No	No	No	No

- تعریف مجدد

بازتعریف یک متغیر جاوااسکریپت با `var` در هر جایی از یک برنامه مجاز است:

```
1 var x = 2;
2 // Now x is 2
3
4 var x = 3;
5 // Now x is 3
```

با `let`، بازتعریف یک متغیر در همان بلوک مجاز نیست:

```
1 var x = 2; // Allowed
2 let x = 3; // Not allowed
3
4 {
5   let x = 2; // Allowed
6   let x = 3; // Not allowed
7 }
8
9 {
10  let x = 2; // Allowed
11  var x = 3; // Not allowed
12 }
```

تعریف مجدد یک متغیر با `let`، در یک بلوک دیگر، مجاز است:

```
1 let x = 2; // Allowed
2
3 {
4   let x = 3; // Allowed
5 }
6
7 {
8   let x = 4; // Allowed
9 }
```

- کلمه کلیدی `const` در ES6 (۲۰۱۵) معرفی شد.

- متغیرهای تعریف شده با `const` نمی‌توانند مجدداً تعریف شوند.

- متغیرهای تعریف شده با `const` نمی‌توانند دوباره اختصاص داده شوند.

- متغیرهای تعریف شده با `const` دارای محدوده بلوک هستند.

- نمی‌توانند دوباره تخصیص داده شوند.

یک متغیر `const` را نمی‌توان دوباره اختصاص داد:

```
1  const PI = 3.141592653589793;
2  PI = 3.14;           // This will give an error
3  PI = PI + 10;        // This will also give an error
```

- باید تخصیص داده شوند.

به متغیرهای `const` جاوااسکریپت باید در زمان تعریف یک مقدار اختصاص داده شود. به طور مثال تعریف متغیر به شکل زیر صحیح است:

```
1  const PI = 3.14159265359;
```

اما تعریف متغیر به شکل زیر غلط است:

```
1  const PI;
2  PI = 3.14159265359;
```

- محدوده بلوک

رفتار `const` در بلوک مانند رفتار `let` است.

```
1  const x = 10;
2  // Here x is 10
3
4  {
5    const x = 2;
6    // Here x is 2
7  }
8
9  // Here x is 10
```

- بازتعریف

تعریف مجدد `var` موجود یا متغیر `let` برای `const`، در همان محدوده، مجاز نیست:

```
1  var x = 2;    // Allowed
2  const x = 2;  // Not allowed
3
4  {
5    let x = 2;   // Allowed
6    const x = 2; // Not allowed
7  }
8
9  {
10   const x = 2; // Allowed
11   const x = 2; // Not allowed
12 }
```

تخصیص مجدد یک متغیر `const` موجود، در همان محدوده، مجاز نیست:

```

1  const x = 2;      // Allowed
2  x = 2;           // Not allowed
3  var x = 2;       // Not allowed
4  let x = 2;       // Not allowed
5  const x = 2;     // Not allowed
6
7  {
8      const x = 2;  // Allowed
9      x = 2;       // Not allowed
10     var x = 2;   // Not allowed
11     let x = 2;   // Not allowed
12     const x = 2; // Not allowed
13 }

```

بازتعریف یک متغیر با `const`، در محدوده دیگری، و یا در یک بلوک دیگر، مجاز است:

```

1  const x = 2;      // Allowed
2
3  {
4      const x = 3;  // Allowed
5  }
6
7  {
8      const x = 4;  // Allowed
9  }

```

استفاده از `let` و `const` متغیر را به شکل صریح تعریف می‌کنند که اغلب به عنوان بهترین روش در نظر گرفته می‌شود زیرا به جلوگیری از انتقال مجدد تصادفی مقادیر متغیرها کمک می‌کند و محدوده سطح بلوک را فراهم می‌کند و به افزایش پیش بینی کد کمک می‌کند.

به طور خلاصه، جاوااسکریپت از تعاریف متغیر ضمنی و صریح پشتیبانی می‌کند. با تعاریف صریح، شما از کلمات کلیدی `let` یا `const` برای اعلام یک متغیر استفاده می‌کنید. در تعاریف ضمنی، یک متغیر زمانی ایجاد می‌شود که یک مقدار به آن اختصاص داده شود. درک تفاوت‌های ظریف تعریف متغیر در جاوااسکریپت می‌تواند در نوشتن کد قابل پیش بینی و قابل نگهداری کمک کند.

در جاوااسکریپت، تعریف متغیرها می‌تواند به صورت ضمنی (Implicit) یا صریح (Explicit) انجام شود. همچنین، اغلب متغیرها به صورت دینامیک (Dynamic) تعریف می‌شوند، که به این معناست که نوع متغیر ممکن است در طول زمان تغییر کند.

تعریف ضمنی:

تعریف ضمنی متغیرها به معنای این است که موتور جاوااسکریپت خودکار نوع متغیر را تشخیص داده و مقدار مناسب را تعیین می‌کند. این موضوع بیشتر در زمان انتساب (assignment) اتفاق می‌افتد.

```
var x = 5; // به طور خودکار به عنوان یک عدد تشخیص داده می‌شود x، تعریف ضمنی
var y = "Hello"; // به طور خودکار به عنوان یک رشته تشخیص داده می‌شود y، تعریف ضمنی
```

تعریف صریح:

تعریف صریح متغیرها به معنای این است که برنامه‌نویس به صورت صریح نوع متغیر را مشخص می‌کند. این کار با استفاده از کلمه‌های کلیدی مانند var، let، یا const انجام می‌شود.

```
var z; // بدون مقدار و نوع خاصی تعریف شده است z، تعریف صریح
let name = "John"; // مشخص شده است name تعریف صریح، نوع رشته برای
const PI = 3.14; // مشخص شده است و مقدار ثابت است PI تعریف صریح، نوع عدد برای
```

تعریف صریح با توجه به نوع:

تا ES6، تعریف صریح نوع در جاوااسکریپت امکان‌پذیر نبود. اما با معرفی let و const در ES6، برنامه‌نویسان می‌توانند به صورت صریح نوع متغیر را مشخص کنند.

```
let myNumber: number = 42; // تعریف صریح با توجه به نوع
let myString: string = "Hello, TypeScript!"; // تعریف صریح با توجه به نوع
```

در مثال بالا، myNumber به عنوان یک عدد و myString به عنوان یک رشته تعریف شده‌اند. این قابلیت اغلب در سیستم‌های جاوااسکریپت توسعه یافته‌تر، مانند TypeScript، مورد استفاده قرار می‌گیرد.

تعریف متغیرها در جاوااسکریپت:

۱. تعریف ضمنی:

در تعریف ضمنی، موتور جاوااسکریپت خودکار نوع متغیر را تشخیص داده و مقدار مناسب را تعیین می‌کند. این به این معناست که شما نیازی به صراحتاً مشخص کردن نوع متغیر ندارید.

```
var x = 5;           // به طور خودکار به عنوان یک عدد تشخیص داده می‌شود x
var y = "Hello";    // به طور خودکار به عنوان یک رشته تشخیص داده می‌شود y
```

۲. تعریف صریح:

تعریف صریح به این معناست که برنامه‌نویس نوع متغیر را به صورت صریح مشخص می‌کند. این عمل با استفاده از کلمات کلیدی مانند var، let یا const انجام می‌شود.

```
var z;               // بدون مقدار و نوع خاصی تعریف شده است z، تعریف صریح
let name = "John";   // مشخص شده است name تعریف صریح، نوع رشته برای
const PI = 3.14;      // مشخص شده است و مقدار ثابت است PI تعریف صریح، نوع عدد برای
```

۳. تعریف صریح با توجه به نوع:

در برخی از توسعه‌های جدید جاوااسکریپت، مانند TypeScript، این امکان وجود دارد که برنامه‌نویس نوع متغیر را صریحاً مشخص کند.

```
let myNumber: number = 42; // تعریف صریح با توجه به نوع
let myString: string = "Hello, TypeScript!"; // تعریف صریح با توجه به نوع
```

در اینجا، myNumber به عنوان یک عدد و myString به عنوان یک رشته با توجه به نوع مشخص شده‌اند.

۴. تعریف ضمنی با توجه به نوع:

از ES6 به بعد، با معرفی let و const، تعریف ضمنی نیز می‌تواند با توجه به نوع انجام شود.

```
let myVar: any = 42; // (به معنای هر نوعی که بخواهد any نوع) تعریف ضمنی با توجه به نوع
```

۵. تعریف ضمنی با استفاده از const:

از آنجایی که const متغیرها را ثابت می‌کند، تعریف ضمنی با استفاده از const ممکن است در برخی موارد باعث ایجاد کد کمتری با خطا شود.

```
const myConstVar = 42; // تعریف ضمنی یا استفاده از const
```

با این توضیحات، شما می‌توانید با استفاده از تعریف ضمنی یا صریح، به وفور با متغیرها در جاوااسکریپت کار کنید و از آن‌ها به عنوان داده‌های دینامیک و قابل تغییر بهره‌مند شوید.

۳-۳ آیا در زبان انتخاب شده متغیرهای ایستا، پویا در پشته، پویا در هیپ به طور صریح، پویا در هیپ به طور ضمنی وجود دارند؟ با مثال توضیح داده شود.

۳-۳-۱ آیا در جاوااسکریپت متغیرهای ایستا در پشته وجود دارد؟ توضیح دهید.

در جاوااسکریپت، متغیرهای ایستا به مفهوم متغیرهایی که در پشته (stack) ذخیره می‌شوند، معنا نمی‌شود. پشته اصطلاحاً برای ذخیره داده‌ها و مراجع به توابع در زمان اجرا استفاده می‌شود، اما متغیرها اصولاً در حافظه (heap) ذخیره می‌شوند.

در جاوااسکریپت، متغیرها در حافظه ذخیره می‌شوند و به طور مستقیم از پشته استفاده نمی‌کنند. پشته بیشتر برای مدیریت فرآیندهای توابع و استفاده از حافظه موقتی برای ذخیره مقادیر محلی توابع استفاده می‌شود.

در جاوااسکریپت، مفهوم "متغیرهای ایستا" به صورت صریح وجود ندارد. جاوااسکریپت یک زبان برنامه‌نویسی دینامیک است و متغیرهای آن به صورت دینامیک در حافظه ذخیره می‌شوند.

متغیرهای ایستا به طور کلی به متغیرهایی اشاره دارند که مقدار آن‌ها در طول زمان تغییر نمی‌کند. در جاوااسکریپت، این مفهوم به صورت صریح وجود ندارد. با این حال، می‌توانید از کلمه کلیدی `const` برای تعریف متغیرهای ثابت استفاده کنید که مقدار آن‌ها پس از تخصیص قابل تغییر نباشد.

مثال:

```
const pi = 3.14;
```

در این مثال، `pi` به عنوان یک متغیر ثابت تعریف شده است و مقدار آن پس از اولین تخصیص قابل تغییر نیست. این متغیر به نوعی به مفهوم متغیرهای ایستا نزدیک‌تر است، اما همچنان به عنوان یک متغیر دینامیک در جاوااسکریپت در نظر گرفته می‌شود.

بنابراین، مفهوم متغیرهای ایستا که به معنای ذخیره در پشته باشد، در جاوااسکریپت صحیح نیست. متغیرها در این زبان به صورت دینامیک در حافظه ذخیره می‌شوند و ممکن است در طول اجرا تغییر کنند.

۲-۳-۳ متغیرهای پویا در جاوااسکریپت چگونه پیاده سازی شده‌اند؟

در جاوااسکریپت، متغیرهای پویا با استفاده از `let` یا `var` پیاده‌سازی می‌شوند. این کلمات کلیدی برای تعریف متغیرهای قابل تغییر در طول اجرای برنامه به کار می‌روند. این متغیرها به عبارتی قابلیت تغییر نوع و مقدار را دارند.

```
let dynamicVar = 5;
dynamicVar = "Hello, World!";
```

در این مثال، `dynamic Var` با استفاده از `let` تعریف شده و ابتدا به یک عدد اختصاص داده شده و سپس مقدار آن به یک رشته تغییر کرده است.

• `let` در ES6 :

کلمه کلیدی `let` در ES6 معرفی شد و به خوبی از `var` جایگزین شده است. از `let` به دلیل مزایای بیشتری مانند داشتن `scope` بلوکی (`block-scoped`) استفاده می‌شود.

```
for (let i = 0; i < 5; i++) {
  console.log(i); // قابل دسترسی در اینجا i
}
console.log(i); // قابل دسترسی نیست i :خطا
```

• `var` و Hoisting :

`var` نیز همچنان در جاوااسکریپت استفاده می‌شود، اما دارای خصوصیت `hoisting` است که به این معناست که متغیرها با `var` ممکن است پیش از تعریف آن‌ها مورد دسترسی قرار گیرند.

```
console.log(myVar); // undefined
var myVar = 5;
```

در مقایسه با `let` که اینگونه کار نمی‌کند:

```
console.log(myVar); // Error: myVar is not defined
let myVar = 5;
```

استفاده در محدوده‌های مختلف:

`let` اطلاعات را در `scope` بلوکی (بین `{}`) نگه می‌دارد و این به معنای آن است که می‌توانید متغیرهایی با همین نام را در بلوک‌های مختلف بسازید.

```
if (true) {
  let x = 10;
  console.log(x); // در دسترس است x اینجا
}
console.log(x); // فایل دسترسی نیست x: خطا
```

استفاده از `let` به عنوان روش اولویتی برای تعریف متغیرها تا جای ممکن توصیه می‌شود، زیرا این کلمه کلیدی اصطلاحاً مزایای بیشتری نسبت به `var` دارد و اشکالات زیادی که در `var` وجود دارد را برطرف کرده است. استفاده از `let` باعث ایجاد کدی خواناتر و قابل ترکیب‌تر می‌شود.

متغیرهای پویا در جاوااسکریپت با استفاده از `let` و گاهی هم `var` تعریف می‌شوند. دو کلمه‌ی کلیدی `let` و `var` به شما اجازه می‌دهند تا متغیرها را تعریف کرده و به آن‌ها مقدار اختصاص دهید. این متغیرها می‌توانند در طول اجرای برنامه تغییر کنند، به خصوص اگر با `let` تعریف شده باشند.

۱. استفاده از `let` :

`let` به شما این امکان را می‌دهد که متغیرها را در `scope` بلوکی (`block scope`) تعریف کنید. این به معنای این است که متغیرها تا انتهای بلوک (بین `{}`) محدود می‌شوند.

```
if (true) {
  let x = 10;
  console.log(x); // اینجا قابل دسترسی است x
}
console.log(x); // اینجا قابل دسترسی نیست x :خطا
```

۲. استفاده از var :

همچنان در جاوااسکریپت وجود دارد، اما به دلیل خصوصیتی نظیر hoisting و قابل دسترسی در خارج از بلوکی که تعریف شده‌اند، کمتر مورد استفاده قرار می‌گیرد.

```
if (true) {
  var y = 20;
  console.log(y); // اینجا قابل دسترسی است y
}
console.log(y); // اینجا نیز قابل دسترسی است y
```

به عنوان یک نکته مهم، var ها hoisting دارند که به این معنی است که متغیرها می‌توانند پیش از تعریف آن‌ها مورد استفاده قرار گیرند.

۳. مبنای Block Scope در let :

let به شما این امکان را می‌دهد که scope بلوکی (بین {}) را ایجاد کنید. این بهبود از نظر اصول برنامه‌نویسی و ایمنی کد است.

```
for (let i = 0; i < 5; i++) {
  console.log(i); // اینجا قابل دسترسی است i
}
console.log(i); // اینجا قابل دسترسی نیست i :خطا
```

۴ Hoisting با let :

let نیز hoisting دارد، اما با این تفاوت که اگر متغیر قبلاً تعریف شده باشد ولی هنوز مقداردهی نشده باشد، به undefined تنظیم می‌شود.

```
console.log(myVar); // تعریف نشده است myVar: خطا
let myVar = 5;
```

۵. تفاوت در محدودیت‌های Block Scope :

let به عنوان یک بهترین عملکرد در محدوده‌های بلوکی استفاده می‌شود، در حالی که var به محدودیت‌های قدیمی‌تری از قبیل محدوده‌های تابعی (function scope) محدود می‌شود.

```
function example() {
  if (true) {
    var x = 10;
    let y = 20;
  }
  console.log(x); // در اینجا قابل دسترسی است x
  console.log(y); // خطا: y در اینجا قابل دسترسی نیست
}
```

در کل، به منظور ایجاد متغیرهای پویا که در طول اجرای کد تغییر می‌کنند، از let برخوردار از Block Scope استفاده کنید. استفاده از let به کدتان انعطاف بیشتری می‌دهد و اشکالات مرتبط با var را کاهش می‌دهد.

۳-۳-۳ آیا در جاوااسکریپت متغیرهای پویا در پشته وجود دارد؟ توضیح دهید.

در جاوااسکریپت، مفهوم "متغیرهای پویا در پشته" به طور کلی درست نیست. در زبان‌های برنامه‌نویسی مانند جاوا و ++C، مفهوم پویا بودن (dynamic) به ارتباط با تخصیص حافظه در زمان اجرا مرتبط است. این زبان‌ها می‌توانند به صورت پویا حافظه را تخصیص دهند یا آن را آزاد کنند.

در جاوااسکریپت، متغیرها به صورت دینامیک در حافظه ذخیره می‌شوند، اما در جاوااسکریپت، ما نیازی به مدیریت مستقیم حافظه نداریم و تمام برنامه‌ها به شکل پویا عمل می‌کنند.

در جاوااسکریپت، متغیرهای اصطلاحاً در حافظه هیپ (Heap) قرار می‌گیرند و مفهوم "متغیرهای پویا در پشته" به صورت مستقیم در این زبان وجود ندارد. در جاوااسکریپت، حافظه پویا به صورت خودکار توسط سطل زباله (garbage collectors) مدیریت می‌شود.

وقتی یک متغیر در جاوااسکریپت تعریف می‌شود و به آن مقداری اختصاص داده می‌شود، متغیر و مقدارش در حافظه هیپ قرار می‌گیرند. اگر دیگر به آن متغیر ارجاعی وجود نداشته باشد، سطل زباله پیش می‌رود و متغیرها و اشیاء غیرقابل دسترس را از حافظه حذف می‌کنند.

مثال:

```
function createObject() {  
    let obj = { data: "some data" };  
    return obj;  
}  
  
let myVar = createObject();  
// حافظه هیپ قرار دارد obj در اینجا  
  
myVar = null;  
// را آزاد کنند obj اشاره نمی‌شود و گزینه‌های زیادی ممکن است حافظه مرتبط با myVar حالا به دیگری از
```

در این مثال، obj یک متغیر است که در حافظه هیپ قرار دارد. هنگامی که myVar به null تنظیم می‌شود، دیگر به obj اشاره نمی‌شود و سطل زباله ممکن است حافظه مرتبط با obj را آزاد کنند. این فرآیند مدیریت حافظه به صورت خودکار و توسط محیط اجرایی جاوااسکریپت انجام می‌شود، بنابراین نیاز به دخالت دستی بر روی پشته وجود ندارد.

بنابراین، می‌توان گفت که در جاوااسکریپت، متغیرها به صورت پویا در حافظه ذخیره می‌شوند، اما نه به صورت مستقیم در پشته. اطلاعات مربوط به توابع و محیط اجرایی در پشته قرار دارند، اما این با مفهوم متغیرهای پویا به طور مستقیم مرتبط نیست.

۳-۳-۴ آیا در جاوااسکریپت متغیرهای پویادر هیپ به طور صریح وجود دارد؟ توضیح دهید.

در جاوااسکریپت، تا حد زیادی مدیریت حافظه به صورت خودکار انجام می‌شود و توسعه‌دهندگان نیازی به مستقیم تخصیص یا آزاد کردن حافظه ندارند. در واقع، مفهوم "متغیرهای پویا در هیپ" به طور مستقیم در جاوااسکریپت وجود ندارد.

متغیرها در جاوااسکریپت به صورت دینامیک در حافظه ذخیره می‌شوند و به صورت خودکار توسط سطل زباله مدیریت می‌شوند. سطل زباله به صورت دوره‌ای اجرا می‌شوند و اشیاء که دیگر به آن‌ها ارجاعی وجود ندارد، حذف می‌شوند.

به عبارت دیگر، توسعه‌دهندگان در جاوااسکریپت نیازی به تخصیص یا آزاد کردن حافظه به صورت صریح ندارند و این عملیات‌ها به صورت اتوماتیک انجام می‌شوند. در نتیجه، متغیرها در هیپ (heap) ذخیره می‌شوند و مدیریت حافظه به صورت متقابل با سطل زباله (Garbage Collector) انجام می‌شود.

در جاوااسکریپت، مفهوم "متغیرهای پویا در هیپ" به صورت صریح وجود ندارد. در این زبان، متغیرها به صورت دینامیک در حافظه ذخیره می‌شوند و تخصیص و آزادسازی حافظه به صورت خودکار توسط سطل زباله (garbage collectors) انجام می‌شود.

مثال:

```
function createObject() {  
    let obj = { data: "some data" };  
    return obj;  
}  
  
let myVar = createObject();  
// حافظه هیپ قرار دارد obj در اینجا  
  
myVar = null;  
// آزاد کنند obj اشاره نمی‌شود و گریه‌های زباله ممکن است حافظه مرتبط با myVar حالا به دیگری از
```

در این مثال، obj یک شیء است که در حافظه هیپ قرار دارد. هنگامی که myVar به null تنظیم می‌شود، دیگر به obj اشاره نمی‌شود و سطل زباله ممکن است حافظه مرتبط با obj را آزاد کنند. این فرآیند توسط محیط اجرایی جاوااسکریپت به صورت خودکار انجام می‌شود و نیازی به مدیریت صریح حافظه در هیپ نداریم.

بنابراین، متغیرها در جاوااسکریپت به صورت پویا در حافظه ذخیره می‌شوند و مفهوم متغیرهای پویا در هیپ به صورت صریح وجود ندارد.

۳-۳-۵ آیا در جاوااسکریپت متغیرهای پویا در هیپ به طور ضمنی وجود دارد؟ توضیح دهید.

بله، در جاوااسکریپت متغیرهای پویا (dynamic variables) در حافظه هیپ (heap) به طور ضمنی وجود دارند. زبان‌هایی که از مدیریت حافظه خودکار استفاده می‌کنند، مانند جاوااسکریپت، اشیاء (objects) به صورت پویا در حافظه هیپ ایجاد می‌شوند.

وقتی یک متغیر از نوع شیء (object) در جاوااسکریپت تعریف می‌شود، این شیء در حافظه هیپ قرار می‌گیرد. به عنوان مثال:

```
let myObject = { key: 'value' };
```

در اینجا، myObject یک متغیر است که یک شیء را نمایندگی می‌کند و این شیء در حافظه هیپ ذخیره می‌شود. سطل زباله جاوااسکریپت به طور دوره‌ای حافظه را پیمایش کرده و اشیاء که دیگر به آن‌ها ارجاعی وجود ندارد را تشخیص داده و حذف می‌کنند.

از آنجا که تخصیص و آزادسازی حافظه به صورت خودکار انجام می‌شود، توسعه‌دهندگان نیازی به نگرانی در مورد مدیریت حافظه به صورت صریح ندارند و متغیرهایشان به صورت ضمنی در حافظه هیپ ذخیره می‌شوند.

بله، در جاوااسکریپت متغیرها و اشیاء در حافظه هیپ به طور ضمنی وجود دارند. زمانی که شیء‌هایی ایجاد می‌شوند و به متغیرها اختصاص داده می‌شوند، این شیء‌ها در حافظه هیپ قرار می‌گیرند و مدیریت حافظه توسط سطل زباله انجام می‌شود.

مثال:

```
function createObject() {
  let obj = { data: "some data" };
  return obj;
}

let myVar = createObject();
// حافظه هیپ قرار دارد obj در اینجا

myVar = null;
// را آزاد کنند obj اشاره نمی‌شود و گریه‌های زباله ممکن است حافظه مرتبط با myVar حالا به دیگری از
```

در این مثال، obj یک شیء است که به آن اختصاص داده شده و در حافظه هیپ قرار دارد. وقتی myVar به null تنظیم می‌شود، دیگر به obj اشاره نمی‌شود و سطل زباله ممکن است حافظه مرتبط با obj را آزاد کنند.

این روند بهبود کارایی و مدیریت حافظه را ساده‌تر می‌کند و توسعه‌دهندگان نیازی به مدیریت صریح حافظه ندارند. حافظه‌هایی که دیگر به آن‌ها اشاره ایجاد نشده باشد، توسط سطل زباله حذف می‌شوند.

۳-۶-۳ متغیرهای ایستا در جاوااسکریپت چگونه پیاده‌سازی شده‌اند؟

در جاوااسکریپت، متغیرهای ایستا با استفاده از const پیاده‌سازی می‌شوند. کلمه‌ی const برای تعریف متغیرها به کار می‌رود که مقدار آن‌ها در طول اجرای برنامه تغییر نخواهد کرد. این کلمه کلیدی در ES6 (ECMAScript 2015) معرفی شد و تا کنون به عنوان روش اصلی برای تعریف متغیرهای ایستا در جاوااسکریپت مورد استفاده قرار می‌گیرد.

```
const pi = 3.14;
```

در این مثال، pi به عنوان یک متغیر ایستا با استفاده از const تعریف شده است. این متغیر نمی‌تواند در طول اجرای برنامه تغییر کند و اگر تلاش کنید مقدار آن را تغییر دهید، با یک خطا مواجه خواهید شد. مزایا و کاربردها:

ثبات کد: (Code Stability) استفاده از متغیرهای ایستا باعث می‌شود که مقدار متغیرها در طول زمان تغییر نکند و کد ثابت‌تر و قابل‌ترکیب‌تری ارائه شود.

پیش‌بینی مقدار: (Value Predictability) اگر یک متغیر با `const` تعریف شود، برنامه‌نویسان و تا اندازه‌ای حتی خوانندگان کد می‌توانند انتظار داشته باشند که مقدار آن تا زمان اجرای برنامه تغییر نخواهد کرد.

کد خواناتر (Readability): تعریف متغیرهای ایستا با `const` می‌تواند کد را خواناتر کند و افراد دیگری که کد شما را مشاهده می‌کنند را در درک مفهوم و مقصود کمک کند.

نکات مهم:

در صورتی که متغیری با `const` تعریف شود، باید به یکبار مقداردهی شود. این یعنی شما نمی‌توانید یک `const` بدون مقداردهی اولیه اعلام کنید.

```
const pi; // این کار یا خطا مواجه خواهد شد
```

متغیرهای ایستا با `const` می‌توانند اشیاء (Objects) و آرایه‌ها را نیز در خود نگهداری کنند. با این حال، تغییر ویژگی‌های داخلی یک شیء یا آرایه با `const` ممکن است باعث شود خطا رخ دهد، اما شناخته‌شده‌اند.

```
const myObject = { key: 'value' };  
myObject.key = 'new value'; // این کار ممکن است و خطا نخواهد داد
```

در کل، متغیرهای ایستا با استفاده از `const` یکی از ابزارهای مهم برای بهبود قابلیت خواندن و پیش‌بینی کد در جاوااسکریپت هستند.

چند نکته بیشتر در مورد متغیرهای ایستا (`const`) در جاوااسکریپت:

۱. تفاوت با `let` و `var`:

`let` برای تعریف متغیرهای محلی استفاده می‌شود که می‌توانند در یک بلوک مشخص (مثل یک حلقه یا یک شرط) تغییر کنند.

const نیز برای تعریف متغیرهای محلی استفاده می‌شود، اما مقدار آن‌ها نمی‌تواند تغییر کند. var که در ES5 معرفی شد، نحوه‌ی تعریف متغیرها را مشخص می‌کند و می‌تواند دچار مشکلاتی مانند hoisting شود.

۲. Hoisting در متغیرهای const :

متغیرهای const همچنان تحت hoisting هستند. این بدان معناست که می‌توانید یک متغیر const را پیش از تعریف آن استفاده کنید، اما به دلیل خصوصیات const که یک بار مقداردهی شده‌اند نمی‌توانند مقدار دیگری را دریافت کنند.

```
console.log(myVar); // undefined
var myVar = 5;

console.log(myConst); // Error: Cannot access 'myConst' before initialization
const myConst = 10;
```

۳. استفاده در حلقه‌ها:

استفاده از const در حلقه‌ها می‌تواند کمک کند تا از تغییر ناخواسته مقادیر جلوگیری شود.

```
for (let i = 0; i < 5; i++) {
  const square = i * i;
  console.log(square);
}
```

۴. آرایه‌ها و const :

در صورتی که یک آرایه با const تعریف شود، می‌توانید عناصر آرایه را تغییر دهید، اما نمی‌توانید یک آرایه جدید نسبت به متغیر ایجاد کنید.

```
const myArray = [1, 2, 3];
myArray.push(4); // ممکن است و خطا نخواهد داد
myArray = [5, 6, 7]; // خطا: Assignment to constant variable.
```

۵. تفاوت با ایستایی در زبان‌های دیگر:

در زبان‌های دیگر مفهوم "ایستا" به معنایی مشخص تر از آن چیزی است که در جاوااسکریپت با `const` تعبیه شده است. در جاوااسکریپت، `const` به معنای "قابل تغییر نیست" است، اما این به معنای این نیست که مقدار آن برابر با یک مقدار ثابت و ایستا است.

۶. توصیه برای استفاده از `const` :

استفاده از `const` به عنوان روش اولوی‌تری برای تعریف متغیرها تا جای ممکن توصیه می‌شود. این کمک می‌کند که شما خواناتر و قابل‌ترکیب‌تر باشد و در صورتی که مقدار یک متغیر تغییر نخواهد کرد، از این امکان بهره‌مند شوید.

۷. نامگذاری متغیرهای ایستا:

بهتر است متغیرهای ایستا با `const` نام‌هایی داشته باشند که از حروف بزرگ تشکیل شده و به صورت `snake_case` یا `camelCase` نوشته شوند تا در کد به راحتی قابل تشخیص باشند.

```
const MAX_COUNT = 100;  
const myConstantVariable = 42;
```

با استفاده از متغیرهای ایستا، می‌توانید کد خود را از اشکالات مرتبط با تغییر ناخواسته مقادیر در طول زمان محافظت کنید و قابلیت خواندن و نگهداری کد خود را افزایش دهید.

۳-۴ مقایسه سرعت تخصیص متغیرهای ایستا و پویا را در جاوااسکریپت

در جاوااسکریپت، سرعت تخصیص متغیرهای ایستا با (`const` و متغیرهای پویا) با `let` یا `var` تأثیر مستقیمی بر عملکرد ندارد، زیرا جاوااسکریپت یک زبان اجرایی (`interpreted language`) است و نه یک زبان کامپایل شده (`compiled language`). در زبان‌های کامپایل شده، تفاوت در تخصیص متغیرها ممکن است به دلیل فاصله زمانی کمتر بین تولید کد و اجرا مشهود باشد، اما در جاوااسکریپت، این تفاوت به دلیل ماهیت دینامیک و تفسیری زبان کمتر قابل مشاهده است.

در واقع، در محیط اجرایی جاوااسکریپت، به طور کلی اهمیت زیادی به نحوه تعریف متغیر (ایستا یا پویا) نمی‌دهند و بیشتر به الگوها و عملکرد کد توجه می‌کنند.

متغیرهای ایستا با `const` تعریف شده‌اند و می‌توانند بهترین عملکرد را در برخی موارد ارائه دهند. از طرف دیگر، متغیرهای پویا با `let` یا `var` قابلیت انجام تغییرات دارند که در برخی موارد انعطاف‌پذیری بیشتری ارائه می‌دهد.

توصیه می‌شود که در انتخاب نوع متغیر بر اساس نیازها و اهداف کد نظر داشته باشید تا برنامه‌ی شما خوانا تر و قابل‌ترکیب‌تر باشد. اهمیت اصلی باید به خوانا‌تری و نگهداری کد داده شود تا بهینه‌سازی‌های جزئی در این زمینه تأثیر محدودی داشته باشند.

سرعت تخصیص متغیرها در جاوااسکریپت به طور کلی از نحوه اجرای کد و ماهیت زبان تفسیری (interpreted) آن تأثیر می‌پذیرد. در زبان‌های تفسیری مانند جاوااسکریپت، کد به صورت مستقیم توسط موتور اجرایی (مانند موتور V8 در مرورگرها یا موتور (Node.js) تفسیر می‌شود و نیازی به یک مرحله کامپایل (compile) قبل از اجرا ندارد.

بنابراین، تفاوت در تخصیص متغیرهای ایستا (`const`) و پویا (`let` یا `var`) در جاوااسکریپت به نظر می‌رسد که تأثیر مستقیمی بر عملکرد کد نداشته باشد. مهم‌ترین نکته این است که متغیرها به صورت دینامیک تایپ (Dynamic Typing) در جاوااسکریپت تعریف می‌شوند که به برنامه‌نویس اجازه می‌دهد تا در طول زمان نوع متغیرها را تغییر دهد.

```
// تعریف متغیرهای ایستا
const staticVar = 10;

// تعریف متغیرهای پویا
let dynamicVar = 20;
```

در این مثال، هر دو `staticVar` و `dynamicVar` به صورت پویا تعریف شده‌اند، اما `staticVar` با `const` تعریف شده که نشان‌دهنده این است که مقدارش تا زمان اجرای برنامه تغییر نخواهد کرد. این تغییر در تعریف متغیرها به تنهایی تأثیر چندانی بر سرعت اجرا نخواهد گذاشت.

برای بهینه‌سازی کد، می‌توانید تلاش کنید که از متغیرهای ایستا (const) در جاهایی که مقدار ثابت استفاده می‌شود و از متغیرهای پویا (let یا var) در جاهایی که نیاز به تغییر مقدار وجود دارد، بهره ببرید. اما تفاوت در سرعت تخصیص به میزان قابل توجهی نبوده و بیشتر توجه به خوانایی و نگهداری کد باید داده شود.

۳-۵ آیا حوزه تعریف در زبان جاوااسکریپت ایستا است یا پویا؟ با ذکر مثال توضیح دهید.

در جاوااسکریپت، دامنه تعاریف متغیر در درجه اول استاتیک است، زیرا بر اساس جایی که متغیرها در کد اعلام می‌شوند و نه در شرایط زمان اجرا تعیین می‌شوند. با این حال، ماهیت پویای جاوااسکریپت همچنین رفتارهای خاصی را معرفی می‌کند که می‌تواند به صورت پویا در مورد این کلمه کلیدی و توابع تو در تو در نظر گرفته شود.

- محدوده استاتیک

در محدوده استاتیک (همچنین به عنوان محدوده واژگانی شناخته می‌شود)، دامنه یک متغیر با مکان آن در کد منبع و زمینه واژگانی که در آن اعلام شده است تعیین می‌شود. موتور جاوااسکریپت در زمان کامپایل می‌داند که باید به کدام محدوده متغیر برای حل یک متغیر نگاه کند. به عنوان مثال:

```
1 function myFunction() {  
2   let localVar = 10;  
3  
4   function nestedFunction() {  
5     console.log(localVar); // Accesses localVar from the outer scope (lexical scoping)  
6   }  
7  
8   nestedFunction();  
9 }  
10  
11 myFunction();  
12
```

در این مورد، NestedFunction به دلیل محدوده استاتیک به localVar دسترسی دارد. محیط واژگانی NestedFunction شامل دامنه myFunction است که به آن اجازه دسترسی به localVar را می‌دهد.

- محدوده داینامیک

در حالی که محدوده متغیر عمدتاً استاتیک است، استفاده از کلمه کلیدی `this` در جاوااسکریپت نوعی رفتار دامنه پویا را معرفی می‌کند. مقدار `this` به صورت پویا بر اساس چگونگی فراخوانی یک تابع تعیین می‌شود، نه جایی که تعریف می‌شود. این رفتار به انعطاف پذیری جاوااسکریپت کمک می‌کند. مثال زیر را در نظر بگیرید:

```
1 function dynamicThisExample() {  
2   console.log(this);  
3 }  
4  
5 dynamicThisExample(); // this will refer to the global object in non-strict mode, or be undefined in strict mode  
6
```

مقدار `this` در `dynamicThisExample` بستگی به این دارد که چگونه تابع فراخوانی می‌شود، و آن را به یک شکل از محدوده پویا در زمینه اجرای تابع تبدیل می‌کند.

به طور خلاصه، دامنه تعاریف متغیر در جاوااسکریپت در درجه اول استاتیک است. با این حال، رفتار پویای کلمه کلیدی `this`، شکلی از محدوده پویا را در توابع معرفی می‌کند. درک تعامل بین محدوده واژگانی، کلمه کلیدی `this` برای نوشتن کد جاوااسکریپت قابل پیش بینی و قابل نگهداری بسیار مهم است.

۳-۶ پیاده‌سازی حوزه تعریف پویا در جاوااسکریپت

اضافه کردن حوزه تعریف پویا (Dynamic Scope) به جاوااسکریپت یک تغییر اساسی در طراحی زبان است و نیاز به تغییرات گسترده‌ای دارد. در حال حاضر، جاوااسکریپت از حوزه تعریف لکسیکال (Lexical Scope) استفاده می‌کند که به عنوان `Scope` یا `Scope Chain` نیز شناخته می‌شود. در حوزه تعریف لکسیکال، `Scope`ها بر اساس مکان تعریف توابع در کد قرار دارند و از این رو، دسترسی به متغیرها بر اساس نحوه تو در تویی از یکدیگر تعیین می‌شود.

برای ایجاد حوزه تعریف پویا، نیاز به تغییرات در نحوه عملکرد `Scope`ها و مکانیسم تعیین متغیرها داریم. این تغییرات به شدت تأثیرگذار بوده و احتمالاً با ساختار فعلی جاوااسکریپت تداخل دارد. با این حال، می‌توانیم یک سناریو ساده را برای نمونه توضیح دهیم:

سناریو ساده:

فرض کنید که می‌خواهیم یک ویژگی تعریف پویا به جاوااسکریپت اضافه کنیم. برای این کار، یک نحوه اجرا و تعیین Scope دیگر بر اساس اجراها ایجاد می‌کنیم.

ایده اصلی:

متغیرهای تعریف شده در یک تابع به صورت پویا بر اساس محیط اجرای فراخوانی‌ها تغییر کنند.

تابع‌ها اطلاعات Scope خود را در هر فراخوانی ذخیره کنند.

اگر یک متغیر در محیط اجرا وجود نداشته باشد، به محیط اجرای قبلی نگاه کند.

پیاده‌سازی ساده:

```
let globalVar = 10;

function example() {
  console.log(globalVar); // مقدار گرفته شود (global) از محیط اجرای فراخوانی قبلی
  let localVar = 20;
  console.log(localVar); // از محیط اجرای فراخوانی فعلی گرفته شود localVar
}

function anotherExample() {
  let localVar = 30;
  example(); // این تابع متغیرها را از محیط اجرای فراخوانی خود بگیرد
}

example(); // خروجی: 20 10
anotherExample(); // خروجی: 30 10
```

این کد ساده نمایانگر یک ایده مبتنی بر حوزه تعریف پویا است. در هر فراخوانی تابع، متغیرها از محیط اجرای آن تابع گرفته شده و اگر یک متغیر در محیط اجرای فعلی وجود نداشته باشد، به محیط اجرای قبلی نگاه می‌کنند.

جاوااسکریپت در حال حاضر از حوزه تعریف لکسیکال استفاده می‌کند که به این معناست که Scope ها بر اساس مکان تعریف توابع در کد قرار دارند. این به این معناست که دسترسی به متغیرها بر اساس نحوه تو در تویی از یکدیگر تعیین می‌شود. حتی اگر یک تابع در داخل یک تابع دیگر تعریف شده باشد، Scope آن تابع داخلی محدود به Scope تابع بیرونی خود خواهد بود.

برای اجرای این تغییر و افزودن حوزه تعریف پویا، نیاز به تغییرات گسترده‌ای در موتورهای اجرای جاوااسکریپت مثل V8 که توسط Google برای مرورگر Chrome ایجاد شده است و استانداردهای جاوااسکریپت داریم. این تغییرات بسیار پیچیده و تأثیرگذار هستند و نیاز به تعیین استانداردهای جدید، تغییرات در موتورهای جاوااسکریپت، و حتی اصلاح یا تغییر در کدهای موجود دارند.

یک پیاده‌سازی واقعی این تغییرات بسیار پیچیده و بسیار وسیع خواهد بود و احتمالاً نیاز به مشارکت جامع از سوی توسعه‌دهندگان و تصمیم‌گیران جاوااسکریپت دارد. در زمینه توسعه زبان‌های برنامه‌نویسی، تغییرات اساسی مانند این نیاز به بررسی دقیق، آزمایش، و تأثیرات کلان روی اکوسیستم جاوااسکریپت دارند.

در حال حاضر، استفاده از حوزه تعریف لکسیکال در جاوااسکریپت به عنوان یک استاندارد تقریباً ثابت شده است و تغییر آن به یک سیاق بزرگ‌تر نیاز دارد که از ابتدا در طراحی زبان لحاظ شود.

۷-۳ بلوک‌ها در جاوااسکریپت چگونه تعریف شده‌اند؟

در جاوااسکریپت، بلوک‌ها (blocks) به صورت گروهی از ادامه دادن دستورات می‌باشند که می‌توانند به عنوان یک واحد اجرایی در نظر گرفته شوند. بلوک‌ها در اغلب موارد با دستورات شرطی (مانند if یا else) و حلقه‌ها (مانند for یا while) مرتبط هستند. یک بلوک با استفاده از زوج آکولاد {} تعریف می‌شود.

مثال از یک بلوک در یک دستور شرطی (if):

```

if (condition) {
    // دستورات اجرایی اگر شرط درست باشد
    console.log("شرط درست است");
} else {
    // دستورات اجرایی اگر شرط درست نباشد
    console.log("شرط درست نیست");
}

```

یک مثال دیگر از یک بلوک در یک حلقه (for):

```

for (let i = 0; i < 5; i++) {
    // دستورات اجرایی در هر مرحله از حلقه
    console.log("مقدار i: " + i);
}

```

در اینجا، بلوک داخلی هر دستور if یا for توسط آکولادها {} تعیین می‌شود. دستورات داخل بلوک باید توسط آکولادها محصور شوند، حتی اگر تنها یک دستور باشند. استفاده از آکولادها از ابهام جلوگیری می‌کند و کد را برای خوانایی بهتر افزایش می‌دهد.

در جاوااسکریپت، بلوک‌ها با استفاده از آکولادها {} تعریف می‌شوند و دستورات داخل آکولادها به عنوان یک واحد اجرایی در نظر گرفته می‌شوند. زمانی که یک دستور شرطی یا یک حلقه تعریف می‌شود، دستورات متعلق به آن دستور داخل آکولادها قرار می‌گیرند.

مثال ۱: دستور شرطی (if-else)

```

let x = 10;

if (x > 5) {
    console.log("بزرگتر از 5 است x");
} else {
    console.log("کوچکتر یا مساوی 5 است x");
}

```

در این مثال، دستورات console.log داخل بلوک if و else قرار دارند.

مثال ۲: حلقه (for)

```
for (let i = 0; i < 5; i++) {  
  console.log("مقدار i: " + i);  
}
```

در این مثال، دستور console.log داخل بلوک حلقه for قرار دارد. بلوک اجرایی حلقه با {} شروع و پایان می‌یابد.

مثال ۳: تعریف تابع

```
function myFunction() {  
  console.log("این یک تابع است");  
  // دستورات دیگر می‌توانند اینجا قرار گیرند  
}
```

در تعریف یک تابع نیز، دستورات تابع داخل یک بلوک آکولاد قرار دارند.

استفاده از بلوک‌ها باعث بهبود خوانایی کد، حفظ ساختار و قابلیت اطمینان در اجرای بخش‌های مختلف کد می‌شود. آکولادها به تعداد یکتا دستورات مربوطه را مشخص می‌کنند و از اشکالات جلوگیری می‌کنند.

۳-۸ آیا کلمات کلیدی ویژه‌ای برای اعمال تغییر در حوزه تعریف متغیرها در جاوااسکریپت وجود دارند؟

در جاوااسکریپت، متغیرها می‌توانند با استفاده از سه کلمه کلیدی var، let و const تعریف شوند. این کلمات کلیدی تفاوت‌هایی در تعریف متغیرها دارند:

۱. var:

var یک متغیر را تعریف می‌کند و از آن به عنوان متغیر تابعی استفاده می‌کند (function-scoped).

ممکن است به هر نقطه‌ای در داخل یک تابع دسترسی داشته باشد، حتی اگر داخل بلوک شرطی یا حلقه باشد.

```
function example() {
  var x = 10;
  if (true) {
    var y = 20;
    console.log(x); // دسترسی به x
  }
  console.log(y); // دسترسی به y
}
```

```
if (true) {
  var x = 10;
}
console.log(x); // قابل دسترسی است x
```

۲. let:

let متغیرها را به عنوان متغیرهای بلوکی تعریف می‌کند (block-scoped).

این به این معناست که دسترسی به let در محدوده تعریف شده (بلوک، شرط، حلقه) محدود است.

```
function example() {
  let x = 10;
  if (true) {
    let y = 20;
    console.log(x); // دسترسی به x
  }
  console.log(y); // قابل دسترسی نیست y: خطا
}
```

```
if (true) {
  let y = 20;
}
console.log(y); // در اینجا قابل دسترسی نیست y: خطا
```

۳. const:

const نیز متغیرها را به عنوان متغیرهای بلوکی تعریف می‌کند.

اما با این تفاوت که یکبار مقدار دهی شده‌اند، نمی‌توان آن‌ها را مجدداً تغییر داد.

```
const PI = 3.14;  
PI = 3.14159; // مقدار: const تغییر داد
```

به کمک این کلمات کلیدی، توسعه‌دهندگان در جاوااسکریپت می‌توانند متغیرهای خود را با توجه به نیازهای مختلف و محدودیت‌ها تعریف کنند.

۳-۹ در مورد همه نوع های داده‌ای در زبان انتخاب شده توضیح داده شود.

متغیر: متغیر مکانی از حافظه است که شما می‌توانید مقادیری را در آن ذخیره کنید. می‌توان آن را به عنوان یک ظرف تصور کرد که داده‌های خود را در آن قرار داده‌اید. محتویات این ظرف می‌تواند پاک شود یا تغییر کند. هر متغیر دارای یک نام نیز هست. که از طریق آن می‌توان متغیر را از دیگر متغیرها تشخیص داد و به مقدار آن دسترسی پیدا کرد. همچنین دارای یک مقدار می‌باشد که می‌تواند توسط کاربر انتخاب شده باشد یا نتیجه یک محاسبه باشد. مقدار متغیر می‌تواند تهی نیز باشد. متغیر دارای نوع نیز هست بدین معنی که نوع آن با نوع داده‌ای که در آن ذخیره می‌شود یکی است.

متغیر دارای عمر نیز هست که از روی آن می‌توان تشخیص داد که متغیر باید چقدر در طول برنامه مورد استفاده قرار گیرد. و در نهایت متغیر دارای محدوده استفاده نیز هست که به شما می‌گوید که متغیر در چه جای برنامه برای شما قابل دسترسی است. ما از متغیرها به عنوان یک انبار موقتی برای ذخیره داده

استفاده می‌کنیم. هنگامی که یک برنامه ایجاد می‌کنیم احتیاج به یک مکان برای ذخیره داده، مقادیر یا داده‌هایی که توسط کاربر وارد می‌شوند، داریم. این مکان، همان متغیر است.

- نام متغیر باید با یکی از حروف الفبا (a-z or A-Z) یا علامت _ شروع شود.
- نمی‌تواند شامل کاراکترهای غیرمجاز مانند \$, ^, ?, # باشد.
- نمی‌توان از کلمات رزرو شده در جاوااسکریپت برای نام متغیر استفاده کرد.
- نام متغیر نباید دارای فضای خالی (spaces) باشد.
- اسامی متغیرها نسبت به بزرگی و کوچکی حروف حساس هستند. در جاوااسکریپت دو حرف مانند a و A دو کاراکتر مختلف به حساب می‌آیند.

دو متغیر با نام‌های my Number و My Number دو متغیر مختلف محسوب می‌شوند چون یکی از آن‌ها با حرف کوچک m و دیگری با حرف بزرگ M شروع می‌شود. متغیر دارای نوع هست که نوع داده‌ای را که در خود ذخیره می‌کند را نشان می‌دهد. در مورد انواع داده‌ها در جاوااسکریپت توضیح می‌دهیم. لیست کلمات کلیدی جاوااسکریپت، که نباید از آن‌ها در نامگذاری متغیرها استفاده کرد در زیر آمده است:

abstract	arguments	*await	boolean
break	byte	case	catch
char	*class	const	continue
debugger	default	delete	do

double	else	*enum	eval
*export	*extends	false	final
finally	float	for	function
goto	if	implements	*import
in	instanceof	int	interface
*let	long	native	new
null	package	private	protected
public	return	short	static
*super	switch	synchronized	this
throw	throws	transient	true
try	typeof	var	void
volatile	while	with	yield

متغیرهای جاوا اسکریپت را می توان به ۴ روش اعلام کرد:

بطور خودکار

با استفاده از var

با استفاده از let

با استفاده از const

کلمه کلیدی var در تمام کدهای جاوا اسکریپت از سال ۱۹۹۵ تا ۲۰۱۵ استفاده شده است. کلمات کلیدی let و const در سال ۲۰۱۵ به جاوا اسکریپت اضافه شدند. کلمه کلیدی var فقط باید در کد نوشته شده برای مرورگرهای قدیمی استفاده شود.

چه زمانی از var، let یا const استفاده کنیم؟

۱. همیشه متغیرها را اعلام کنید

۲. اگر مقدار نباید تغییر کند، همیشه از const استفاده کنید

۳. اگر نوع نباید تغییر کند (آرایه‌ها و اشیاء) همیشه از `const` استفاده کنید.

۴. فقط اگر نمی‌توانید از `const` استفاده کنید از `let` استفاده کنید

۵. فقط در صورتی از `var` استفاده کنید که باید از مرورگرهای قدیمی پشتیبانی کنید.

همه متغیرهای جاوا اسکریپت باید با نام‌های منحصر به فرد شناسایی شوند. این اسامی منحصر به فرد را شناسه می‌نامند. شناسه‌ها می‌توانند نام‌های کوتاه (مانند `x` و `y`) یا نام‌های توصیفی‌تر (سن، مجموع، حجم کل) باشند. قوانین کلی برای ساخت نام برای متغیرها (شناسه‌های منحصر به فرد) عبارتند از:

نام‌ها می‌توانند شامل حروف، اعداد، زیرخط و علامت دلار باشند.

نام‌ها باید با یک حرف شروع شوند.

نام‌ها همچنین می‌توانند با `$` و `_` شروع شوند (اما در این آموزش از آن استفاده نمی‌کنیم).

نام‌ها به حروف بزرگ و کوچک حساس هستند و متغیرهای متفاوتی هستند.

کلمات رزرو شده (مانند کلمات کلیدی جاوا اسکریپت) را نمی‌توان به عنوان نام استفاده کرد.

شناسه‌های جاوا اسکریپت به حروف بزرگ و کوچک حساس هستند. در جاوا اسکریپت، نوع‌های داده متغیرها به طور پویا تعیین می‌شوند، یعنی یک متغیر می‌تواند در طول اجرای برنامه از یک نوع به نوع دیگر تغییر کند. این ویژگی اغلب به عنوان "تایپ پویا" یا "dynamic typing" شناخته می‌شود.

انواع داده جاوا اسکریپت

جاوا اسکریپت دارای ۱۰ نوع داده است:

۱. String

```
let greeting = "Hello, World!";
```

۲. Number: اعداد صحیح و اعشاری

۳. BigInt: برای اعداد صحیح بزرگ

```
let integerNumber = 42;  
let floatingNumber = 3.14;  
let bigIntNumber = 123456789012345678901234567890n;
```

۴. Boolean

```
let isTrue = true;  
let isFalse = false;
```

۵. Undefined: نیز یک مقدار خاص در جاوا اسکریپت است که نشان‌دهنده این است که یک متغیر تعریف شده است، اما به آن هیچ مقداری اختصاص نیافته است یا یک ویژگی موجود در یک شیء تعیین نشده است. در موارد بسیاری، `undefined` به عنوان مقدار پیش‌فرض برای متغیرها استفاده می‌شود، اگر مقداری به آن اختصاص داده نشده باشد. این همچنین ممکن است در مواردی که یک تابع یک مقدار را بازگردانده نشود، به عنوان مقدار بازگردانده‌شده توسط تابع نیز دیده شود. در کل، `undefined` به نوعی نشان‌دهنده‌ی عدم تعریف یا تخصیص مقدار به یک متغیر یا ویژگی است.

```
let myVar;  
console.log(myVar); // دارد undefined این خروجی
```

۶. Null: در جاوا اسکریپت یک مقدار خاص است که به عنوان نشانه‌ای برای نشان دادن عدم وجود یک مقدار یا مقدار خالی استفاده می‌شود. در واقع، `null` نشان‌دهنده‌ی این است که یک متغیر یا یک ویژگی از یک شیء به صورت همدوسی مقداردهی نشده است.

```
let myVar = null;
```

۷. **Symbol**: نیز یک نوع داده در جاوا اسکریپت است. این نوع داده برای ایجاد یک مقدار یکتا (unique) و غیرقابل تغییر (immutable) به منظور استفاده در عناوین ویژگی‌های شیء یا ایجاد کلیدهای منحصر به فرد در مجموعه‌ها (Sets) استفاده می‌شود.

```
const mySymbol = Symbol('description');
```

Symbol به دلیل یکتایی و غیرقابل تغییر بودن، می‌تواند به خوبی برای جلوگیری از تداخل نام در ویژگی‌ها یا کلیدهای مجموعه‌ها مورد استفاده قرار گیرد.

```
const mySymbol1 = Symbol('key');
const mySymbol2 = Symbol('key');

console.log(mySymbol1 === mySymbol2); // false
```

در مثال بالا، هر **Symbol** به عنوان یک مقدار منحصر به فرد در نظر گرفته می‌شود.

۸. **Object**: اشیاء یا شیء (Objects) در جاوا اسکریپت یک نوع داده است که به شما این امکان را می‌دهد که یک مجموعه از ویژگی‌ها یا ویژگی و مقدارهای مرتبط را در یک واحد ذخیره کنید. این به شما این امکان را می‌دهد که اطلاعات را به صورت ساختاری و منظم نگه دارید.

```
let person = {
  name: "John",
  age: 30,
  isStudent: false,
  greet: function() {
    console.log("Hello, I'm " + this.name);
  }
};
```

در این مثال، **person** یک شیء است که ویژگی‌های **age**، **name** و **isStudent** دارد. همچنین یک تابع به نام **greet** نیز به عنوان یکی از ویژگی‌ها در آن وجود دارد. می‌توانیم به ویژگی‌های یک شیء به صورت مستقیم دسترسی پیدا کنید:

```
console.log(person.name); // "John"
console.log(person.age); // 30
```

همچنین می‌توانید توابع را از طریق شیء صدا بزنید:

```
person.greet(); // "Hello, I'm John"
```

۹. **Array**: یک مجموعه از عناصر.

```
let fruits = ["apple", "orange", "banana"];
```

داده	توضیح
عددی (Numeric)	شامل اعداد مثبت و منفی (اعشاری و صحیح) می باشد.
رشته ای (String)	به مجموعه ای از کاراکترها که بین دو علامت کوتیشن یا دابل کوتیشن قرار گرفته باشند، اطلاق می شود.
آرایه (Array)	مجموعه ای از آیتم ها هستند که بین دو علامت [] قرار گرفته و با علامت کاما (,) از هم جدا شده اند.

مجموعه ای از آیتم ها هستند که به صورت کلید و مقدار بوده، بین دو علامت {} قرار گرفته، و با علامت کاما (,) از هم جدا شده اند.	شیء (Object)
شامل دو مقدار true یا false می باشد.	boolean
یک مقدار خاص است که به معنای هیچ چیز، خالی و یا مقدار ناشناخته است.	Null
اگر متغیری تعریف شود و مقداردهی اولیه نشود آنگاه مقدار آن برابر با undefined خواهد بود	undefined

استفاده از متغیرها

بر خلاف زبان هایی مثل جاوا و سی شارپ، که هنگام تعریف متغیر باید نوع متغیر را هم مشخص می کردیم، در JavaScript کافیت که ابتدا کلمه کلیدی var و سپس نام متغیر را نوشته و به وسیله علامت مساوی یک مقدار به آن اختصاص دهیم :

`var variableName = Value`

در مثال زیر نحوه تعریف و مقداردهی متغیرها نمایش داده شده است :

```

1 var numericVar = 10
2 var boolVar = true
3 var StringVar = "Hello World!"
4 var arrayVar = [1, 5, 8]
5 var objectVar = { 'Name': 'jack', 'family': 'Scalia', 'Age': 7 }
6
7 console.log("numericVar = ", numericVar)
8 console.log("boolVar = ", boolVar)
9 console.log("StringVar = ", StringVar)
10 console.log("arrayVar = ", arrayVar)
11 console.log("objectVar = ", objectVar)

```

```

numericVar = 10
boolVar = true
StringVar = Hello World!
arrayVar = Array(3)
  0: 1
  1: 5
  2: 8
  length: 3
objectVar = Object
  Name: "jack"
  family: "Scalia"
  Age: 7

```

در خطوط ۵-۱، متغیرها تعریف شده اند. اما نوع این متغیرها چیست؟ JavaScript نوع متغیرها را بسته به مقداری که به آنها اختصاص داده می شود در نظر می گیرد. مثال نوع متغیر StringVar در خط ۳ از نوع رشته است، چون یک مقدار رشته ای به آن اختصاص داده شده است. به خطوط ۴ و ۵ کد بالا توجه کنید. در خط ۴ یک متغیر تعریف شده است و نوع داده ای که به آن اختصاص داده شده است از نوع array است. همانطور که در درس قبل اشاره شد، برای تعریف array علامت [] به کار می رود و آیتم های داخل آن به وسیله کاما از هم جدا می شوند :

```
var arrayVar = [1,5,8]
```

در خط ۵ هم یک متغیر تعریف شده است و یک مقدار از نوع Object به آن اختصاص داده شده است. در تعریف Object به جای علامت [] از {} استفاده می شود. آیتم ها به صورت کلید/مقدار تعریف می شوند. بین کلید و مقدار علامت: و بین هر دو کلید/مقدار، هم علامت , قرار می گیرد.

```
var object Var = {Key1: Value1, Key2: Value2, Key3: Value3}
```

مثلا در مثال بالا یک Object تعریف کرده ایم که سه آیتم یا کلید/مقدار دارد که بین آن ها علامت کاما قرار داده ایم. ولی بین یک کلید و مقدار مربوط به آن علامت : قرار گرفته است. برای اختصاص یک مقدار به چند متغیر می توان به صورت زیر عمل کرد :

```
var identifier1 = identifier2 = ...indentifierN = Value
```

```
var num1 = num2 = num3 = num4 = num5 = 10
var message1 = message2 = message3 = "Hello World!"

console.log(num1)
console.log(num4)
console.log(message1)
console.log(message3)
```

```
10
10
Hello World!
Hello World!
```

دقت کنید که برای متغیرهای تعریف شده در حالت بالا یک خانه حافظه تخصیص داده می‌شود، یعنی مقدار ۱۰ در حافظه ذخیره شده و متغیرهای num1 و num2 و num3 و num4 و num5 به آن خانه از حافظه اشاره می‌کنند. همچنین می‌توان چند متغیر را تعریف کرد و برای هر یک از آن‌ها مقدار جداگانه‌ای مشخص نمود: یک متغیر جاوا اسکریپت می‌تواند هر نوع داده‌ای را در خود نگه دارد. توجه داشته باشید که جاوا اسکریپت یک زبان پویا و تایپ پویا است، بنابراین متغیرها می‌توانند در طول اجرای برنامه از یک نوع به نوع دیگر تغییر کنند.

```
var [num1, num2, message1] = [10, 12.5, "Hello World!"]
```

```
console.log(num1)
console.log(num2)
console.log(message1)
```

```
10
12.5
Hello World!
```

در کد بالا مقدار num1 برابر ۱۰ ، num2 برابر ۱۲.۵ و message1 برابر ! World Hello می‌باشد. در JavaScript متغیرها هم باید تعریف و هم مقداردهی شوند. یعنی اگر متغیری را تعریف کرده و به آن مقداری را اختصاص ندهید و برنامه را اجرا کنید با خطا مواجه می‌شوید :

```
var number
```

```
console.log(number)
```

```
Uncaught ReferenceError: number is not defined
```

یک رشته در اصل یک مجموعه از کاراکترهاست که در داخل علامت " " یا ' ' قرار دارند. هرکدام از این کاراکترها دارای یک اندیس است که به وسیله آن اندیس قابل دسترسی هستند. اندیس کاراکترها در رشته از ۰ شروع می‌شود.

```
var message = "Hello World!"
```

در رشته بالا اندیس کاراکتر O برابر ۴ است. برای درک بهتر به شکل زیر توجه کنید :

```
H e l l o   W o r l d !
0 1 2 3 4 5 6 7 8 9 10 11
```

حال برای چاپ یک کاراکتر (مثال W) از این رشته کافیست که به صورت زیر عمل کنیم :

```
var message = "Hello World! "  
console.log(message[6])
```

W

و اما در مورد Object، شما باید نام کلید را بنویسید تا مقدار آن برای شما نمایش داده شود:

```
var objectVar = { 'Name': 'jack', 'Family': 'Scalia', 'Age': 7 }  
console.log(objectVar['Family'])
```

Scalia

نکته‌ای که بهتر است در همین جا به آن اشاره کنیم این است که کلید/مقدارها در Object می‌توانند از هر نوعی باشند و شما برای چاپ مقدار مربوط به یک کلید باید نام کلید را دقیق بنویسید. به مثال زیر توجه کنید :

```
var objectVar = { 1: 'Jack', '2': 'Scalia', 3: 7 }  
console.log(objectVar['2'])
```

Scalia

در مثال بالا ما مقدار کلید '۲' را چاپ کرده‌ایم. حال اگر به جای '۲' عدد ۲ را بنویسیم، نتیجه همان است :

```
var objectVar = { 1: 'Jack', '2': 'Scalia', 3: 7 }  
console.log(objectVar[2])
```

Scalia

ثابت‌ها

اگر مقداردهی اولیه شوند، انواعی هستند که مقدار آنها در طول برنامه تغییر نمی‌کند. بعد از این که به ثابت‌ها مقدار اولیه اختصاص داده شد هرگز در زمان اجرای برنامه نمیتوان آن را تغییر داد. برای تعریف ثابت‌ها باید از کلمه کلیدی **const** استفاده کرد. معمولاً نام ثابت‌ها را طبق قرارداد با حروف بزرگ می‌نویسند تا تشخیص آنها در برنامه راحت باشد. نحوه تعریف ثابت در زیر آمده است :

```
const data_type identifier = initial_value;
```

```
const NUMBER = 1;
```

```
NUMBER = 10; //Uncaught TypeError: Assignment to constant variable.
```

در این مثال میبینید که مقدار دادن به یک ثابت، که قبلاً مقدار دهی شده برنامه را با خطا مواجه میکند. ممکن است این سؤال برایتان پیش آمده باشد که دلیل استفاده از ثابت‌ها چیست؟ اگر مطمئن هستید که مقادیری در برنامه وجود دارند که هرگز در طول برنامه تغییر نمیکنند بهتر است که آنها را به صورت ثابت تعریف کنید. این کار هر چند کوچک کیفیت برنامه شما را بالا میبرد.

تبدیل انواع داده

در زبان جاوااسکریپت امکان تبدیل یک نوع به نوع دیگر وجود دارد. این زبان دارای مجموعه‌ای از توابع از پیش تعریف شده است، که می‌توانند مقادیر را از یک نوع به نوع دیگر تبدیل کنند. جاوااسکریپت دو متد برای تبدیل انواع غیر عددی به عددی فراهم کرده است:

- parseInt()
- parseFloat()

توجه کنید که حروف F و I باید به صورت حرف بزرگ نوشته شوند.

این متدها فقط بر روی رشته‌های حاوی عدد کار می‌کنند و بر روی بقیه انواع مقدار NaN را بر می‌گردانند. متد parseInt() از اولین کاراکتر رشته شروع می‌کند اگر عدد بود آن را بر می‌گرداند در غیر این صورت مقدار NaN را بر می‌گرداند. این روند تا آخرین کاراکتر ادامه پیدا میکند تا اینکه به کاراکتری غیر عددی برسد. به مثال زیر توجه کنید:

```
console.log(parseInt("25Number"));
```

25

متد parseFloat() نیز همانند parseInt() عمل کرده و از اولین کاراکتر شروع به جستجو می‌کند. البته در این متد اولین کاراکتر نقطه حساب نمی‌شود و آن را به همان صورت می‌گرداند. به مثال زیر توجه کنید:

```
console.log(parseFloat("25.5Number"));  
console.log(parseFloat("25.45.2Number"));
```

```
25.5  
25.45
```

در جاوااسکریپت امکان استفاده از روشی موسوم به Casting Type برای تبدیل انواع وجود دارد. سه متد برای Casting Type وجود دارد:

- Boolean
- Number
- String

متد Boolean() زمانی مقدار true را بر میگرداند که پارامتر دریافتیش، رشتهای شامل حداقل یک کاراکتر، یک عدد غیر از صفر و یا یک شیء باشد. مقدار false را نیز زمانی بر میگرداند که پارامتر دریافتیش رشتهای تهی، عدد صفر یا یکی مقادیر null و undefined باشد:

```
var b1 = Boolean("");  
var b2 = Boolean("String");  
var b3 = Boolean(100);  
var b4 = Boolean(null);  
var b5 = Boolean(0);  
var b6 = Boolean(new Object());  
  
console.log(b1);  
console.log(b2);  
console.log(b3);  
console.log(b4);  
console.log(b5);  
console.log(b6);
```

```
false  
true  
true  
false  
false  
true
```

متد Number() کاری شبیه به متدهای parseInt() و parseFloat() انجام میدهد ولی تفاوت‌هایی هم با این دو متد دارد. اگر به یاد داشته باشید متدهای parseInt() و parseFloat() آرگومان دریافتی را فقط تا اولین کاراکتر بی ارزش بر میگردانند.

```
console.log(Number(false));
console.log(Number(true));
console.log(Number(undefined));
console.log(Number(null));
console.log(Number("5.5"));
console.log(Number("56"));
console.log(Number("5.6.7"));
console.log(Number(new Object()));
console.log(Number(100));
```

```
0
1
NaN
0
5.5
```

ساده‌ترین متد هم `String()` است که همان چیزی را که میگیرد به عنوان رشته بر میگرداند:

```
console.log(String(null));
```

```
null
```

در جاوا اسکریپت یک متد به نام `typeof()` وجود دارد که از آن برای تشخیص نوع متغیر استفاده می‌شود. به مثال زیر توجه کنید :

```
var value1 = 10.0;
var value2 = "Hello World!";

console.log(typeof (value1));
console.log(typeof (value2));
```

```
number
string
```

۳-۱۰ هر یک از این نوع‌های داده‌ای چگونه در حافظه تخصیص داده می‌شوند. هریک از این نوع‌ها چگونه پیاده‌سازی شده‌اند؟ هر یک از نوع‌ها چه ویژگی‌هایی دارند و در چه مواردی استفاده می‌شوند؟

بر خلاف زبان‌های سطح پایین مانند C، جاوا اسکریپت به طور خودکار حافظه را در هنگام ایجاد اشیاء اختصاص می‌دهد و زمانی که دیگر استفاده نمی‌شود آن را آزاد می‌کند (جمع‌آوری زباله). C دارای تکنیک‌های مدیریت حافظه مانند `malloc()` و `free()` است. این بدان معنا نیست که ما نیازی به نگرانی در مورد مدیریت حافظه در جاوا اسکریپت نداریم.

چرخه عمر حافظه: صرف نظر از زبان برنامه نویسی، چرخه عمر حافظه مراحل زیر را دنبال می‌کند:

۱. حافظه مورد نیاز ما را تخصیص می دهد: جاوا اسکریپت حافظه را به شی ایجاد شده اختصاص می دهد.

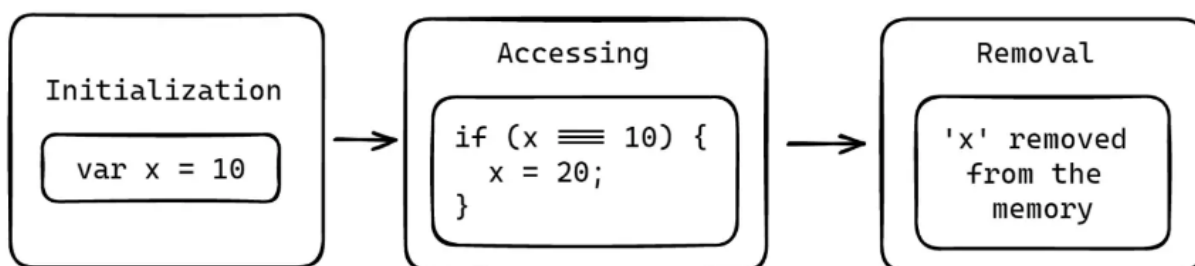
```
let name = 'codedamn';
```

۲. از حافظه اختصاص داده شده استفاده کنید.

```
if (name === 'codedamn') {  
  // Changing the value  
  name = 'Codedamn!';  
}
```

۳. زمانی که از حافظه استفاده نمی شود، حافظه را آزاد کنید: زمانی که حافظه اختصاص داده شده آزاد شد، برای مقاصد دیگر استفاده می شود. حذف، تخصیص حافظه رزرو شده توسط متغیرها است. پس از توزیع، نمی توانیم از کد جاوا اسکریپت خود به متغیر دسترسی پیدا کنیم. به عنوان مثال، ما نمی توانیم به متغیرهای ایجاد شده در داخل یک تابع دسترسی داشته باشیم.

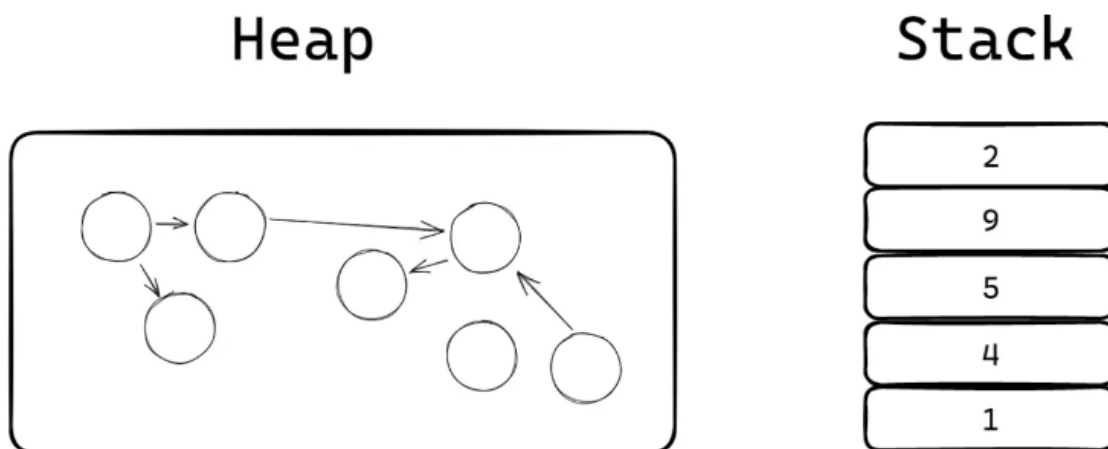
```
function sum() {  
  let x = 10;  
  let y = 20;  
  let sum = x + y;  
}  
  
console.log(x); // This will throw an error!
```



مرحله دوم برای همه زبان‌ها یکسان است. با این حال، مرحله اول و آخر در زبان‌های سطح بالا مانند جاوا اسکریپت به صورت ضمنی است. توجه: "اشیاء" در این زمینه نه تنها به معنی اشیاء در جاوا اسکریپت است، بلکه به معنی توابع و محدوده عملکرد نیز می‌باشد. در جاوا اسکریپت، تخصیص و مدیریت حافظه توسط موتور جاوا اسکریپت انجام می‌شود که بخشی از محیط زمان اجرا است که کد جاوا اسکریپت شما در آن اجرا می‌شود. مشخصات نحوه تخصیص انواع داده در حافظه ممکن است بین موتورهای مختلف جاوا اسکریپت متفاوت باشد. مقداردهی اولیه ایجاد متغیر است. در جاوا اسکریپت، ما این کار را با استفاده از کلمات کلیدی `let`، `const` یا `var` انجام می‌دهیم. در این مرحله، موتور جاوا اسکریپت حافظه را برای متغیرها ذخیره می‌کند و مقادیر را در فضای حافظه اختصاص داده شده ذخیره می‌کند.

جاوا اسکریپت چگونه متغیرها را ذخیره می‌کند؟

ما در مورد تخصیص و واگذاری صحبت کردیم، اما سوال اینجاست که این تخصیص‌ها و واگذاری‌ها کجا اتفاق می‌افتد؟

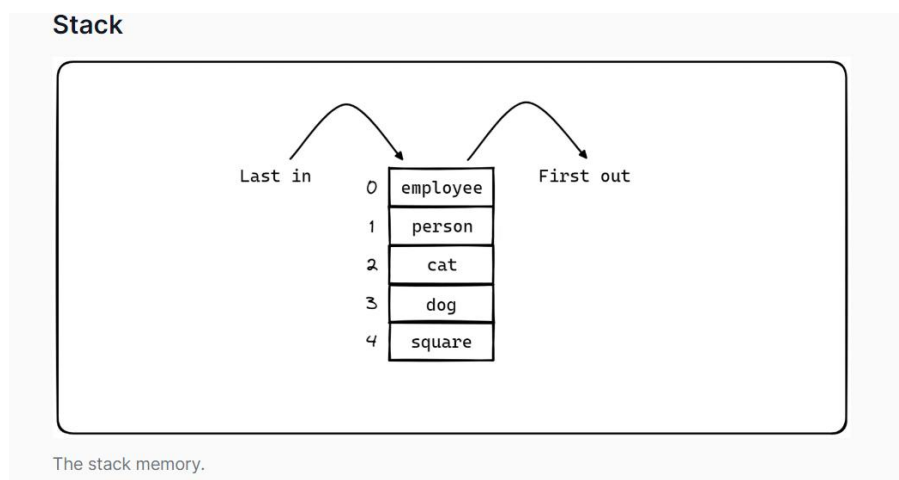


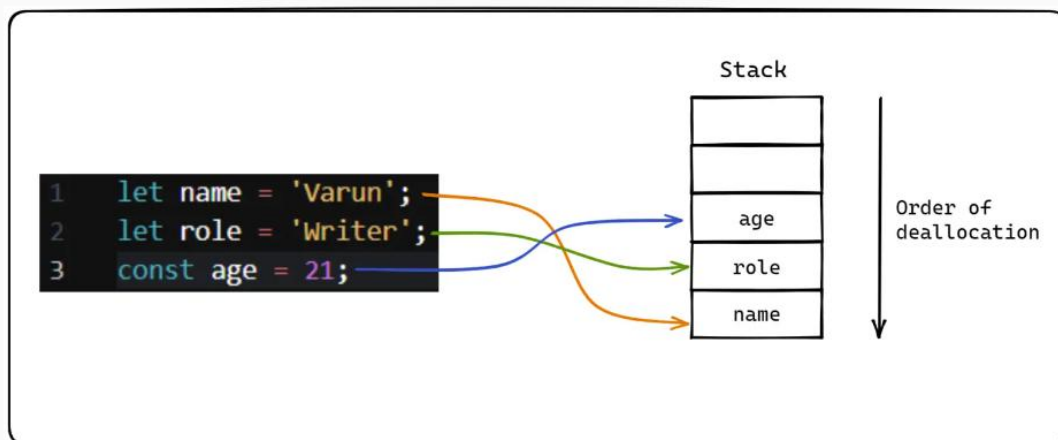
جاوا اسکریپت از دو نوع فضای حافظه یعنی `Stack` و `Heap` برای ذخیره متغیرها تشکیل شده است. تفاوت بین آن‌ها این است که چه متغیرهایی در کدام نوع ذخیره سازی ذخیره می‌شوند. بیایید نگاهی دقیق‌تر به پشته‌ها بیندازیم.

موتورهای جاوا اسکریپت دو مکان برای ذخیره داده ها دارند:

پشته: ساختار داده‌ای است که برای ذخیره داده‌های استاتیک استفاده می‌شود. داده‌های استاتیک به داده‌هایی اطلاق می‌شود که اندازه آنها توسط موتور در طول زمان کامپایل شناخته می‌شود. در جاوا اسکریپت، داده‌های ایستا شامل مقادیر ابتدایی مانند رشته‌ها، اعداد، بولی، تهی و تعریف نشده است. ارجاعاتی که به اشیا و توابع اشاره می‌کنند نیز گنجانده شده است. مقدار ثابتی از حافظه برای داده‌های استاتیک اختصاص داده شده است. این فرآیند به عنوان تخصیص حافظه ثابت شناخته می‌شود.

Heap: برای ذخیره اشیا و توابع در جاوا اسکریپت استفاده می‌شود. موتور مقدار ثابتی از حافظه را تخصیص نمی‌دهد. در عوض، فضای بیشتری را در صورت نیاز اختصاص می‌دهد.



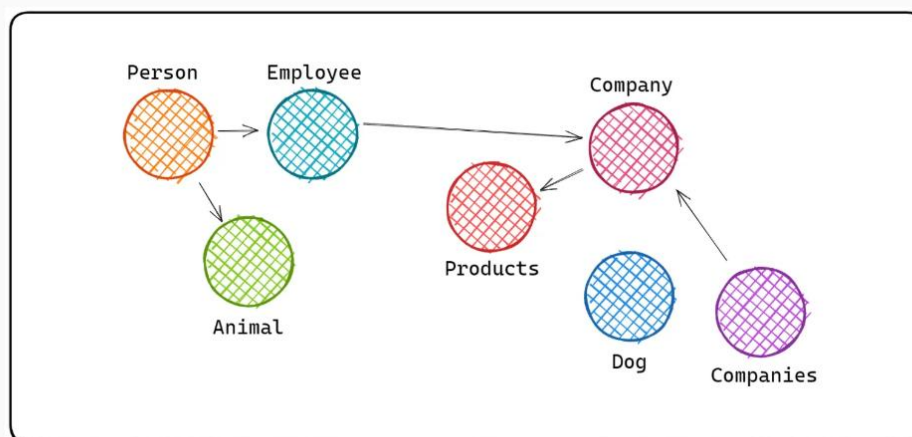


Allocation and deallocation of memory in the stack.

فرض کنید چند متغیر دارید که یکی یکی به پشته اضافه می‌کنید. آخرین متغیری که فشار داده شده در بالا است، به این معنی که وقتی حافظه تخصیص داده می‌شود، اولین متغیری است که حذف می‌شود.

جاوا اسکریپت از ساختار داده پشته برای ذخیره داده‌های ثابت استفاده می‌کند. این شامل تمام اعداد، رشته‌ها، بولی‌ها و سایر انواع داده‌های اولیه می‌شود. این نوع داده‌ها دارای اندازه ثابتی هستند که در زمان کامپایل شناخته می‌شوند. متغیرهایی مانند اشیاء، آرایه‌ها و غیره در پشته ذخیره نمی‌شوند زیرا اندازه آنها در طول زمان اجرا متفاوت است.

Heap



The heap memory.

از آنجایی که Stack فقط می تواند داده های ثابت یا ثابت را ذخیره کند، جاوا اسکریپت داده های پویا را در کجا ذخیره می کند؟ پاسخ Heap است. حافظه پشته، بر خلاف حافظه پشته، محدودیتی در اندازه ثابت ندارد، یعنی حافظه به صورت پویا تخصیص می یابد.

جاوا اسکریپت از یک پشته برای ذخیره متغیرهایی استفاده می کند که اندازه آنها در زمان کامپایل ناشناخته است یا ممکن است در زمان اجرا متفاوت باشد، مانند اشیاء، آرایه ها، توابع و غیره.

موتور جاوا اسکریپت به صورت پویا حافظه را به پشته اختصاص می دهد. در ابتدا، اندازه پشته به حافظه سیستم در دسترس بستگی دارد و می توان آن را به صورت پویا بر اساس نیاز افزایش/کاهش داد.

- **Primitives (Numbers, Strings, Booleans, Undefined, Null):** اولیه ها

مستقیماً در محل حافظه ای که متغیر به آن دسترسی دارد ذخیره می شود. آنها معمولاً انواع کوچک و ساده ای هستند که می توانند مستقیماً در مقدار ثابتی از حافظه ذخیره شوند. به عنوان مثال، یک عدد یا یک بولی در مقدار ثابتی از حافظه متناسب با اندازه آن نوع داده ذخیره می شود.

- **Objects (Objects, Arrays, Functions):** اشیاء ساختارهای داده پیچیده تری هستند

و معمولاً به عنوان مرجع در حافظه ذخیره می شوند. داده های واقعی برای یک شی در جای دیگری در حافظه ذخیره می شود و متغیر یک مرجع (آدرس حافظه) را به آن مکان نگه می دارد. این مرجع به اشیاء اجازه می دهد تا از نظر اندازه و محتوا انعطاف پذیرتر باشند.

- **Symbols:** نمادها منحصر به فرد و تغییر ناپذیر هستند و به روشی مشابه نمادهای اولیه ذخیره

می شوند. موتور جاوا اسکریپت تضمین می کند که هر مقدار نماد منحصر به فرد است، بنابراین می توان آن را مستقیماً در حافظه ذخیره کرد.

- **Functions:** توابع اشیایی در جاوا اسکریپت هستند و مشابه اشیاء ذخیره می شوند. کد تابع

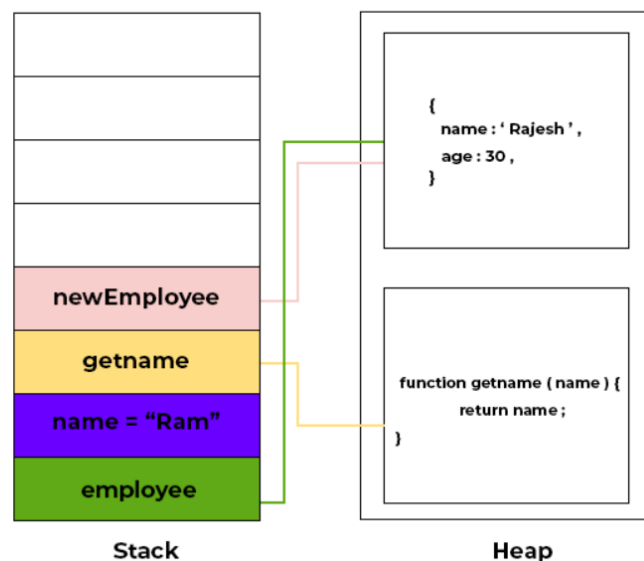
در حافظه ذخیره می شود و متغیر مرجعی به آن کد دارد.

Stack	Heap
Primitive data types and references	Objects and functions
Size is known at compile time	Size is known at run time
Fixed memory allocated	No limit for object memory

```
const employee = {
  name: 'Rajesh',
  age: 30,
};
const name = "Ram";

function getName ( name ) {
  return name;
}

const newEmployee = employee;
```



۱۱-۳ عبارات و عملگرها

ابتدا با دو کلمه آشنا شوید :

- عملگر: نمادهایی هستند که اعمال خاص انجام میدهند.
 - عملوند: مقادیری که عملگرها بر روی آنها عملی انجام میدهند.
- مثال $Y+X$: یک عبارت است که در آن X و Y عملوند و علامت $+$ عملگر به حساب میآیند.

زبان‌های برنامه نویسی جدید دارای عملگرهایی هستند که از اجزاء معمول زبان به حساب می‌آیند. جاوااسکریپت دارای عملگرهای مختلفی از جمله عملگرهای ریاضی، تخصیصی، مقایسه‌ای، منطقی و بیتی می‌باشد. از عملگرهای ساده ریاضی میتوان به عملگر جمع و تفریق اشاره کرد. سه نوع عملگر در جاوااسکریپت وجود دارد:

- یگانی (Unary) - به یک عملوند نیاز دارد
 - دودویی (Binary) - به دو عملوند نیاز دارد
 - سه تایی (Ternary) (به سه عملوند نیاز دارد)
- انواع مختلف عملگر که در این بخش مورد بحث قرار میگیرند، عبارتند از :

- عملگرهای ریاضی
- عملگرهای تخصیصی
- عملگرهای مقایسه‌ای
- عملگرهای منطقی
- عملگرهای بیتی

عملگرهای ریاضی

این زبان از عملگرهای ریاضی برای انجام محاسبات استفاده می کند. جدول زیر عملگرهای ریاضی JavaScript را نشان می دهد

عملگر	دسته	مثال	نتیجه
+	Binary	<code>var1 = var2 + var3;</code>	var1 برابر است با حاصل جمع var2 و var3

-	Binary	<code>var1 = var2 - var3;</code>	Var1 برابر است با حاصل تفریق var2 و var3
*	Binary	<code>var1 = var2 * var3;</code>	Var1 برابر است با حاصلضرب var2 در var3
**	Binary	<code>var1 = var2 ** var3;</code>	Var1 برابر است با var2 به توان var3
/	Binary	<code>var1 = var2 / var3;</code>	Var1 برابر است با حاصل تقسیم var2 بر var3
%	Binary	<code>var1 = var2 % var3;</code>	Var1 برابر است با باقیمانده تقسیم var2 و var3
+	Unary	<code>var1 = +var2;</code>	Var1 برابر است با مقدار var2
-	Unary	<code>var1 = -var2;</code>	Var1 برابر است با مقدار var2 ضربدر -1

دیگر عملگرهای JavaScript عملگرهای کاهش و افزایش هستند. این عملگرها مقدار ۱ را از متغیرها کم یا به آنها اضافه می کنند. از این متغیرها اغلب در حلقه ها استفاده می شود :

عملگر	دسته	مثال	نتیجه
++	Unary	<code>var1 = ++var2;</code>	مقدار var1 برابر است با var2 بعلاوه 1
--	Unary	<code>var1 = --var2;</code>	مقدار var1 برابر است با var2 منهای 1
++	Unary	<code>var1 = var2++;</code>	مقدار var1 برابر است با var2 به متغیر var2 یک واحد اضافه می شود
--	Unary	<code>var1 = var2--;</code>	مقدار var1 برابر است با var2 از متغیر var2 یک واحد کم می شود

به این نکته توجه داشته باشید که محل قرارگیری عملگر در نتیجه محاسبات تاثیر دارد. اگر عملگر قبل از متغیر var2 بیاید افزایش یا کاهش var1 اتفاق می افتد. چنانچه عملگرها بعد از متغیر var2 قرار بگیرند ابتدا var1 برابر var2 می شود و سپس متغیر var2 افزایش یا کاهش می یابد. به مثال های زیر توجه کنید :

```
x = 0;
y = 1;

x = ++y;

console.log("x = ", x);
console.log("y = ", y);
```

```
x = 2
```

```
y = 2
```

```
x = 0;
y = 1;

x = --y;

console.log("x = ", x);
console.log("y = ", y);
```

```
x=0
y=0
```

همانطور که در دو مثال بالا مشاهده می‌کنید، درج عملگرهای ++ و -- قبل از عملوند Y باعث می‌شود که ابتدا یک واحد از Y کم و یا یک واحد به Y اضافه شود و سپس نتیجه در عملوند X قرار بگیرد. حال به دو مثال زیر توجه کنید :

```
x = 0;
y = 1;

x = y--;

console.log("x = ", x);
console.log("y = ", y);
```

```
x = 1
y = 0
```

```
x = 0;
y = 1;

x = y++;

console.log("x = ", x);
console.log("y = ", y);
```

```
x = 1
y = 2
```

همانطور که در دو مثال بالا مشاهده می کنید، درج عملگرهای $++$ و $--$ بعد از عملوند y باعث می شود که ابتدا مقدار y در داخل متغیر x قرار بگیرد و سپس یک واحد از y کم و یا یک واحد به آن اضافه شود. حال می‌توانیم با ایجاد یک برنامه نحوه عملکرد عملگرهای ریاضی در جاوااسکریپت را یاد بگیریم :

```
1 //Variable declarations
2 var num1, num2;
3 var msg1, msg2;
4
5 //Assign test values
6 num1 = 5;
```

```
7 num2 = 3;
8
9 //Demonstrate use of mathematical operators
10 console.log("The sum of", num1, "and", num2, "is", (num1 + num2));
11 console.log("The difference of", num1, "and", num2, "is", (num1 - num2));
12 console.log("The product of", num1, "and", num2, "is", (num1 * num2));
13 console.log("The quotient of", num1, "and", num2, "is", (num1 / num2).toFixed(2));
14 console.log("The remainder of", num1, "divided by", num2, "is", (num1 % num2));
15 console.log("The result of", num1, "powered by", num2, "is", (num1 ** num2));
16
17 //Demonstrate concatenation on strings using the + operator
18 msg1 = "Hello ";
19 msg2 = "World!";
20 console.log(msg1 + msg2);
```

```
The sum of 5 and 3 is 8.
The difference of 5 and 3 is 2.
The product of 5 and 3 is 15.
The quotient of 5 and 3 is 1.67.
The remainder of 5 divided by 3 is 2
The result of 5 powered by 3 is 125
Hello World!
```

برنامه بالا نتیجه هر عبارت را نشان می‌دهد. در این مثال و در خط ۱۳ برای اینکه ارقام کسری بعد از عدد حاصل، دو رقم باشند، از متد `toPrecision` استفاده کرده ایم. عدد ۲ در این جا بدین معناست که عدد را تا دو رقم اعشار نمایش بده. در خط ۲۰ مشاهده میکنید که دو رشته به وسیله عملگر + به هم متصل شده‌اند. نتیجه استفاده از عملگر + برای چسباندن دو کلمه "Hello" و "World" رشته "Hello World" خواهد بود. به فاصله‌های خالی بعد از اولین کلمه توجه کنید اگر آنها را حذف کنید از خروجی برنامه نیز حذف میشوند.

عملگرهای تخصیصی (جایگزینی)

نوع دیگر از عملگرهای Javascript عملگرهای جایگزینی نام دارند. این عملگرها مقدار متغیر سمت راست خود را در متغیر سمت چپ قرار میدهند. جدول زیر انواع عملگرهای تخصیصی در Javascript را نشان میدهد:

عملگر	مثال	نتیجه
=	<code>var1 = var2;</code>	مقدار var1 برابر است با مقدار var2
+=	<code>var1 += var2;</code>	مقدار var1 برابر است با حاصل جمع var1 و var2
-=	<code>var1 -= var2;</code>	مقدار var1 برابر است با حاصل تفریق var1 و var2
*=	<code>var1 *= var2;</code>	مقدار var1 برابر است با حاصل ضرب var1 در var2
/=	<code>var1 /= var2;</code>	مقدار var1 برابر است با حاصل تقسیم var1 بر var2
%=	<code>var1 %= var2;</code>	مقدار var1 برابر است با باقیمانده تقسیم var1 بر var2

از عملگر += برای اتصال دو رشته نیز میتوان استفاده کرد. استفاده از این نوع عملگرها در واقع یک نوع خالصه نویسی در کد است. شکل اصلی کد `var2 += var1` به صورت `var2 + var1 = var2` میباشد. این حالت کدنویسی زمانی کارایی خود را نشان میدهد که نام متغیرها طولانی باشد. برنامه زیر چگونگی استفاده از عملگرهای تخصیصی و تأثیر آنها را بر متغیرها نشان میدهد:

```

var number;

console.log("Assigning 10 to number...");
number = 10;
console.log("Number = ", number);

console.log("Adding 10 to number...");
number += 10;
console.log("Number = ", number);

console.log("Subtracting 10 from number...");
number -= 10;

console.log("Number = ", number);

```

```

Assigning 10 to number...
Number = 10
Adding 10 to number...
Number = 20
Subtracting 10 from number...
Number = 10

```

در برنامه از ۳ عملگر تخصیصی استفاده شده است. ابتدا یک متغیر و مقدار ۱۰ با استفاده از عملگر = به آن اختصاص داده شده است. سپس به آن با استفاده از عملگر += مقدار ۱۰ اضافه شده است. و در آخر به وسیله عملگر -= عدد ۱۰ از آن کم شده است.

عملگرهای مقایسه‌ای

از عملگرهای مقایسه‌ای برای مقایسه مقادیر استفاده میشود. نتیجه این مقادیر یک مقدار بولی (منطقی) است. این عملگرها اگر نتیجه مقایسه دو مقدار درست باشد مقدار true و اگر نتیجه مقایسه اشتباه باشد مقدار false را نشان میدهند. این عملگرها به طور معمول در دستورات شرطی به کار میروند به این ترتیب که باعث ادامه یا توقف دستور شرطی میشوند. جدول زیر عملگرهای مقایسه‌ای در Javascript را نشان میدهد:

عملگر	مثال	نتیجه
==	var1 = var2 == var3	var1 در صورتی true است که مقدار var2 با مقدار var3 برابر باشد در غیر اینصورت false است

var1 در صورتی true است که مقدار var2 با مقدار var3 برابر نباشد در غیر اینصورت false است	var1 = var2 != var3	!=
var1 در صورتی true است که مقدار var2 کوچکتر از var3 مقدار باشد در غیر اینصورت false است	var1 = var2 < var3	<
var1 در صورتی true است که مقدار var2 بزرگتر از مقدار var3 باشد در غیر اینصورت false است	var1 = var2 > var3	>
var1 در صورتی true است که مقدار var2 کوچکتر یا مساوی مقدار var3 باشد در غیر اینصورت false است	var1 = var2 <= var3	<=
var1 در صورتی true است که مقدار var2 بزرگتر یا مساوی مقدار var3 باشد در غیر اینصورت false است	var1 = var2 >= var3	>=

برنامه زیر نحوه عملکرد این عملگرها را نشان میدهد :

```
var num1 = 10;
var num2 = 5;

console.log(num1 + " == " + num2 + " :", num1 == num2);
console.log(num1 + " != " + num2 + " :", num1 != num2);
console.log(num1 + " < " + num2 + " :", num1 < num2);
console.log(num1 + " > " + num2 + " :", num1 > num2);
console.log(num1 + " <= " + num2 + " :", num1 <= num2);
console.log(num1 + " >= " + num2 + " :", num1 >= num2);
```

```
10 == 5 : false
10 != 5 : true
10 < 5 : false
10 > 5 : true
10 <= 5 : false
10 >= 5 : true
```

در مثال بالا ابتدا دو متغیر را که می‌خواهیم با هم مقایسه کنیم را ایجاد کرده و به آنها مقادیری اختصاص می‌دهیم. سپس با استفاده از یک عملگر مقایسه‌ای آنها را با هم مقایسه کرده و نتیجه را چاپ می‌کنیم. به این نکته توجه کنید که هنگام مقایسه دو متغیر از عملگر == به جای عملگر = باید استفاده شود. عملگر = عملگر تخصیصی است و در عبارتی مانند $y = x$ مقدار y را در به x اختصاص می‌دهد. عملگر == عملگر مقایسه‌ای است که دو مقدار را با هم مقایسه میکند مانند $y == x$ و اینطور خوانده میشود x برابر است با y .

عملگرهای منطقی

عملگرهای منطقی بر روی عبارات منطقی عمل می‌کنند و نتیجه آنها نیز یک مقدار بولی است. از این عملگرها اغلب برای شرط های پیچیده استفاده می‌شود. همانطور که قبلا یاد گرفتیم مقادیر بولی می‌توانند false یا true باشند. فرض کنید که var2 و var3 دو مقدار بولی هستند.

عملگر	نام	مثال
&&	منطقی AND	var1 = var2 && var3;
	منطقی OR	var1 = var2 var3;
!	منطقی NOT	var1 = !var1;

عملگر منطقی (&&) And

اگر مقادیر دو طرف عملگر AND، true باشد، عملگر AND مقدار true را بر می‌گرداند. در غیر اینصورت اگر یکی از مقادیر یا هر دوی آنها false باشند مقدار false را بر می‌گرداند. در زیر جدول درستی عملگر AND نشان داده شده است :

X	Y	X && Y
true	true	true
true	false	false
false	true	false
false	false	false

برای درک بهتر تاثیر عملگر AND یاد آوری می‌کنم که این عملگر فقط در صورتی مقدار true را نشان می‌دهد که هر دو عملوند مقدارشان true باشد. در غیر اینصورت نتیجه تمام ترکیب های بعدی false خواهد شد. استفاده از عملگر AND مانند استفاده از عملگرهای مقایسه‌ای است. به عنوان مثال نتیجه عبارت زیر درست (true) (است اگر سن (age) (بزرگتر از ۱۸ و salary کوچکتر از ۱۰۰۰ باشد.

```
result = (age > 18) && (salary < 1000);
```

عملگر AND زمانی کارآمد است که ما با محدود خاصی از اعداد سر و کار داریم. مثال عبارت $x \leq 10$ بدین معنی است که x میتواند مقداری شامل اعداد ۱۰ تا ۱۰۰ را بگیرد. حال برای انتخاب اعداد خارج از این محدوده می توان از عملگر منطقی AND به صورت زیر استفاده کرد.

```
inRange = (number <= 10) && (number >= 100);
```

عملگر منطقی (||) or

اگر یکی یا هر دو مقدار دو طرف عملگر OR (درست) true باشد، عملگر OR مقدار true را بر می گرداند. جدول درستی عملگر OR در زیر نشان داده شده است :

X	Y	X Y
true	true	true
true	false	true
false	true	true
false	false	false

در جدول بالا مشاهده می کنید که عملگر OR در صورتی مقدار false را بر می گرداند که مقادیر دو طرف آن false باشند. کد زیر را در نظر بگیرید. نتیجه این کد در صورتی درست (true) است که رتبه نهایی دانش آموز (finalGrade) بزرگتر از ۷۵ یا یا نمره نهایی امتحان آن ۱۰۰ باشد.

```
isPassed = (finalGrade >= 75) || (finalExam == 100);
```

عملگر منطقی (!) Not

برخلاف دو اپراتور OR و AND عملگر منطقی NOT یک عملگر یگانی است و فقط به یک عملوند نیاز دارد. این عملگر یک مقدار یا اصطلاح بولی را نفی می کند. مثال اگر عبارت یا مقدار true باشد آنرا false و اگر false باشد آنرا true می کند. جدول زیر عملکرد اپراتور NOT را نشان می دهد :

عملگر	نام	مثال
&	بیتی AND	<code>x = y & z;</code>
	بیتی OR	<code>x = y z;</code>
^	بیتی XOR	<code>x = y ^ z;</code>
~	بیتی NOT	<code>x = ~y;</code>
&=	بیتی AND Assignment	<code>x &= y;</code>
=	بیتی OR Assignment	<code>x = y;</code>
^=	بیتی XOR Assignment	<code>x ^= y;</code>

عملگر بیتی And(&)

عملگر بیتی AND کاری شبیه عملگر منطقی AND انجام میدهد با این تفاوت که این عملگر بر روی بیتها کار میکند. اگر مقادیر دو طرف آن ۱ باشد مقدار ۱ را بر میگرداند و اگر یکی یا هر دو طرف آن صفر باشد مقدار صفر را بر میگرداند. جدول درستی عملگر بیتی AND در زیر آمده است:

X	Y	X AND Y
1	1	1
1	0	0
0	1	0
0	0	0

در زیر نحوه استفاده از عملگر بیتی AND آمده است :

```
var result = 5 & 3;
console.log(result);
```

```
1
```

همانطور که در مثال بالا مشاهده میکنید نتیجه عملکرد عملگر AND بر روی دو مقدار ۵ و ۳ عدد یک میشود. اجازه بدهید ببینیم که چطور این نتیجه را به دست میآید:

XOR(^) عملگر پیتی

جدول درستی این عملگر در زیر آمده است :

X	Y	X XOR Y
1	1	0
1	0	1
0	1	1
0	0	0

در صورتیکه عملوندهای دو طرف این عملگر هر دو صفر یا هر دو یک باشند نتیجه صفر در غیر اینصورت نتیجه یک میشود. در مثال زیر تأثیر عملگر بیتی XOR را بر روی دو مقدار مشاهده میکنید :

```
var result = 5 ^ 7;
```

```
console.log(result);
```

2

[illegible]

عملگر بیتی NOT(~)

این عملگر یک عملگر یگانی است و فقط به یک عملوند نیاز دارد. در زیر جدول درستی این عملگر آمده است:

	X	NOT X
1	1	0
0	0	1

```
7: 00000000000000000000000000000000111
-----
-8: 11111111111111111111111111111111000
```

مشاهده میکنید که همه بیتها به اندازه دو واحد به سمت چپ منتقل شده اند. در این انتقال دو صفر از صفرهای سمت چپ کم میشود و در عوض دو صفر به سمت راست اضافه میشود.

6

هر بیت به اندازه ۴ واحد به سمت راست منتقل میشود، بنابراین ۴ بیت اول سمت راست حذف شده و چهار صفر به سمت چپ اضافه میشود.

۱۲-۳ اگر لیست ها و رشته ها و آرایه های انجمنی در زبان تعریف شده اند، پیاده سازی آن ها چگونه است؟

۳-۱۲-۱ کار با آرایه ها

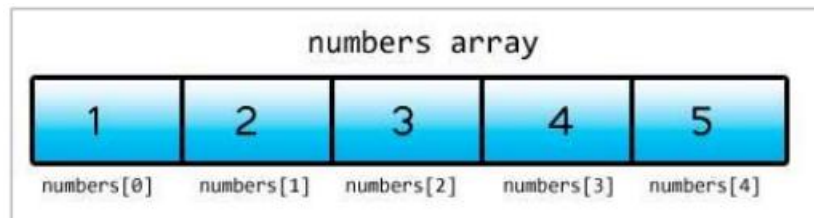
آرایه نوعی متغیر است که لیستی از آدرسهای مجموعه ای از داده های هممنوع یا غیر هممنوع را در خود ذخیره می کند. تعریف چندین متغیر از یک نوع برای هدفی یکسان بسیار خسته کننده است. مثال اگر بخواهید صد متغیر از نوع اعداد صحیح تعریف کرده و از آنها استفاده کنید. مطمئناً تعریف این همه متغیر بسیار کسالت آور و خسته کننده است. اما با استفاده از آرایه می توان همه آنها را در یک خط تعریف کرد. در زیر راهی ساده برای تعریف یک آرایه نشان داده شده است :

آرایه‌ها در جاوا اسکریپت یکی از مهمترین ساختارهای داده هستند. یک آرایه می‌تواند مجموعه‌ای از عناصر را در خود نگه دارد. بیا نگاهی به چند مفهوم اصلی در کار با آرایه‌ها در جاوا اسکریپت بیندازیم:

تعریف در آرایه: برای تعریف یک آرایه در جاوا اسکریپت از نماد `[]` استفاده می‌شود. مثلاً:

```
let numbers = [1, 2, 3, 4, 5];  
let fruits = ['Apple', 'Banana', 'Orange'];
```

اندیس یک آرایه از صفر شروع شده و به یک واحد کمتر از طول آرایه ختم می شود. به عنوان مثال شما یک آرایه ۵ عضوی دارید، اندیس آرایه از ۰ تا ۴ می باشد چون طول آرایه ۵ است پس ۵-۱ برابر است با ۴. این بدان معناست که اندیس ۰ نشان دهنده اولین عضو آرایه است و اندیس ۱ نشان دهنده دومین عضو و الی آخر. برای درک بهتر مثال بالا به شکل زیر توجه کنید:



دسترسی به عناصر:

عناصر آرایه با استفاده از ایندکس (شماره) دسترسی پیدا می کنند. در جاوا اسکریپت، ایندکس ها از صفر شروع می شوند. برای مثال:

```
console.log(numbers[0]); // خروجی: 1
console.log(fruits[1]); // خروجی: Banana
```

طول آرایه:

طول یک آرایه با استفاده از ویژگی length قابل دریافت است:

```
console.log(numbers.length); // خروجی: 5
console.log(fruits.length); // خروجی: 3
```

تغییر مقدار یک عنصر:

می توانید مقدار یک عنصر را با استفاده از ایندکس تغییر دهید:

```
numbers[2] = 10;
console.log(numbers); // خروجی: [5, 4, 10, 2, 1]
```

افزودن و حذف عناصر:

برای افزودن عناصر به آرایه از `push` استفاده می‌شود و برای حذف از `pop`. مثال:

```
fruits.push('Grapes');  
console.log(fruits); // خروجی: ['Apple', 'Banana', 'Orange', 'Grapes']  
  
fruits.pop();  
console.log(fruits); // خروجی: ['Apple', 'Banana', 'Orange']
```

حلقه‌ها و آرایه‌ها:

می‌توانید با استفاده از حلقه‌ها (مثل `for` یا `forEach`) از طول آرایه برای گردش و دسترسی به هر عنصر استفاده کنید:

```
for (let i = 0; i < numbers.length; i++) {  
  console.log(numbers[i]);  
}  
  
// یا با forEach  
numbers.forEach(function(number) {  
  console.log(number);  
});
```

یک روش دیگر برای تعریف آرایه استفاده از کلمه کلیدی `new` به صورت زیر است:

```
var numbers = new Array();
```

آرایه بالا را به صورت زیر هم می‌توان تعریف کرد:

```
var numbers = new Array(5);
```

```
numbers[0] = 1;  
numbers[1] = 2;  
numbers[2] = 3;  
numbers[3] = 4;  
numbers[4] = 5;
```

آرایه چند بعدی

آرایه‌های چند بعدی آرایه‌هایی هستند که برای دسترسی به هر یک از عناصر آنها باید از چندین اندیس استفاده کنیم. یک آرایه چند بعدی را میتوان مانند یک جدول با تعدادی ستون و ردیف تصور کنید. با افزایش اندیسها اندازه ابعاد آرایه نیز افزایش مییابد و آرایه‌های چند بعدی با بیش از دو اندیس به وجود می‌آیند. نحوه ایجاد یک آرایه با دو بعد به صورت زیر است :

```
var array_name = [[value1, value2, value3], [val1, val2, val3]];
```

می توان گفت که یک آرایه دو بعدی، خود آرایه ای از آرایه هاست. یعنی هر عنصر این نوع آرایه، خود یک آرایه است. آرایه دو بعدی رو می توان به صورت یک جدول تصور کرد که دارای سطر و ستون می باشد. در یک آرایه دو بعدی برای دسترسی به هر یک از عناصر به دو مقدار نیاز داریم، یکی اندیس سطر و دیگری اندیس ستونی که آن عنصر در آن قرار دارد. یک مثال از آرایه دو بعدی در زیر آمده است :

```
var numbers = [  
    [ 1, 2, 3, 4, 5],  
    [ 6, 7, 8, 9, 10],  
    [11, 12, 13, 14, 15]  
];
```

در کد بالا یک آرایه به نام numbers تعریف شده است که دارای سه عنصر است. البته هر کدام از این عناصر خود یک آرایه می باشند. یعنی آرایه numbers آرایه است از ۳ آرایه. هر کدام از این آرایه ها هم ۵ عنصر دارند. پس می توان گفت که آرایه numbers در واقع یک جدول با ۳ سطر و ۵ ستون می باشد. میتوان مقدار دهی به عناصر را به صورت دستی انجام داد مانند :

```
var numbers = [];  
  
numbers[0][0] = 1;  
numbers[0][1] = 2;  
numbers[0][2] = 3;  
numbers[0][3] = 4;  
numbers[0][4] = 5;  
numbers[1][0] = 6;  
numbers[1][1] = 7;  
numbers[1][2] = 8;  
numbers[1][3] = 9;  
numbers[1][4] = 10;  
numbers[2][0] = 11;  
numbers[2][1] = 12;  
numbers[2][2] = 13;  
numbers[2][3] = 14;  
numbers[2][4] = 15;
```

۳-۱۲-۲ لیست ها

لیست‌ها در جاوا اسکریپت به واقعیت با آرایه‌ها تطابق دارند. در واقع، تا ES6 (ECMAScript 2015)، جاوا اسکریپت لیست‌ها به نام "آرایه" دارد. با اضافه شدن ویژگی‌های جدید به جاوا اسکریپت، مانند Map و Set، امکان استفاده از ساختارهای داده متنوع‌تری نیز وجود دارد.

آرایه‌ها (Arrays):

همانطور که قبلاً گفتیم، آرایه‌ها در واقع لیست‌ها هستند:

```
let fruits = ['Apple', 'Banana', 'Orange'];
```

Map: یک ساختار داده‌ی کلید-مقدار است که می‌تواند به عنوان یک نوع لیست به حساب بیاید:

```
let fruitMap = new Map();  
fruitMap.set('A', 'Apple');  
fruitMap.set('B', 'Banana');  
fruitMap.set('O', 'Orange');
```

Set: یک مجموعه‌ی منحصر به فرد از مقادیر بدون تکرار است، معمولاً برای نگهداری لیست‌های متفاوت:

```
let uniqueFruits = new Set(['Apple', 'Banana', 'Orange']);
```

:Objects

اگرچه شیء‌ها در جاوا اسکریپت اصطلاحاً "لیست" نیستند، اما می‌توانند یک مجموعه از ویژگی‌ها (کلید-مقدار) را نگه دارند:

```
let fruitObject = {  
  A: 'Apple',  
  B: 'Banana',  
  O: 'Orange'  
};
```

NodeList: وقتی با DOM (Document Object Model) در جاوا اسکریپت کار می‌کنید، گاهی اوقات یک NodeList را به عنوان لیستی از المان‌ها می‌بینید:

```
let paragraphs = document.querySelectorAll('p');
```

۳-۱۲-۳ رشته‌ها

کار با رشته‌ها (String) در جاوا اسکریپت یکی از پایه‌ای‌ترین وظایف است. رشته‌ها در جاوا اسکریپت به صورت اشیاء (objects) نمایان می‌شوند، که معنایی آن است که شما می‌توانید به ویژگی‌ها و توابع مرتبط با رشته دسترسی داشته باشید.

تعریف رشته

```
let myString = 'Hello, World!';
```

طول رشته

```
console.log(myString.length); // خروجی: 13
```

دسترسی به کاراکترها

```
console.log(myString[0]); // خروجی: 'H'  
console.log(myString.charAt(7)); // خروجی: 'W'
```

اتصال رشته‌ها (Concatenation)

```
let firstName = 'John';  
let lastName = 'Doe';  
let fullName = firstName + ' ' + lastName;  
console.log(fullName); // خروجی: 'John Doe'
```

تغییر حالت حروف

```
console.log(myString.toUpperCase()); // خروجی: 'HELLO, WORLD!'  
console.log(myString.toLowerCase()); // خروجی: 'hello, world!'
```

جایگزینی زیررشته

```
let replacedString = myString.replace('World', 'Universe');  
console.log(replacedString); // خروجی: 'Hello, Universe!'
```

جدا کردن رشته

```
let words = myString.split(', '); // جدا کردن بر اساس ویرگول و فاصله  
console.log(words); // خروجی: ['Hello', 'World!']
```

جستجو در رشته

```
let position = myString.indexOf('World');  
console.log(position); // خروجی: 7
```

مقایسه رشته‌ها

```
let str1 = 'abc';  
let str2 = 'ABC';  
console.log(str1 === str2); // خروجی: false  
console.log(str1.toLowerCase() === str2.toLowerCase()); // خروجی: true
```

۳-۱۳ اگر اشاره گر‌ها و متغیرهای مرجع در زبان تعریف شده اند، پیاده سازی آن‌ها چگونه است؟
چه سازوکارهایی برای رفع مشکلات نشتی حافظه و اشاره گر معلق وجود دارد؟

در جاوا اسکریپت، اشاره گر‌ها به صورت مستقیم پشتیبانی نمی‌شوند، به این معنا که شما نمی‌توانید به طور مستقیم به یک مکان حافظه اشاره کنید یا از اشاره گر‌ها استفاده کنید مانند زبان‌هایی مانند C یا ++C. اما، با وجود عدم پشتیبانی از اشاره گر‌ها، شما همچنان با اشیاء (objects) و متغیرهای مرجع کار می‌کنید. هنگامی که یک متغیر به یک شیء یا آرایه اشاره می‌کند، شما در واقع یک نوع از "مرجع" را دارید.

```
let obj1 = { name: 'John' };  
let obj2 = obj1; // obj2 به همان شیء اشاره می‌کند که obj1  
  
console.log(obj1.name); // خروجی: John  
console.log(obj2.name); // خروجی: John  
  
obj1.name = 'Jane';  
console.log(obj2.name); // خروجی: Jane (زیرا هر دو به همان شیء اشاره می‌کنند)
```

در این مثال، هرچند ما اشاره گر‌ها را به صورت مستقیم نداریم، اما obj1 و obj2 هر دو به یک شیء اشاره می‌کنند، بنابراین تغییراتی که در یکی از آنها انجام می‌شود، بر روی دیگری نیز تأثیر می‌گذارد.
در جاوا اسکریپت، مشکلات نشتی حافظه به صورت مستقیم قابل رفع نیستند زیرا زبان‌هایی مانند C یا ++C که اشاره گر‌های دستی را مدیریت مستقیم حافظه ارائه می‌دهند، در جاوا اسکریپت وجود ندارند.

اما برخی از راهکارها و سازوکارهایی که می‌توانند کمک کننده باشند:

۱. استفاده از **Garbage Collection** جمع‌آوری زباله: جاوااسکریپت از یک مکانیزم جمع‌آوری زباله

برخوردار است که به طور خودکار اشیاء غیرقابل دسترس را تشخیص داده و از حافظه حذف می‌کند.

این به طور کلی مشکلات ناشی حافظه را کاهش می‌دهد.

۲. رفع مشکلات در کد: برنامه نویسی بهینه و پیشگیری از خطاهای مرتبط با مدیریت حافظه می‌تواند

کمک کننده باشد. استفاده از متغیرهای محلی به جای متغیرهای سراسری، آزمون دقیق کد، و

استفاده از الگوهای بهتر برنامه‌نویسی از جمله راهکارهایی هستند.

۳. استفاده از ابزارهای نظارتی: (**Monitoring Tools**) استفاده از ابزارهایی که مانیتورینگ برنامه و

مصرف حافظه را فراهم می‌کنند، می‌تواند کمک کند. این ابزارها می‌توانند نقاط ضعف و ناشی حافظه

را شناسایی کنند.

۴. استفاده از **Strict Mode**: استفاده از **Strict Mode** در جاوااسکریپت ممکن است باعث شناسایی

و اعلام خطاهای احتمالی در کد شود که می‌تواند کمک کند تا مشکلات ناشی حافظه به سرعت

شناسایی و رفع شوند.

۵. استفاده از اصول برنامه‌نویسی خوب: اجتناب از انجام عملیات‌های پیچیده و غیرضروری، استفاده از

متغیرهای محلی به جای متغیرهای سراسری، و به کارگیری اصول برنامه‌نویسی خوب می‌تواند کمک

کند.

همچنین، بسیاری از مشکلات ناشی حافظه در برنامه‌های جاوااسکریپت که در محیط مرورگر اجرا می‌شوند،

به نحو معقولی توسط مرورگرها مدیریت می‌شوند.

۳-۱۴ آیا بازیافت‌کننده حافظه وجود دارد و در صورت وجود چگونه پیاده‌سازی شده است. در

صورتی که بازیافت‌کننده حافظه وجود ندارد، زبان با یک زبان دیگر که بازیافت‌کننده حافظه دارد

مقایسه شود.

بله، بازیافت‌کننده حافظه (Memory Allocator) عموماً یک جزء از سیستم مدیریت حافظه است که

مسئول تخصیص و آزادسازی حافظه است. این مسئولیت شامل تخصیص بخش‌هایی از حافظه به برنامه‌ها

در زمان اجرا (تخصیص دینامیک حافظه) و آزادسازی حافظه که دیگر نیازی به آن نیست، می‌شود.

پیاده‌سازی بازیافت کننده حافظه در زبان‌ها و محیط‌های مختلف می‌تواند متفاوت باشد، اما در اصول، یک بازیافت کننده حافظه عموماً دارای دو عملکرد اصلی است:

تخصیص حافظه: (Allocation)

در زمان اجرا، بازیافت کننده حافظه مسئول تخصیص بخش‌هایی از حافظه به برنامه است. این تخصیص ممکن است به صورت استاتیک (مثلاً در زمان کامپایل) یا دینامیک (در زمان اجرا) انجام شود.

آزادسازی حافظه: (Deallocation)

زمانی که یک برنامه دیگر نیاز به یک بخش از حافظه ندارد، بازیافت کننده حافظه مسئول آزادسازی (رها کردن) آن بخش از حافظه است. این مرحله مهم است تا حافظه نه‌تنها تخصیص دهی شود بلکه همچنین قابلیت آزادسازی داشته باشد.

در زبان‌هایی مثل C و C++، بازیافت کننده حافظه به صورت مستقیم قابل دسترسی است و برنامه‌نویس مسئولیت تخصیص و آزادسازی حافظه را دارد. اما در زبان‌هایی مثل جاوا یا C#، این فرآیند به صورت خودکار توسط مکانیزم‌های جمع‌آوری زباله (Garbage Collection) انجام می‌شود.

بله، در جاوا اسکریپت نیز مکانیزم جمع‌آوری زباله (Garbage Collection) به عنوان بازیافت کننده حافظه وجود دارد. این مکانیزم به صورت خودکار اشیاء غیرقابل دسترس را شناسایی و حذف می‌کند. در جاوا اسکریپت، مرورگرها مسئول اجرای کد هستند و مکانیزم جمع‌آوری زباله در مرورگرها پیاده‌سازی می‌شود.

پیاده‌سازی جمع‌آوری زباله در مرورگرهای معروف مانند Chrome، Firefox، و Safari از الگوریتم‌های مختلفی استفاده می‌کند. به عنوان مثال:

۱. **V8 (Chrome):** مرورگر Chrome از موتور اجرای V8 استفاده می‌کند که یک مکانیزم

جمع‌آوری زباله پیشرفته دارد V8. از الگوریتم Generational Incremental Garbage

Collection برای بهبود عملکرد استفاده می‌کند.

۲. **Spider Monkey (Firefox):** مرورگر Firefox از موتور اجرای Spider Monkey استفاده

می‌کند که نیز از الگوریتم‌های جمع‌آوری زباله Generational GC برای بهینه‌سازی مصرف حافظه استفاده می‌کند.

۳. **JavaScript Core (Safari):** مرورگر Safari از موتور اجرای JavaScript Core یا

Nitro استفاده می‌کند که از الگوریتم‌های جمع‌آوری زباله به نام Write Barrier و GC و Thread-Assisted GC استفاده می‌کند.

پیاده‌سازی دقیق الگوریتم‌ها و رفتار جمع‌آوری زباله ممکن است بر اساس نسخه مرورگر و تغییرات در مدل‌های جدید مرورگرها متغیر باشد.

منابع:

- [1] K. Simpson, "You Don't Know JS (book series)," O'Reilly Media, 2014.
- [2] D. Crockford, "JavaScript: The Good Parts," O'Reilly Media, 2008.
- [3] M. Haverbeke, "Eloquent JavaScript," No Starch Press, 2018.
- [4] D. Flanagan, "JavaScript: The Definitive Guide," O'Reilly Media, 2020.
- [5] M. Casciaro and L. Mammino, "Node.js Design Patterns," Packt Publishing, 2014.

تشکر از توجه شما.....